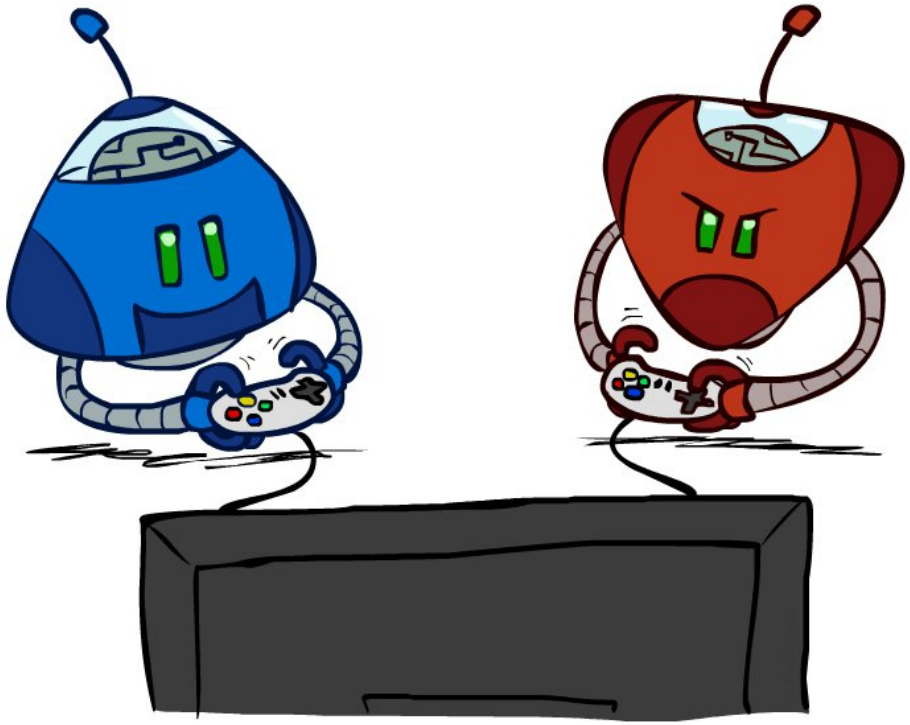
Adversarial Search

MINIMAX Search



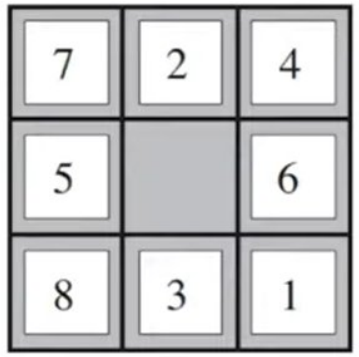
Adversarial Search

- When google maps finds a route, does the road try to stop you?
- What if someone is actively trying to stop you from reaching your goal?
- Previously, environments never opposed us, now the environment contains another intelligent agent whose objective conflicts with ours.
- This creates adversarial search problem

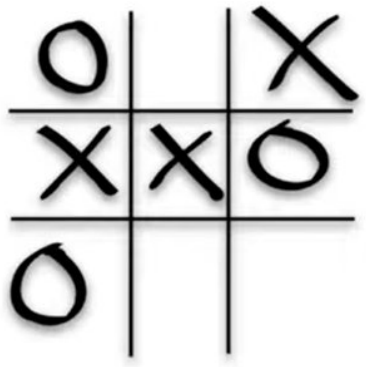
Adversarial Search

- Games are good examples of adversarial search.
  - States are easy to represent (unlike many other real world problems)
  - Agents are restricted to a small number of actions
  - Outcome of agent actions defined by precise rules
- If your opponent suddenly changes strategy, do you continue playing the same way?

Adversarial Search







Which one/s are adversarial search?

- Which of the above games require an opponent?
- Do you change your strategy according to opponents move?
- The games that have an interference of an opponent are adversarial games
- In tic tac toe our search is to basically achieve all three same moves in any direction (vertical, horizontal or diagonal) but this search is adversarial because the opponent interferes with our moves

MAEs and Games

- **Multi-agent environment**: every agent needs to consider the actions of the other agents, in order to optimize its own welfare

Cooperative Environment	**Competitive Environment**
Goal: Same	**Goal:** Opposite
Agents help each other	Agents try to defeat each other
Everyone succeeds together	One player's success is another's failure
Minecraft Survival	Chess
Relay Race	Tic-Tac-Toe
Collaboration	Adversarial Search

MAE and Games

- Is Cricket cooperative or competitive?
- Multiple agents playing in one team, so competitive or cooperative when considering players within a team?
- Multiple agents playing in opposite teams team, so competitive or cooperative when considering players in opposite teams?

MAEs and Games

- **Multi-agent environment**: every agent needs to consider the actions of the other agents, in order to optimize its own welfare

  - **Cooperative**:
    - Agents act collectively to achieve a common goal
    - **Minecraft (Multiplayer Survival)**
      Players cooperate to build, explore, and survive.
    - **Relay Race (Same Team)**
    - Each student runs one part of the race
    - Success depends on **everyone completing their part properly**
    - **Concept:** Sequential cooperation (each agent contributes)

  - **Competitive**:
    - Agents compete against each other
    - Their goals are in conflict
    - Gives rise to the concept of **adversarial** search problems – often known as **games**.

Games vs. Search problems

- In typical search problems, we optimize a measure to acquire the goal: there is no opponent

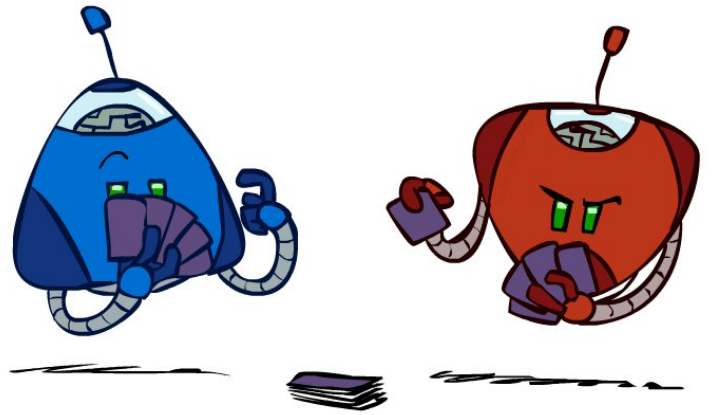
- In games, there is an "Unpredictable" opponent
  - Need to specify a move for every possible opponent reply
    Strict penalty on an inefficient move
  - Stringent time constraints
  - Unlikely to find goal, must approximate
  - Requires some type of a decision to move the search forward.

Types of Games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information	battleships, blind tictactoe	bridge, poker, scrabble nuclear war

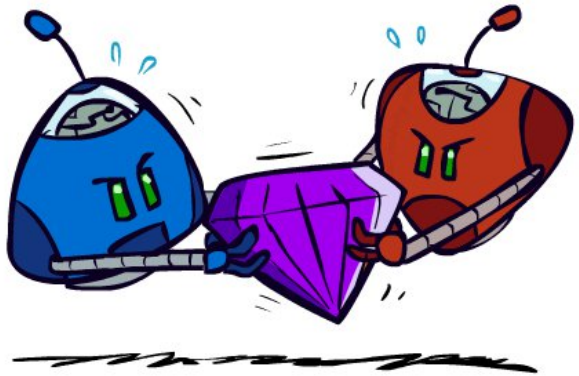
- Many different kinds of games!

- Axes:
  - Deterministic or stochastic?
    - **Deterministic:** No randomness; outcome depends only on player actions . Same moves, Same result
    - Example: Chess
    - **Stochastic:** Involves randomness (dice, cards)
    - Example: Ludo, Poker
  - One, two, or more players?
  - Zero sum?
    - **Zero-sum:** One player's win = other's loss
    - Example: Tic-Tac-Toe
    - **Non-zero-sum: Players can win or lose together (cooperation possible)**
  - Perfect information (can you see the state)?
    - **Perfect information:** All players can see the full game state
    - Example: Chess, Tic tac toe
    - **Imperfect information:** Some information is hidden
    - Example: Poker, hidden cards



- Want algorithms for calculating a **strategy (policy)** which recommends a move from each state

Zero-Sum Games





- Zero-Sum Games
  - Agents have opposite utilities (values on outcomes)
  - Lets us think of a single value that one maximizes and the other minimizes
  - Adversarial, pure competition

- General Games
  - Agents have independent utilities (values on outcomes)
  - Cooperation, indifference, competition, and more are all possible

Game Formulation

- **States**: S (start at Initial State s_0)
  - Every board configuration
- **Players**: P={1…N} (usually take turns)
- **Actions**: A (may depend on player / state)
  - Legal moves
- **Transition Function**: SxA S
  - Current state X action performed new state
- **Terminal State Test**: S {true,false}
  - Is the game finished?
- **Terminal Utilities**: State (S) x Player (P) R
  - Defines the final numeric value for a game that ends in terminal state (s) for player (p)
  - For example in tic tac toe the outcome is win, draw or loss with the values 1,0,-1 (For max the winning outcome is 1 and for min the winning outcome is -1)

Basic Strategy

- Grow a search tree – and examines enough nodes to allow a player to determine what move to make.
- Only one player can make move at each turn.
- Assume we can assign a payoff (utility value) at final position.
- Assume opponent always makes moves worst for us.
- Pick best move on own turn.

Utility function

- For Tic-Tac-Toe, the function could be as simple as returning:
  - +1/ +C (+10); it means that MAX wins
  - -1/ -C (-10); it means that MIN wins
  - 0 otherwise.

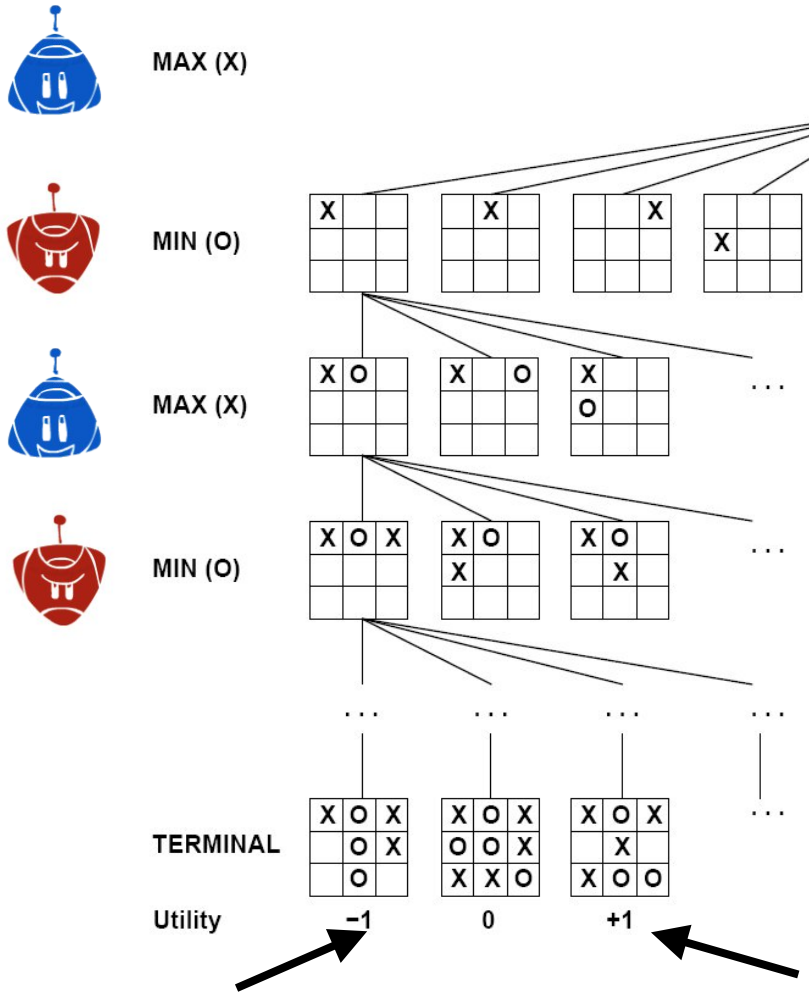
- However, **this simple evaluation function may require deeper search**

- Complete tree needs to be generated to calculate moves
  - May be slow/infeasible in some cases!

Optimal Strategy For MAX

- This is a Multi Agent Environment
  - In discovering the sequence of optimal moves by MAX, we have to consider that MIN is taking moves as well
  - So, the optimal strategy will tell how should MAX play against MIN (by considering the moves of the latter), in order to win the game
    - Strategy for MAX
      - Specify moves in the initial state
      - Observe every response by MIN
      - Specify moves in response
      - Repeat until terminal state
- Let us consider a shortened game tree of TIC-TAC-TOE
  - Numbers are the utilities
  - Ply: a move by MAX followed by a move from MIN.

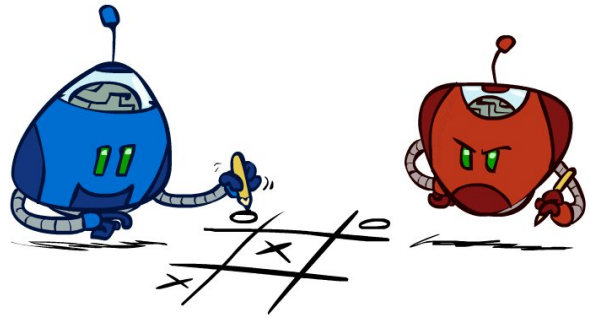
Tic-Tac-Toe Game Tree



Max has 9 possible moves

Min has 8 possible move for each of Max's move
8 X 9 Moves at this state

Max has 7 possible move for each of Min's move
7 X (8 X 9) Moves at this state



This terminal state is one of the worst for MAX and one of the best for MIN

This terminal state is one of the best for MAX and one of the worst for MIN

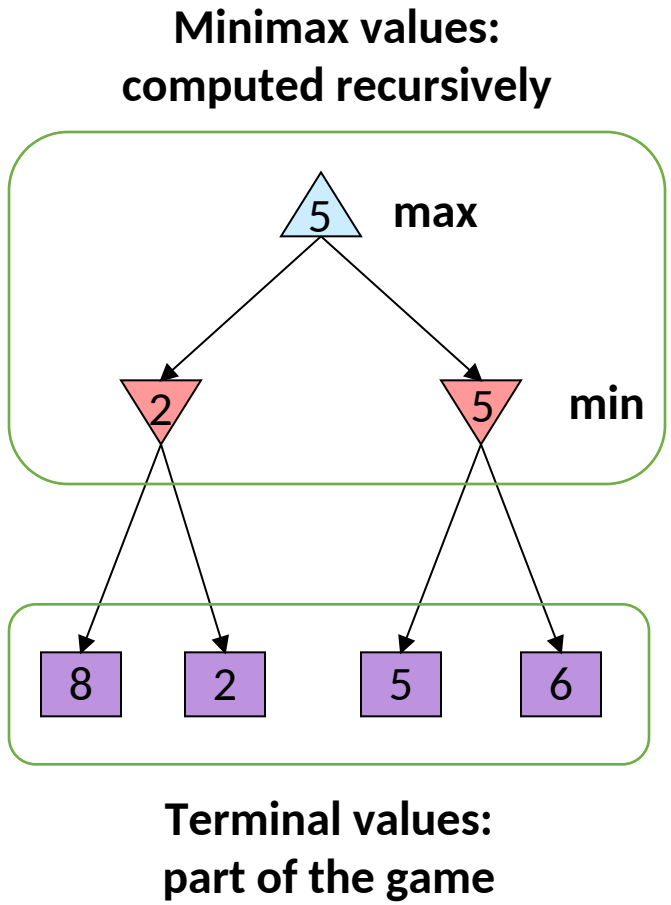
Minimax

- When it is the turn of MAX, it will always take an action in order to **maximize** its utility, because it's winning configurations have high utilities
- When it is the turn of MIN, it will always take an action in order to **minimize** its utility, because it's winning configurations have low utilities
- In order to implement this, we need to define a measure in each state that takes the move of the opponent into account.
- The term "**MINIMAX**" is used because:
  - the opponent is always trying to **minimize** the utility of the player, and
  - the player is always trying to **maximize** this minimized selection of the opponent.

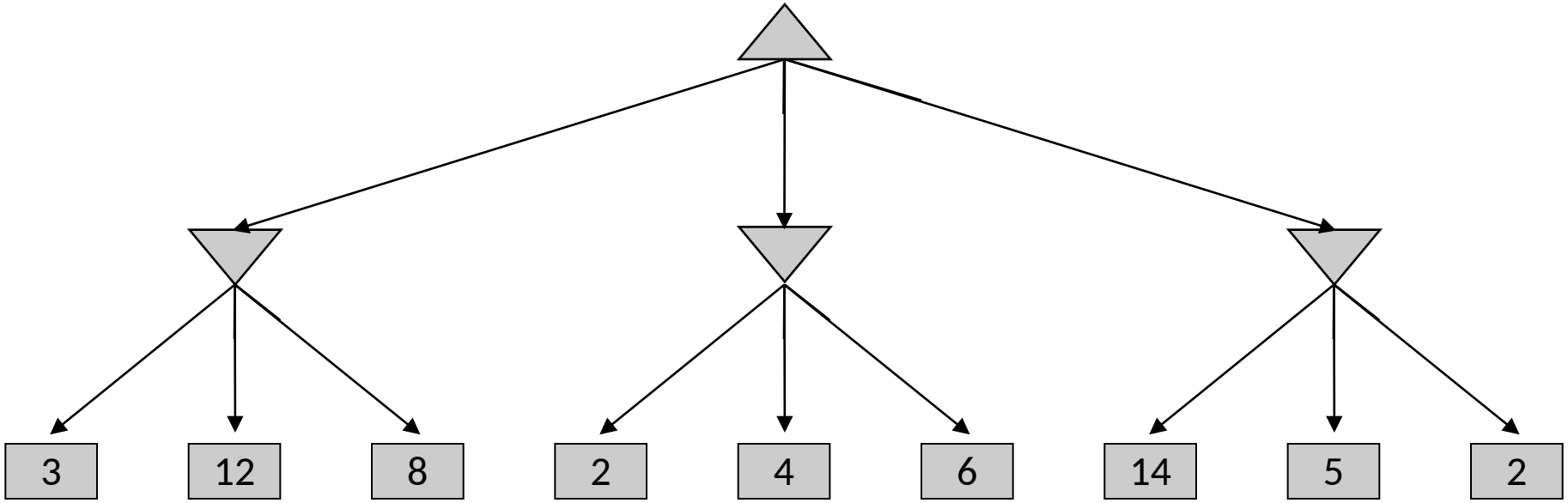
Adversarial Search (Minimax)

- Deterministic, zero-sum games:
  - Tic-tac-toe, chess, checkers
  - One player maximizes result
  - The other minimizes result

- Minimax search:
  - Backtracking algo
  - A state-space search tree
  - Players alternate turns
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary

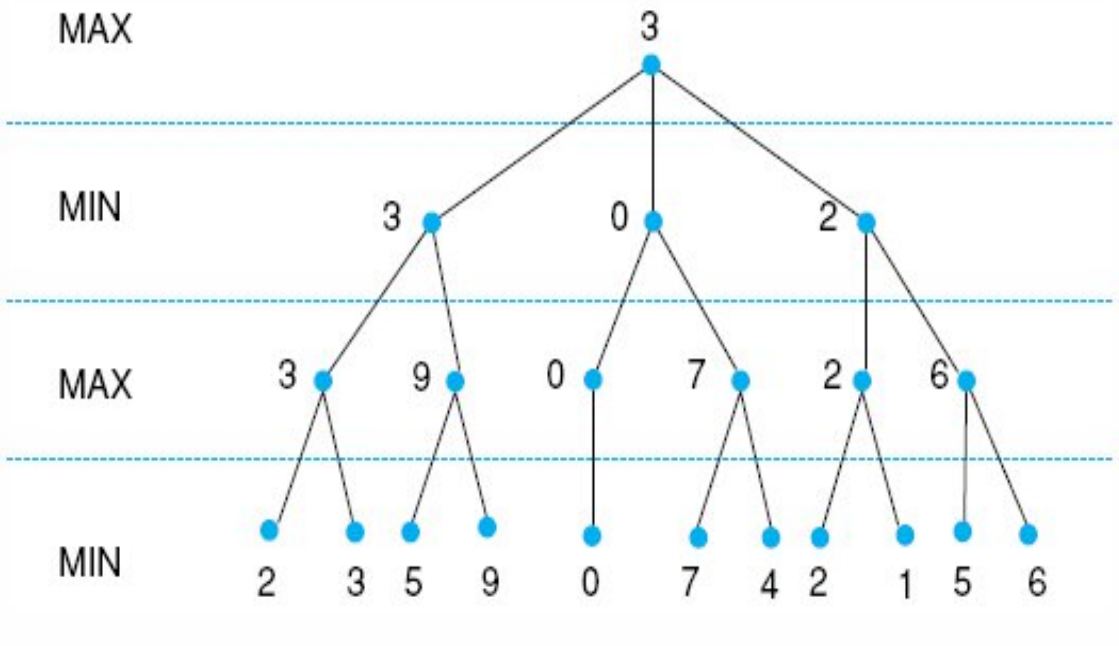


Minimax Example



- At each node, **MAX** will always select the action with **highest minimax value** (it wants to reach states with higher utilities)
- At each node, **MIN** will always select the action with **lowest minimax** value (it wants to reach states with lower utilities).

Minimax to a hypothetical state space. Leaf states show heuristic values; internal states show backed-up values.



$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$

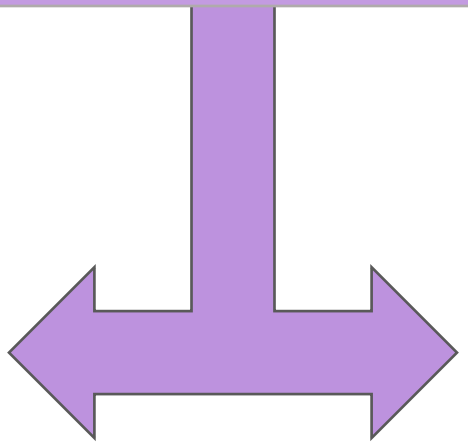
Recursive Function

Takes state s as input and returns a value that tells how good or bad that state is

- If terminal state return utility of state
- If player is max and state is not terminal then output of minimax function is the maximum of all possible actions after applying minimax to that node
- If player is min and state is not terminal then output of minimax function is the minimum of all possible actions after applying minimax to that node

Minimax Implementation (Dispatch)

```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

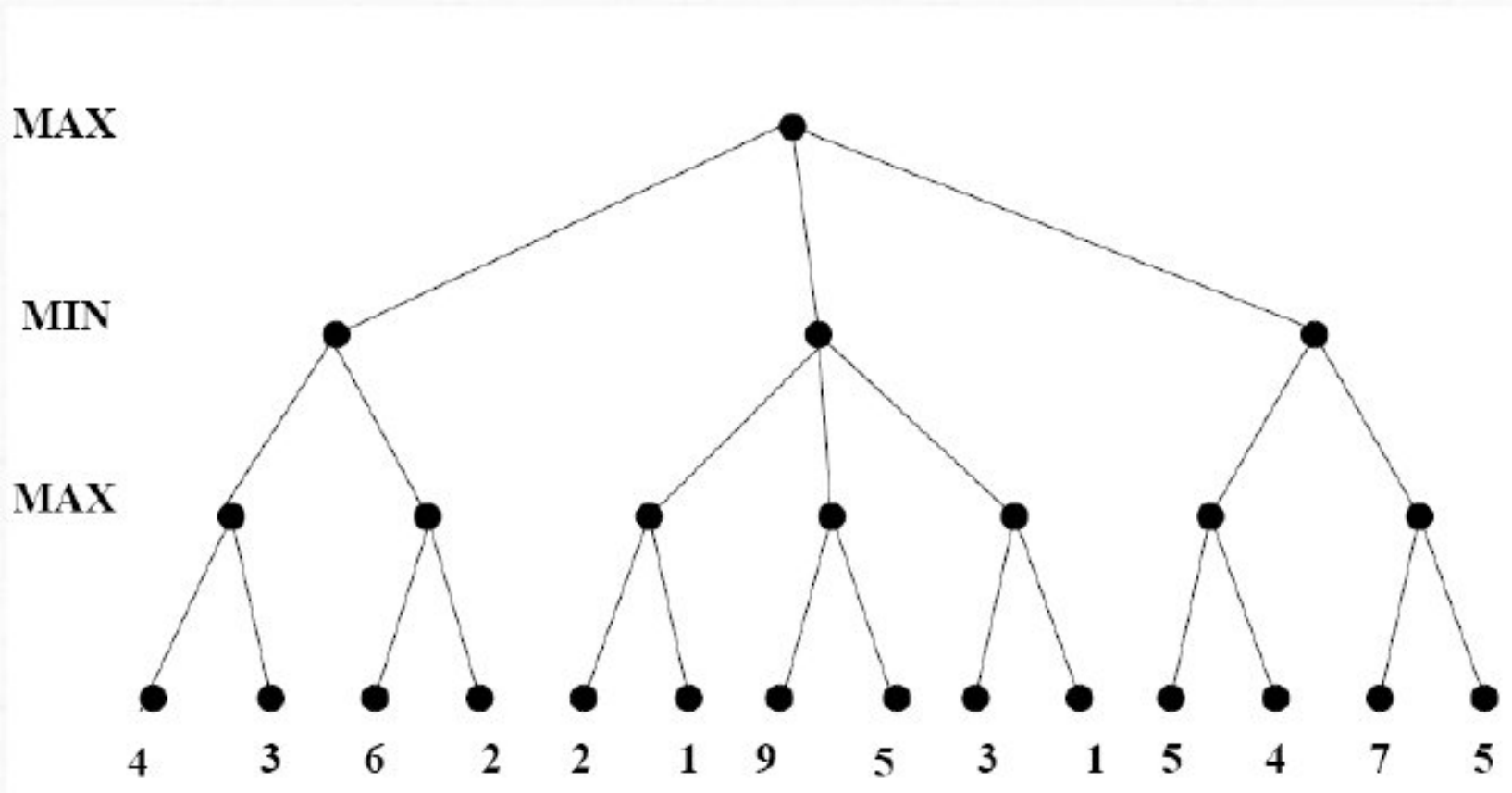


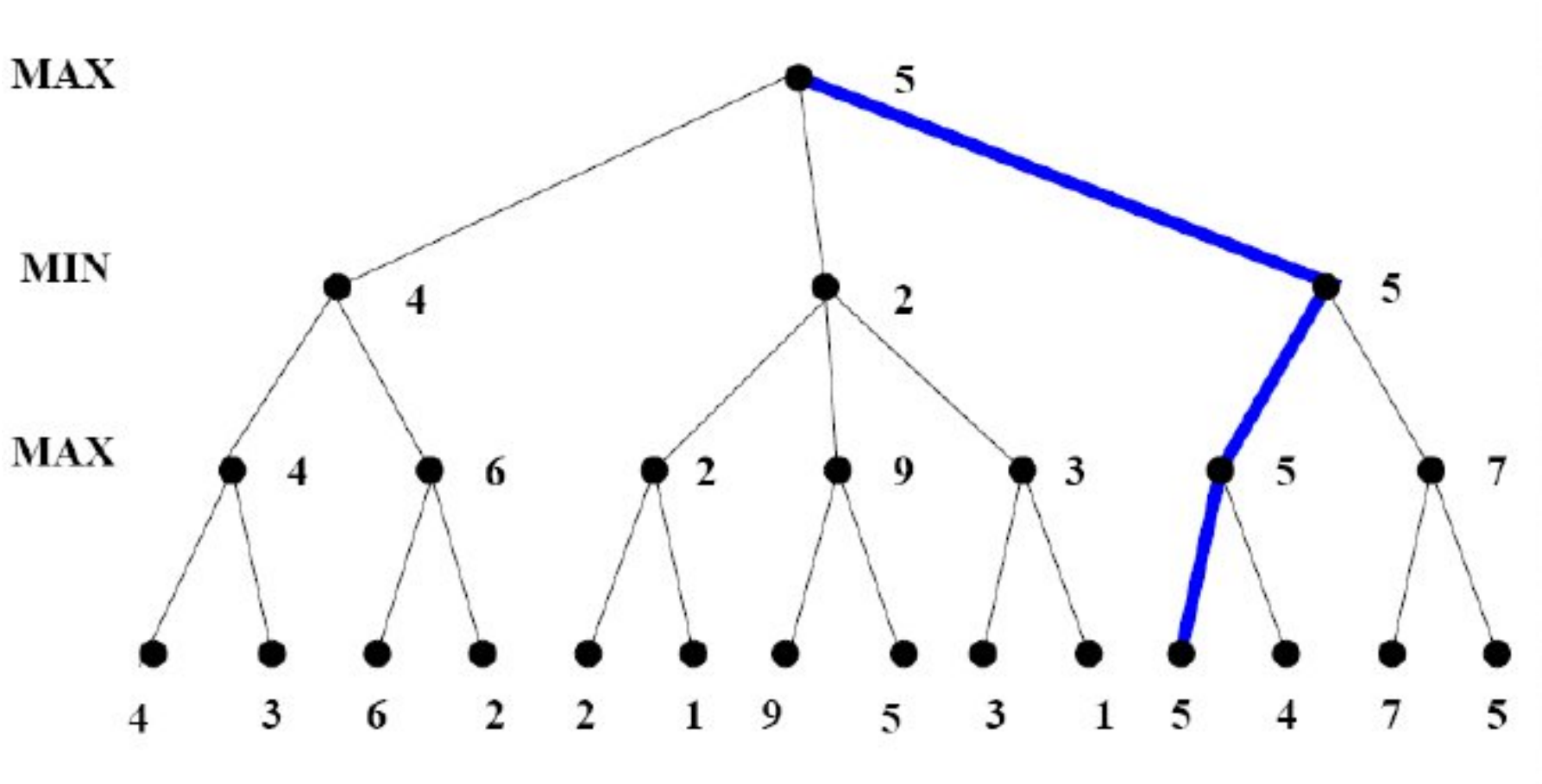
```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

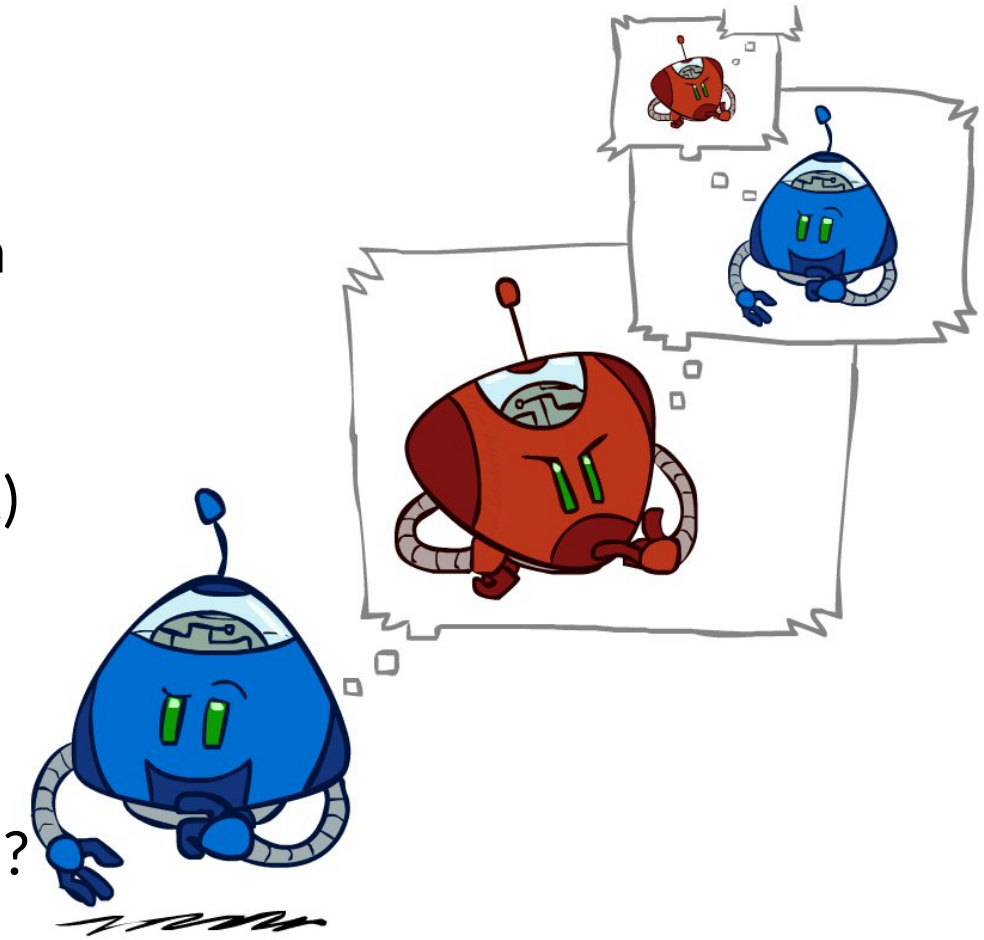

MAX
MIN
MAX
4
3
6
2
2
1
9
5
3
1
5
4
7
5


MAX
MIN
MAX
5
4
2
5
4
6
2
9
3
5
7
4
3
6
2
2
1
9
5
3
1
5
4
7
5

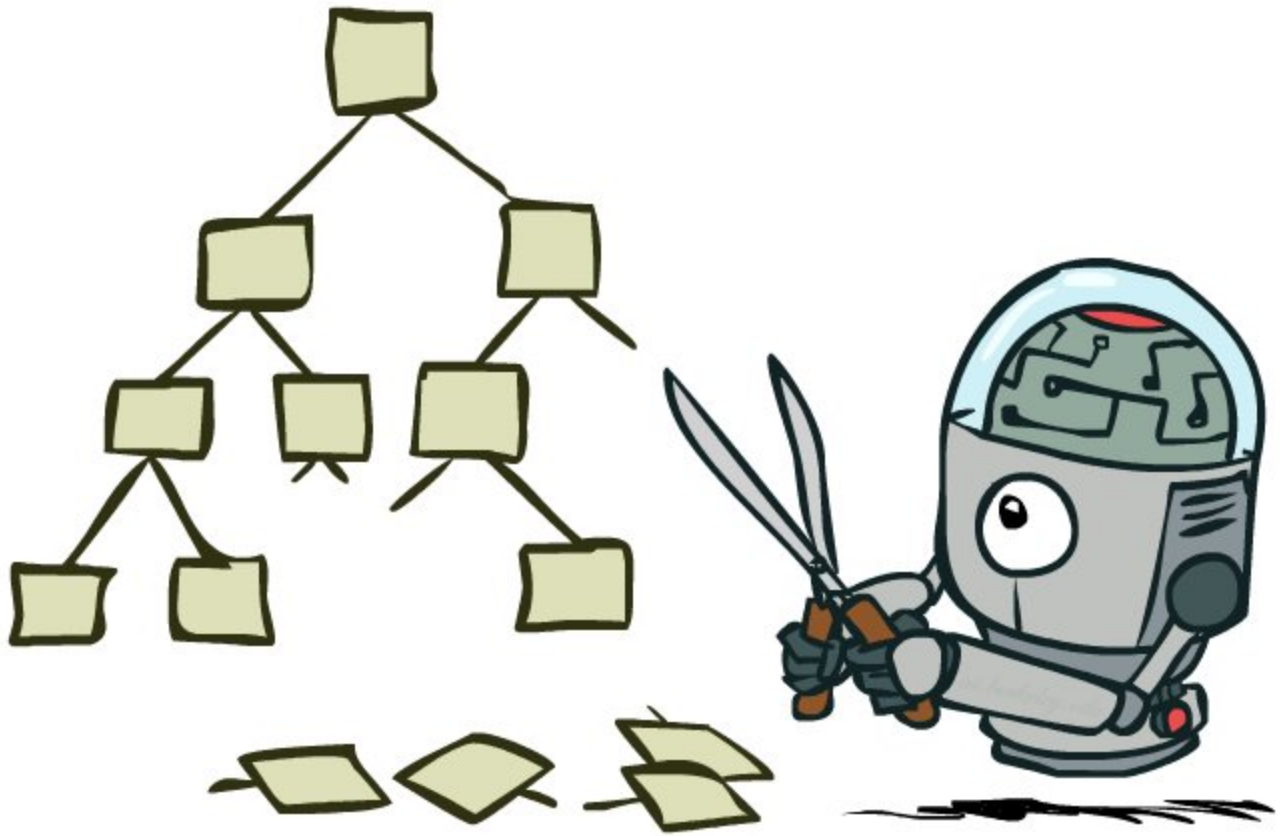
Minimax Efficiency

- How efficient is minimax?
  - Just like (exhaustive) DFS
  - Time: $O(b^d)$ b = branching factor, d=depth
  - Space: O(bd)
  - Complete: **Yes** (if tree is finite)
  - Optimal? **Yes** (against an optimal opponent)

- Example: For chess, b 35, m 100
  - Exact solution is completely infeasible
  - But, do we need to explore the whole tree?



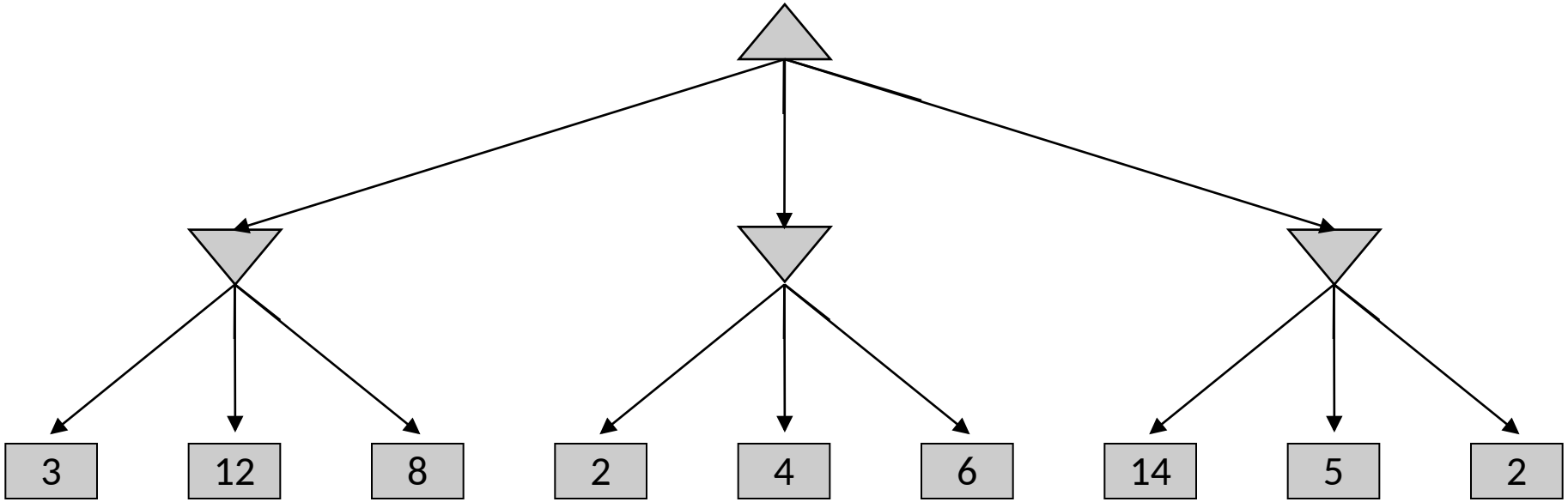
Game Tree Pruning



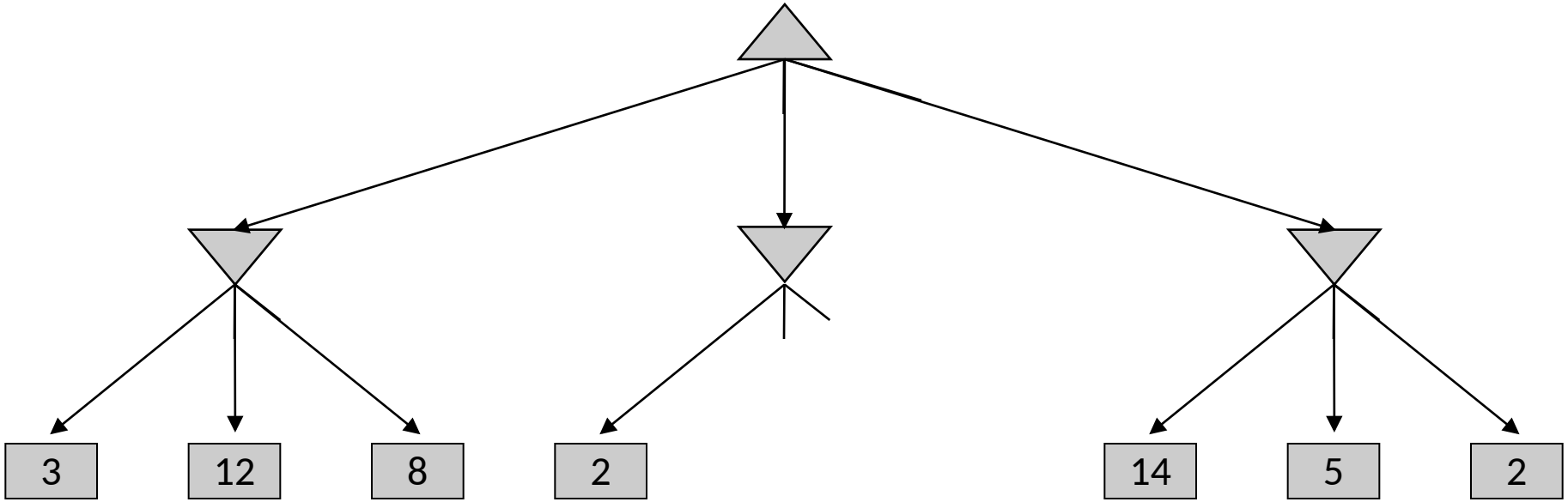
Minimax Search

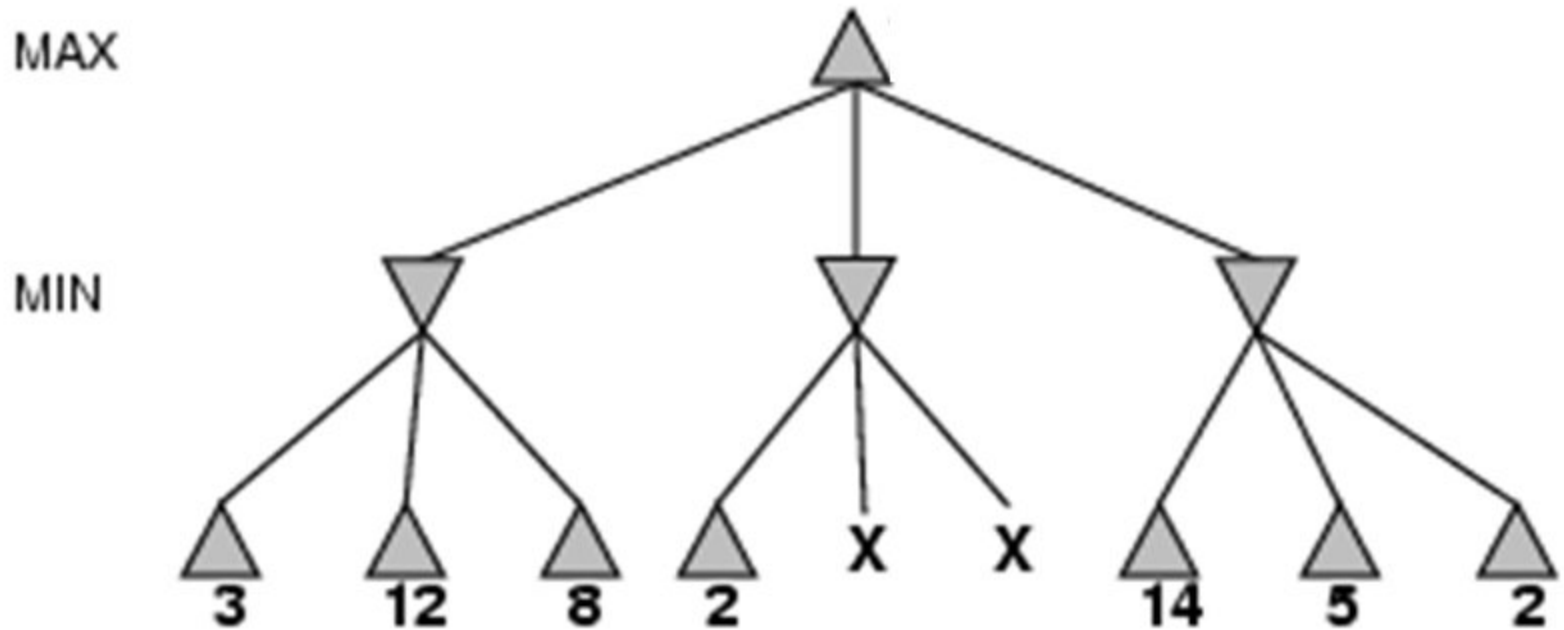
- Straight minimax requires a two-pass analysis of the search space,
  - the first to descend to the level depth and
  - the second to propagate values back up the tree.
- Minimax pursues all branches in the space, including many that could be ignored or pruned by a more intelligent algorithm.
- Recursion: Winds all the way to the terminal nodes, and then unwinds back by backing up the values

Minimax Example



Minimax Pruning



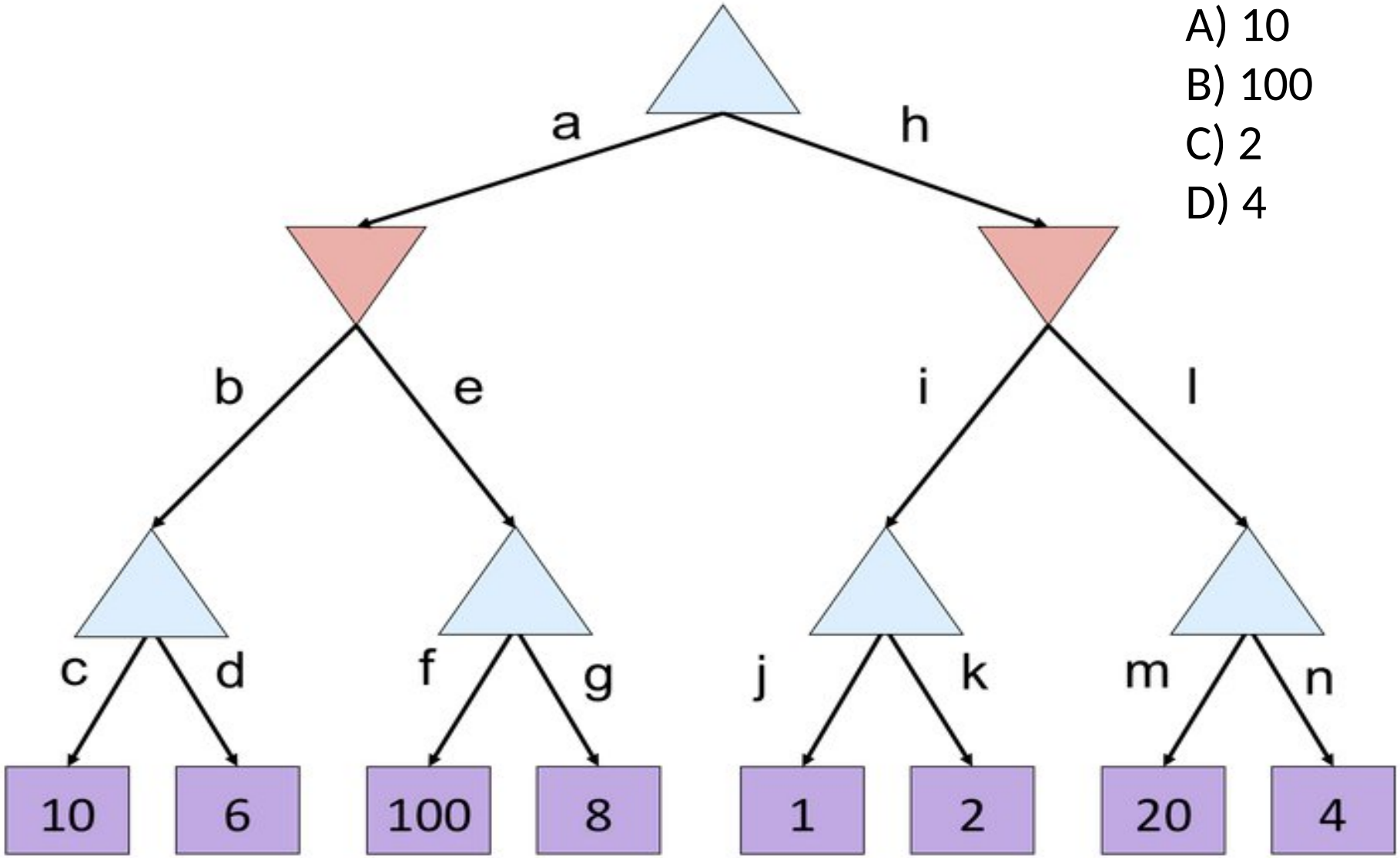


$$
\begin{aligned}
\textsc{Minimax}(root) &= \max(\min(3,12,8), \min(2,x,y), \min(14,5,2)) \\
&= \max(3, \min(2,x,y), 2) \\
&= \max(3,z,2) \qquad \text{where } z = \min(2,x,y) \leq 2 \\
&= 3.
\end{aligned}
$$

Minimax Quiz

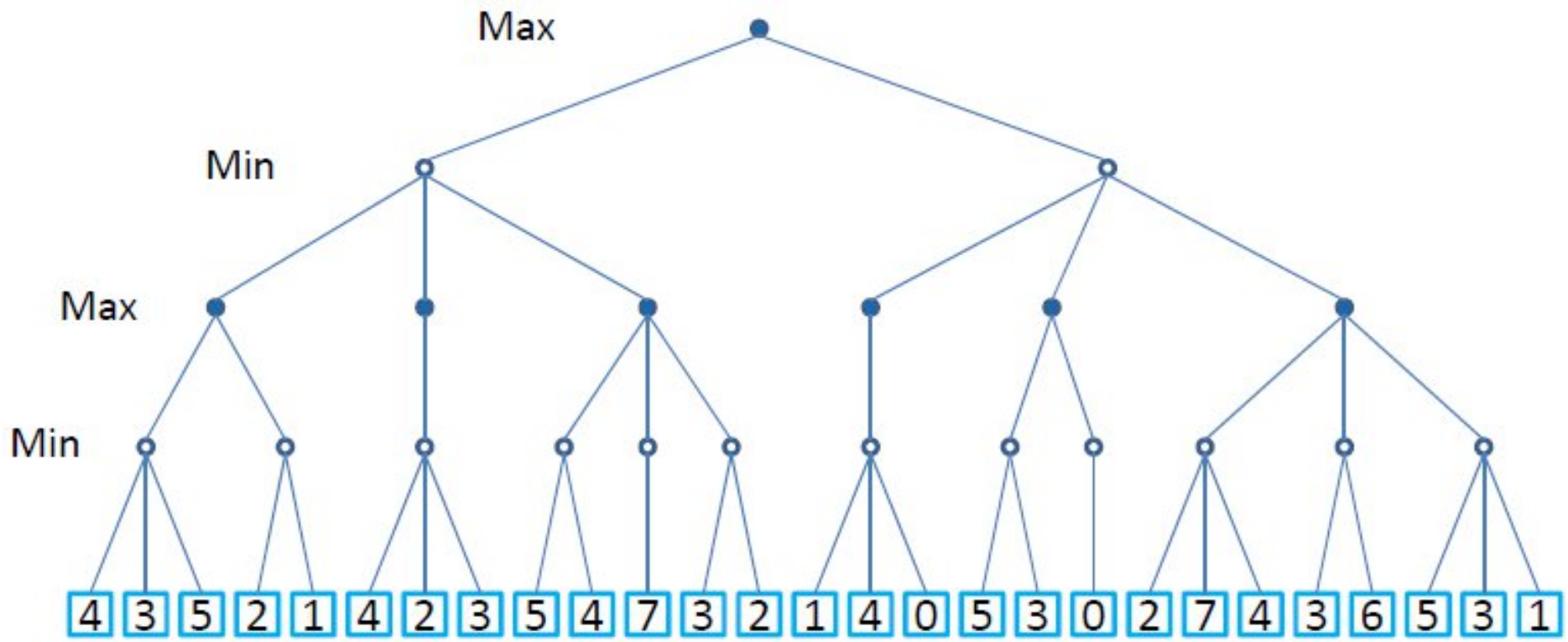
What is the value of the top node?

A) 10
B) 100
C) 2
D) 4

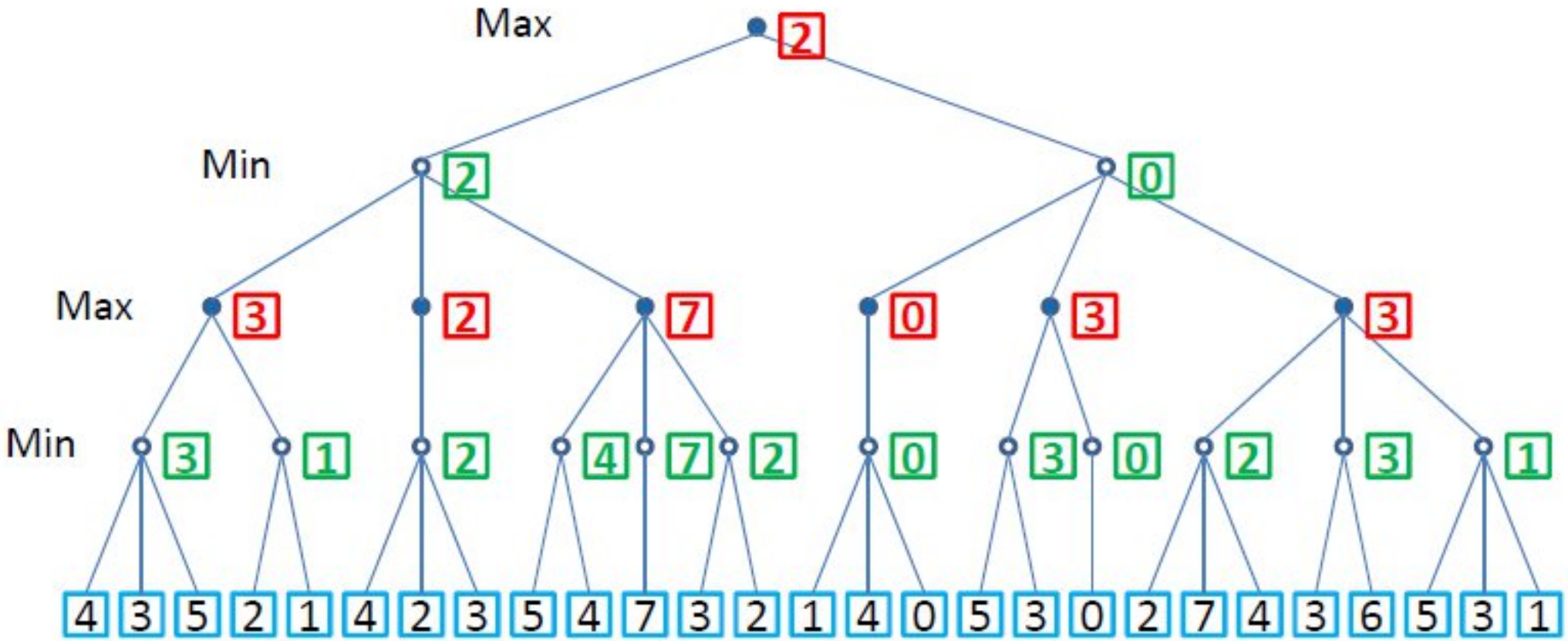


Minimax: Quiz

- Perform the minimax algorithm on the figure below.



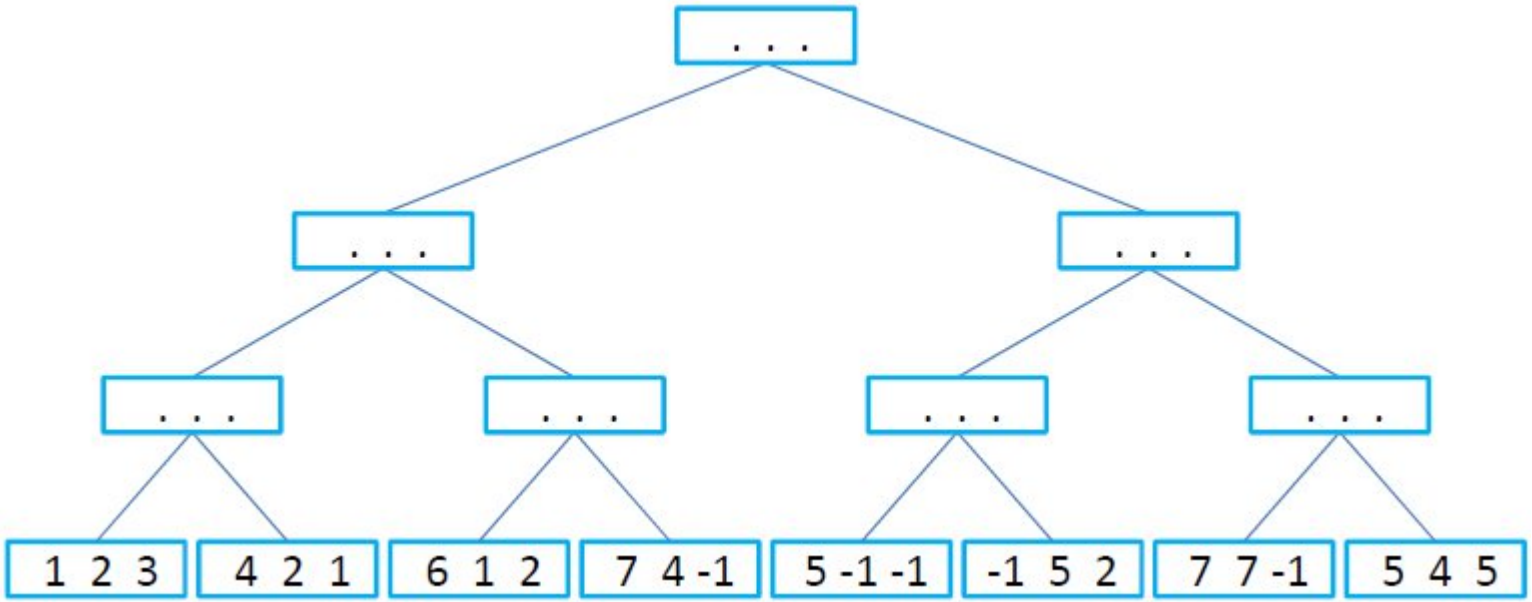
Minimax Search



Minimax for 3 player

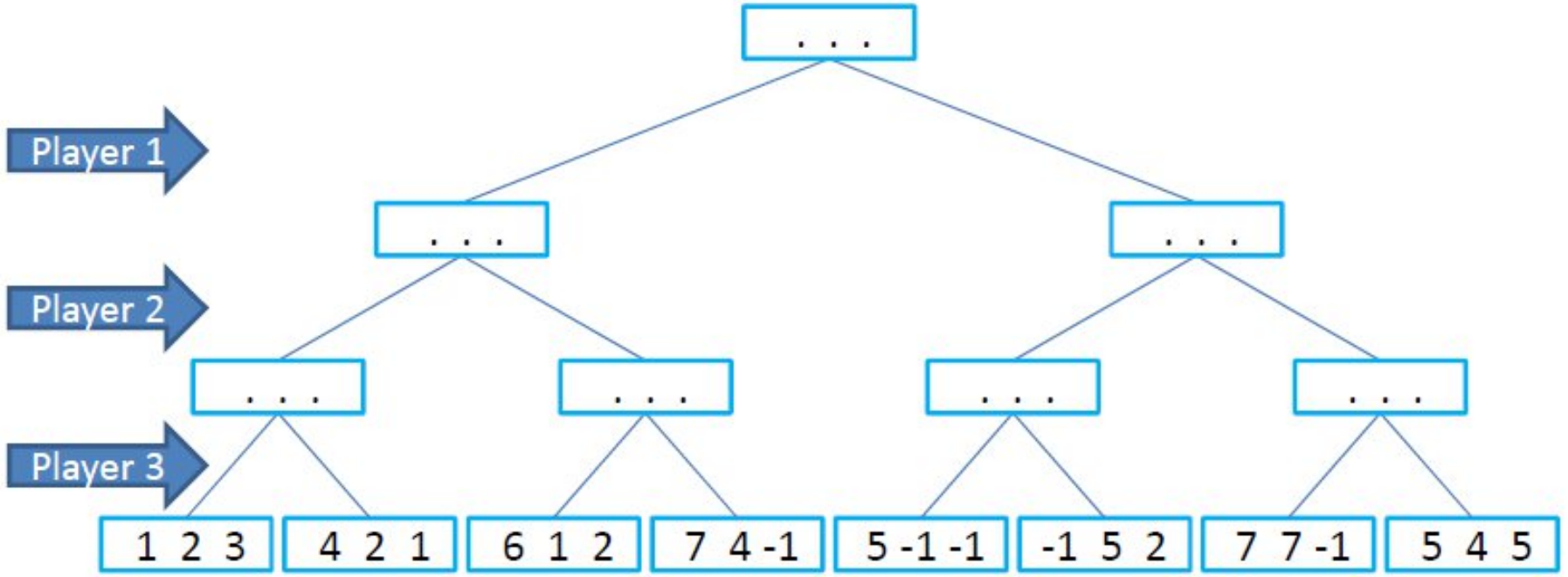
Minimax for 3-Player Game

- All players are MAX.
- Evaluation function given by vectors.



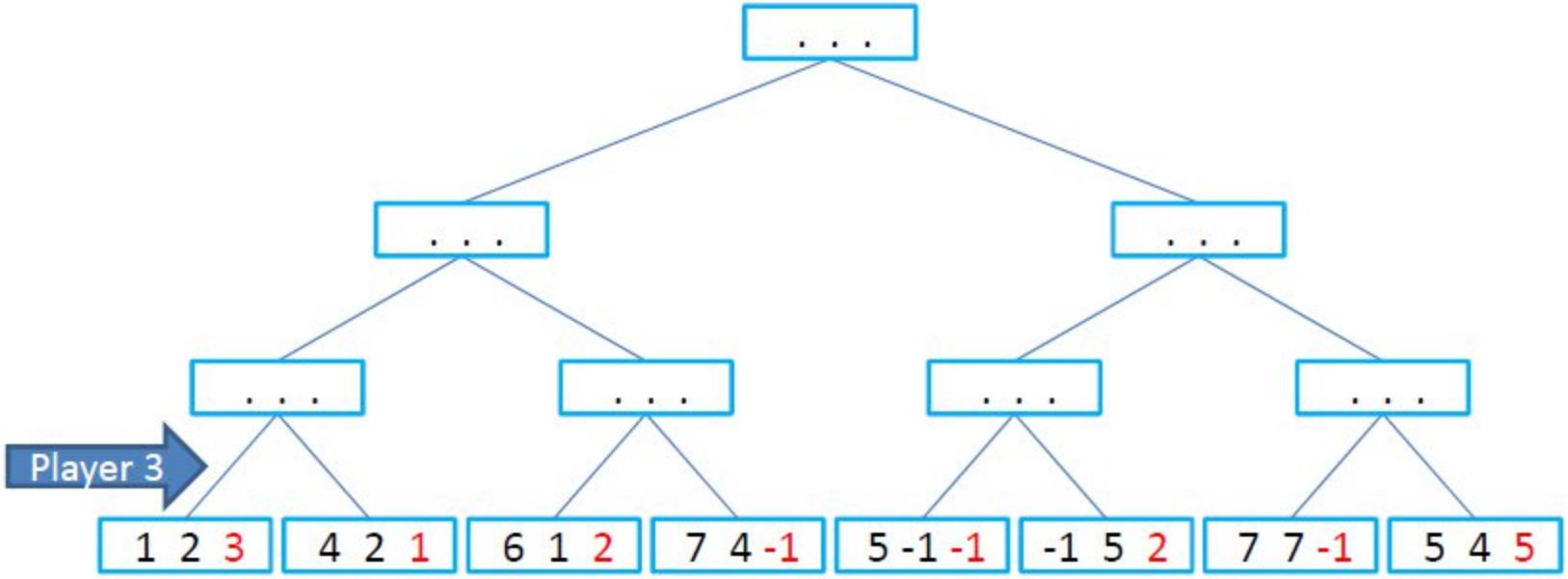
MiniMax For 3 Players

- Each layer assigned to 1 player
- Turn: every 3 layers



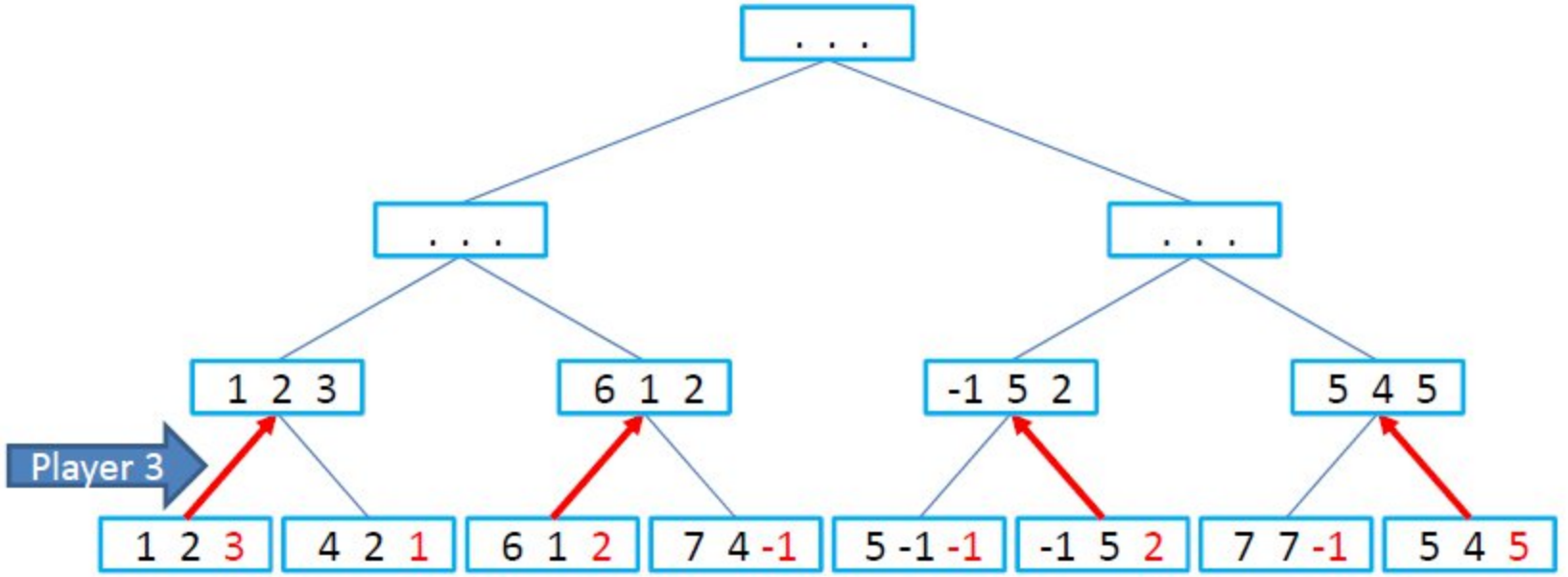
MiniMax For 3 Players

- Max third player: third position of vector



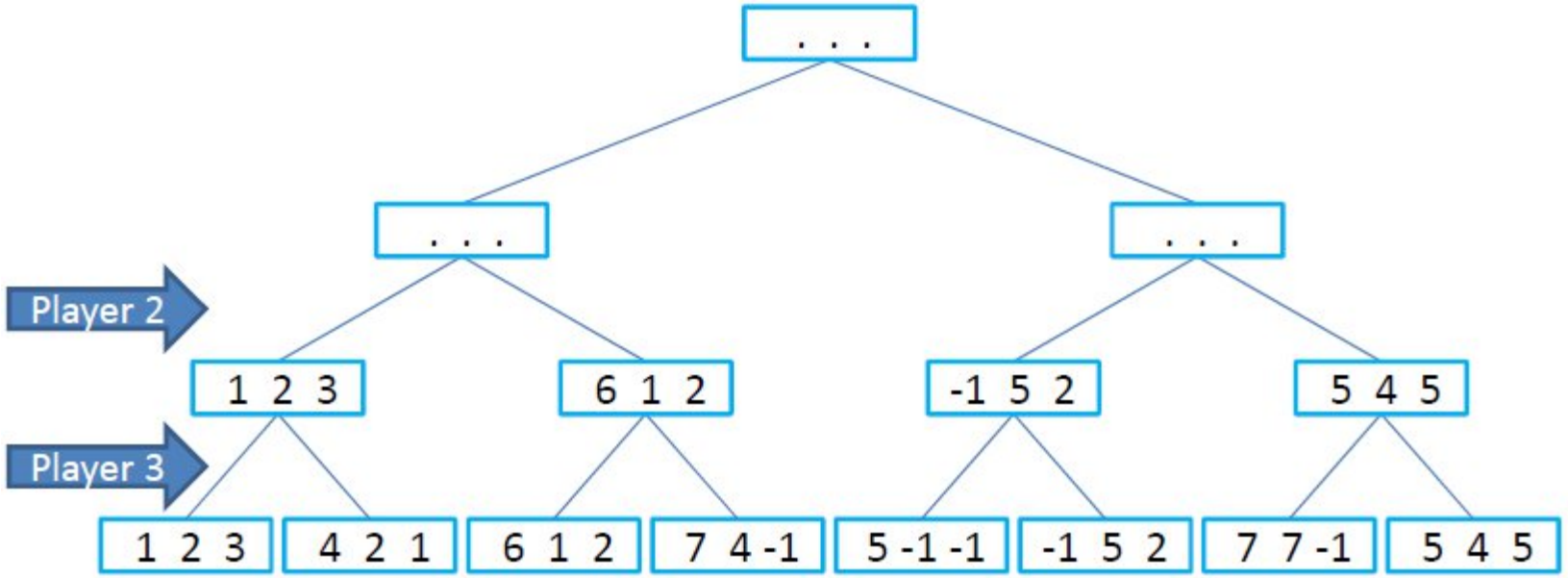
MiniMax For 3 Players

- MaxThirdPlayer([1,2,3],[4,2,1]) = [1,2,3]
- MaxThirdPlayer([6,1,2],[7,4,-1]) = [6,1,2]
- MaxThirdPlayer([5,-1,-1],[-1,5,2]) = [-1,5,2]
- MaxThirdPlayer([7,7,-1],[5,4,5]) = [5,4,5]



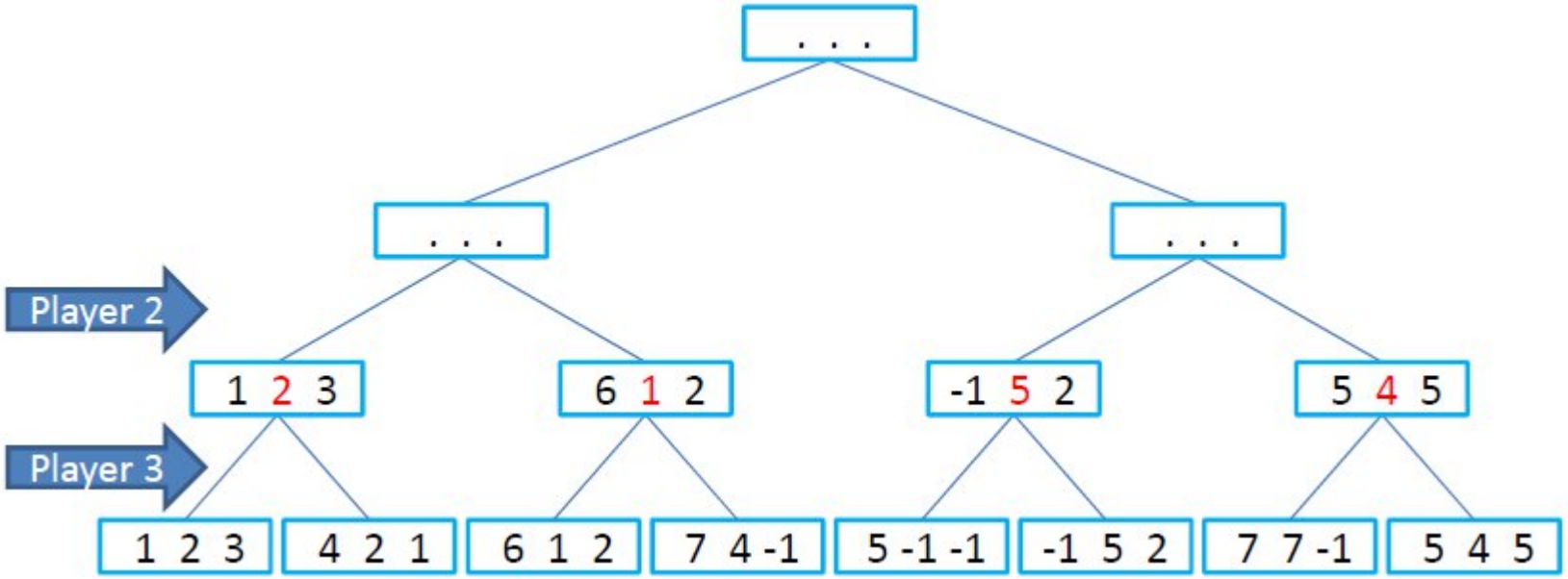
MiniMax For 3 Players

- Second player’s move



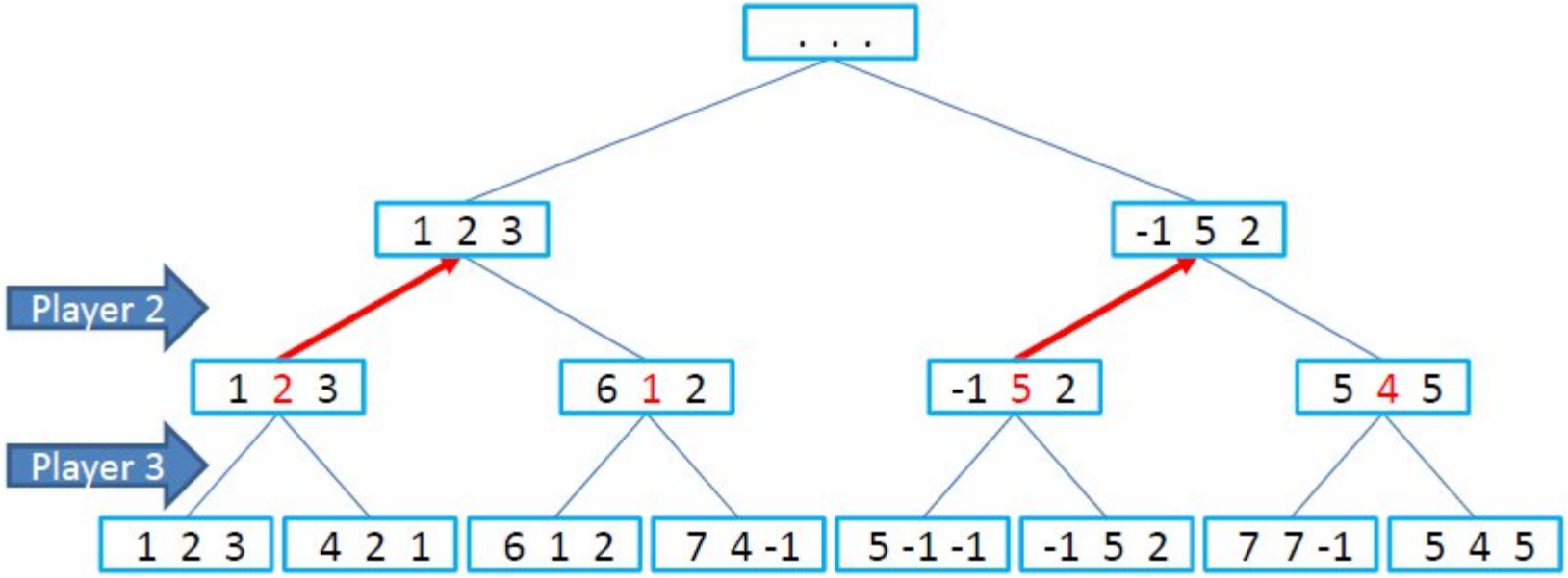
MiniMax For 3 Players

- Max second player: second position of vector



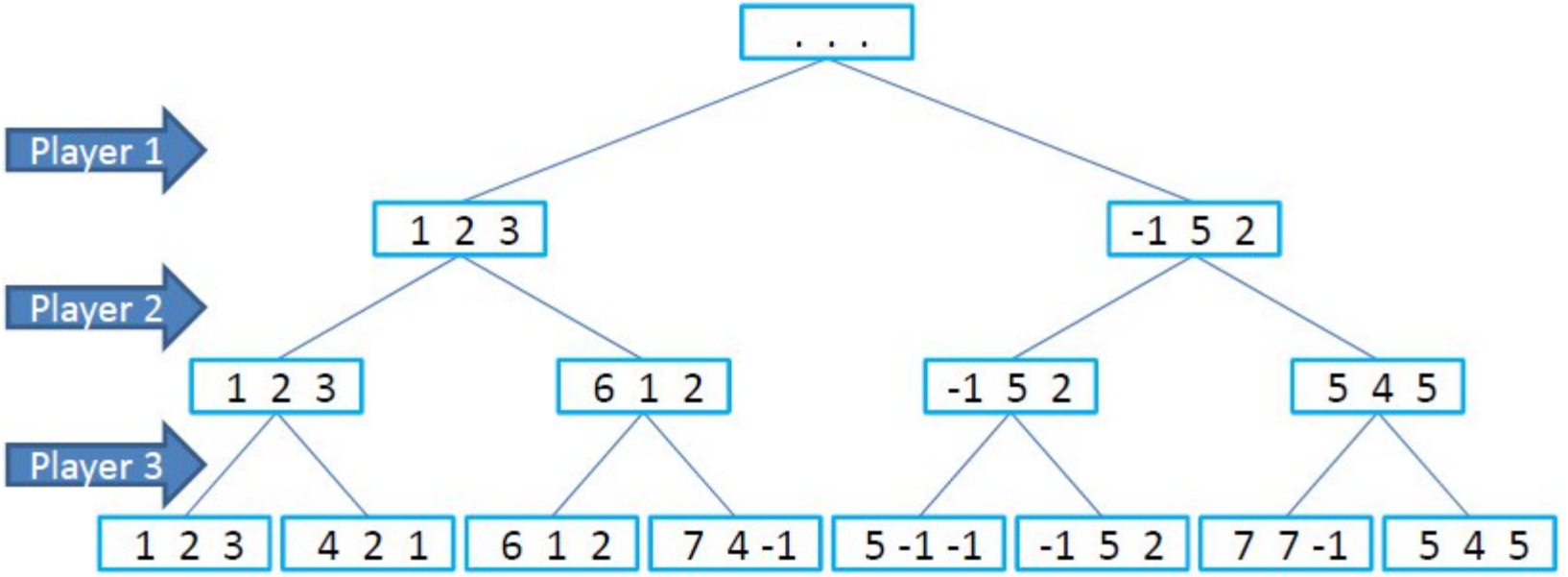
MiniMax For 3 Players

- MaxSecondPlayer([1,2,3],[6,1,2]) = [1,2,3]
- • MaxSecondPlayer([-1,5,2],[5,4,5]) = [-1,5,2]



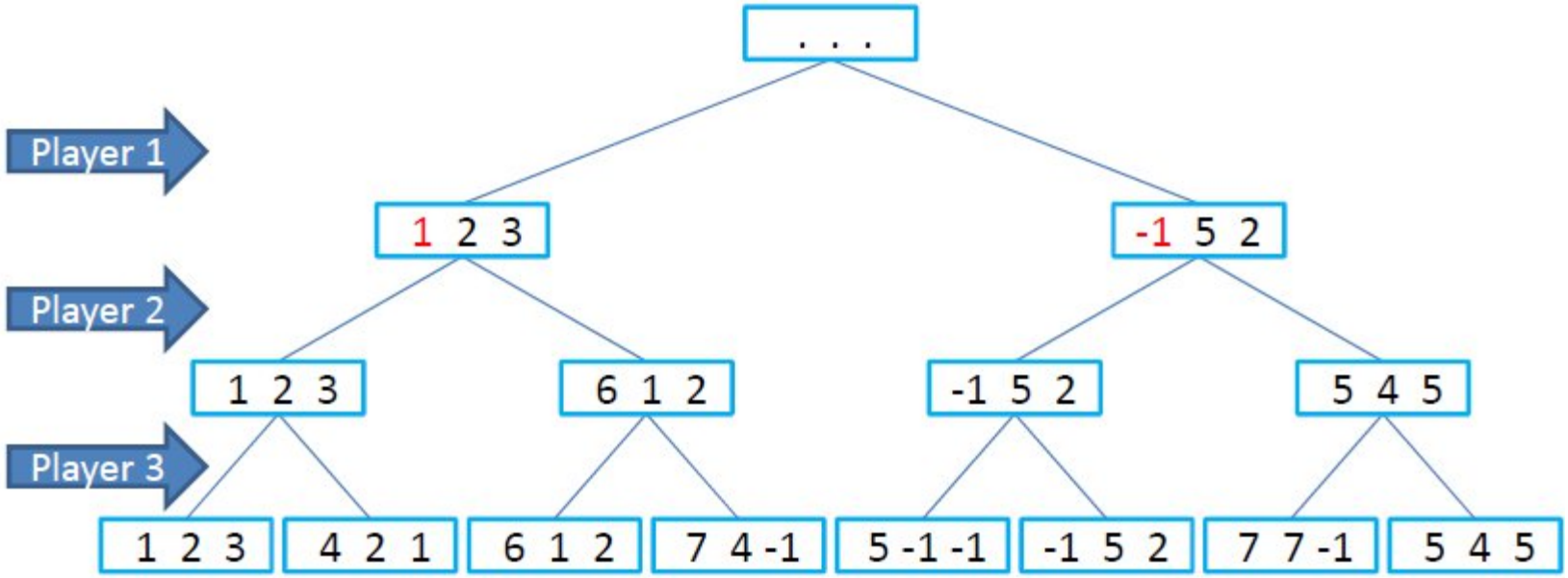
MiniMax For 3 Players

- First player's move



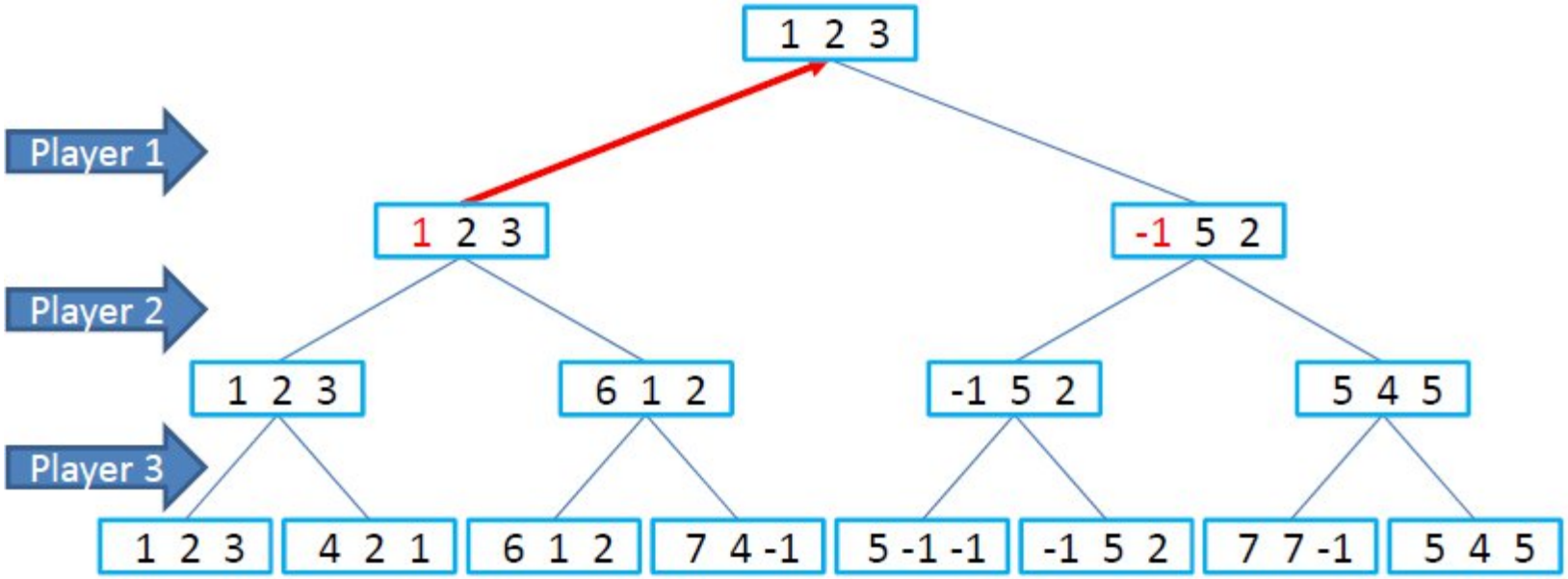
MiniMax For 3 Players

- Max first player: first position of vector



MiniMax For 3 Players

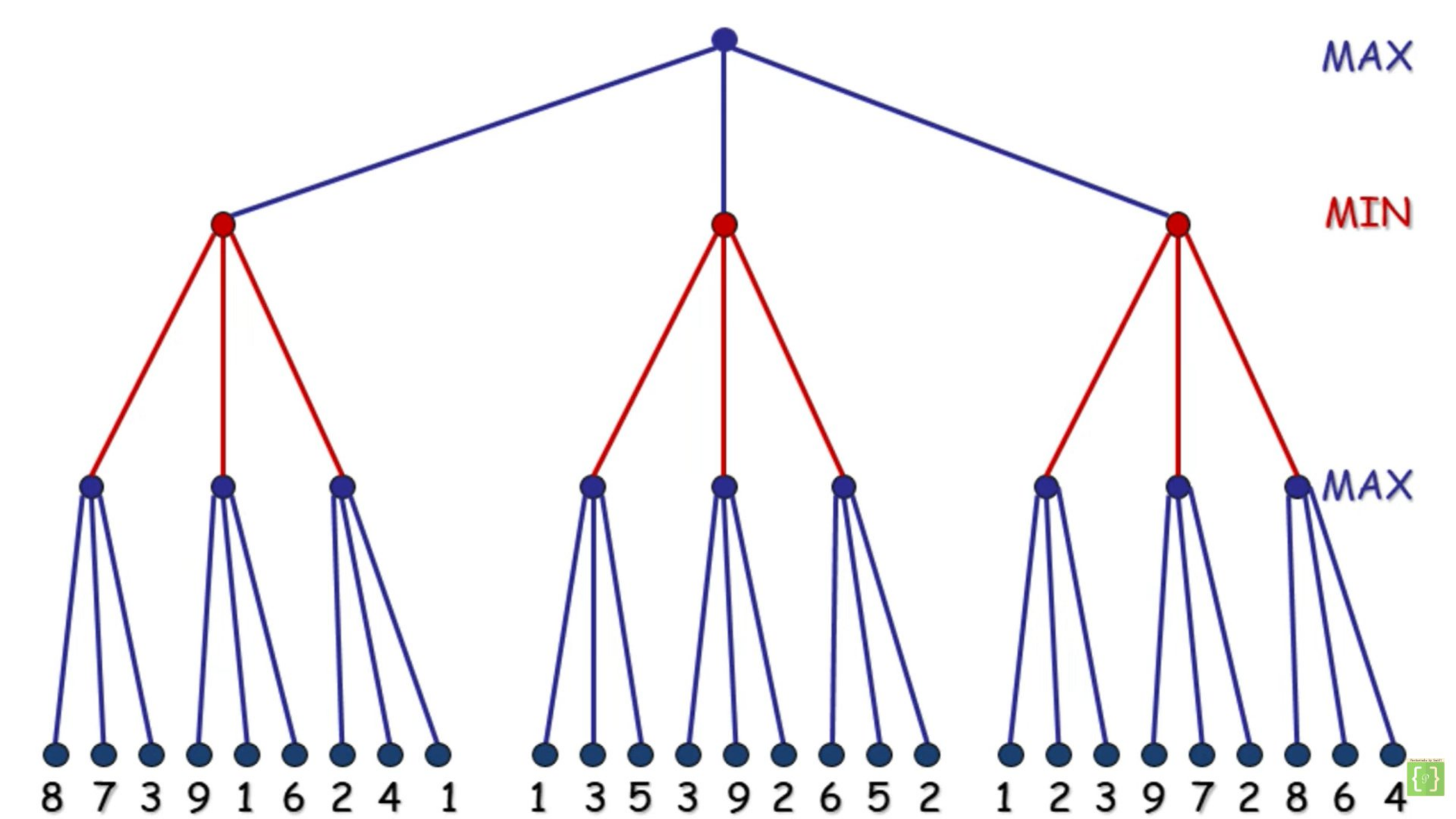
- MaxFirstPlayer([1,2,3],[-1,5,4]) = [1,2,3]

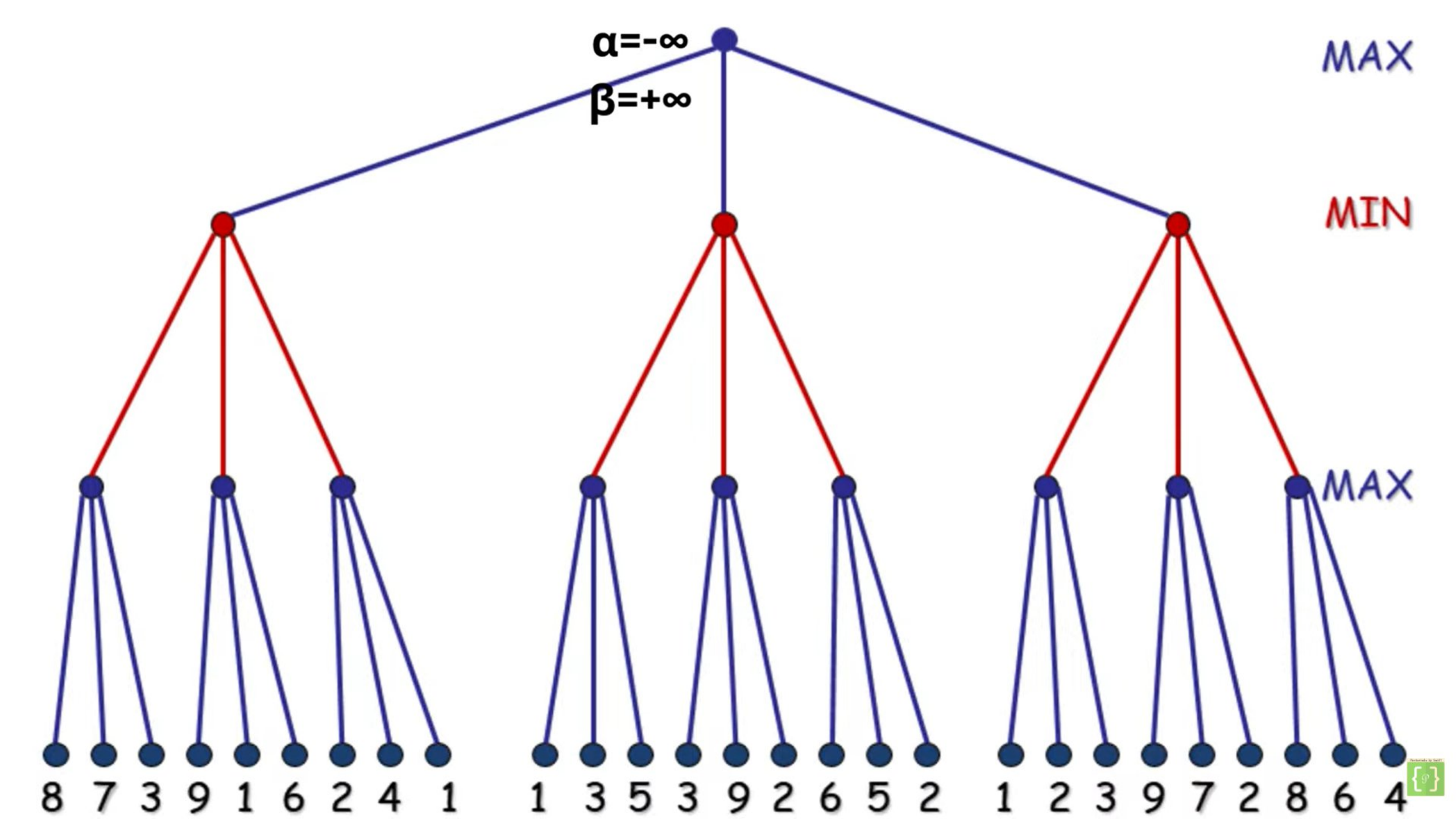


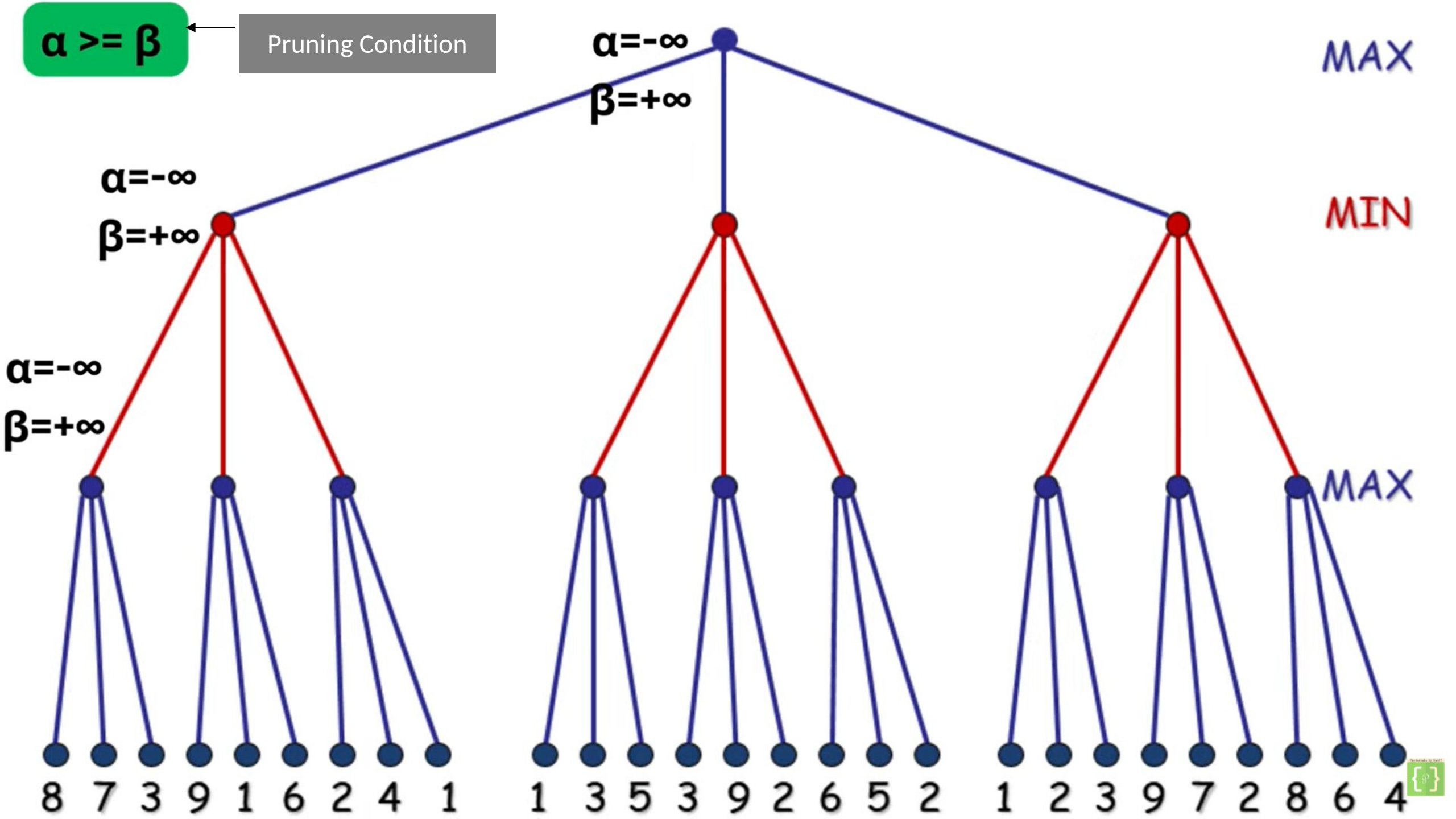
Alpha Beta Pruning

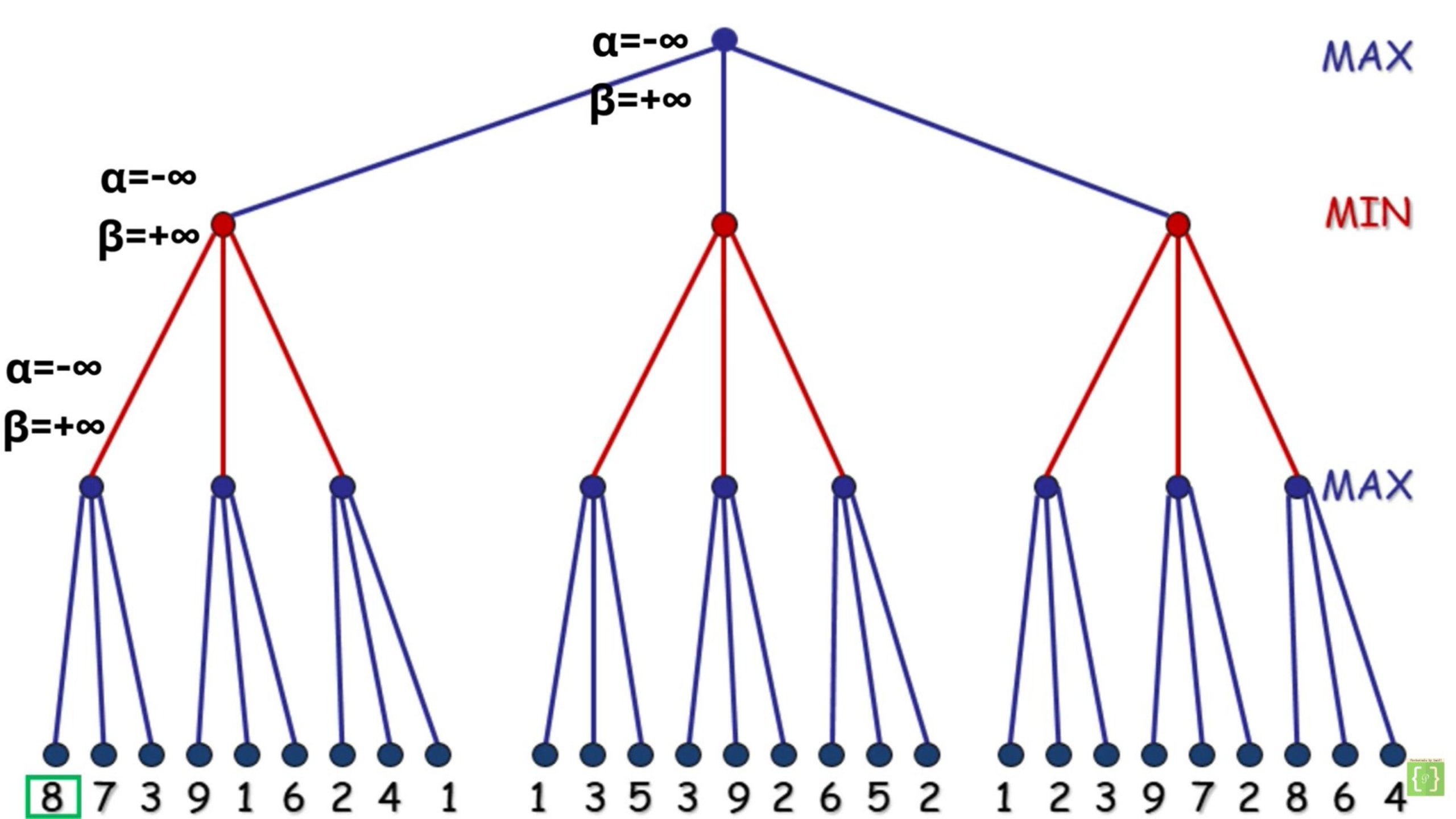
- Max starts with α=-∞ (wants the highest value)
- Min starts with β= ∞ (wants the lowest value)
- Each level updates α and β to keep track of the best / worst possible moves
- If α >= β, we stop searching that branch – its already worse than another move.

Alpha Beta Pruning Example

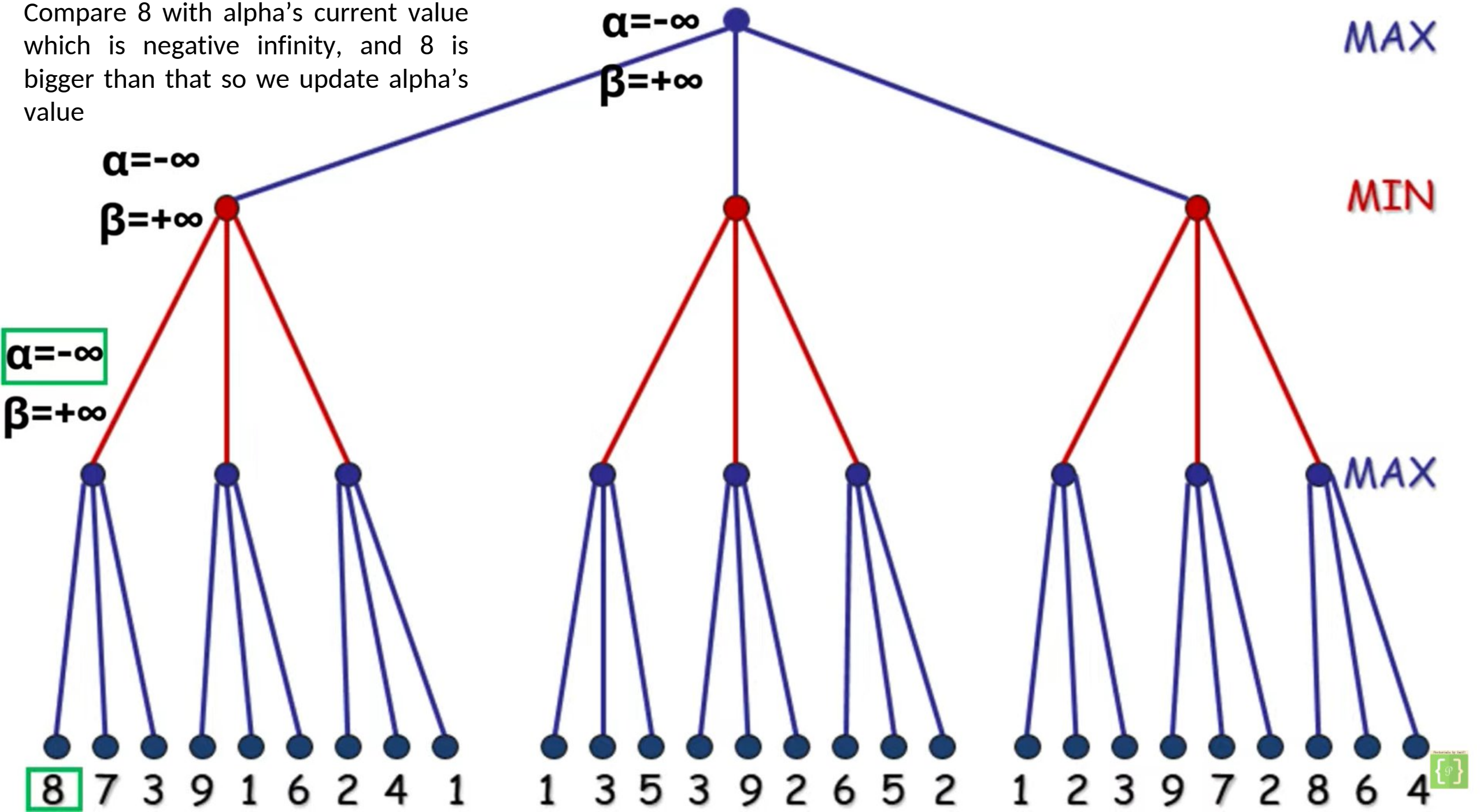

MAX
MIN
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4


α=-∞
β=+∞
MAX
MIN
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4


α >= β
Pruning Condition
α=-∞
β=+∞
MAX
α=-∞
β=+∞
MIN
α=-∞
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

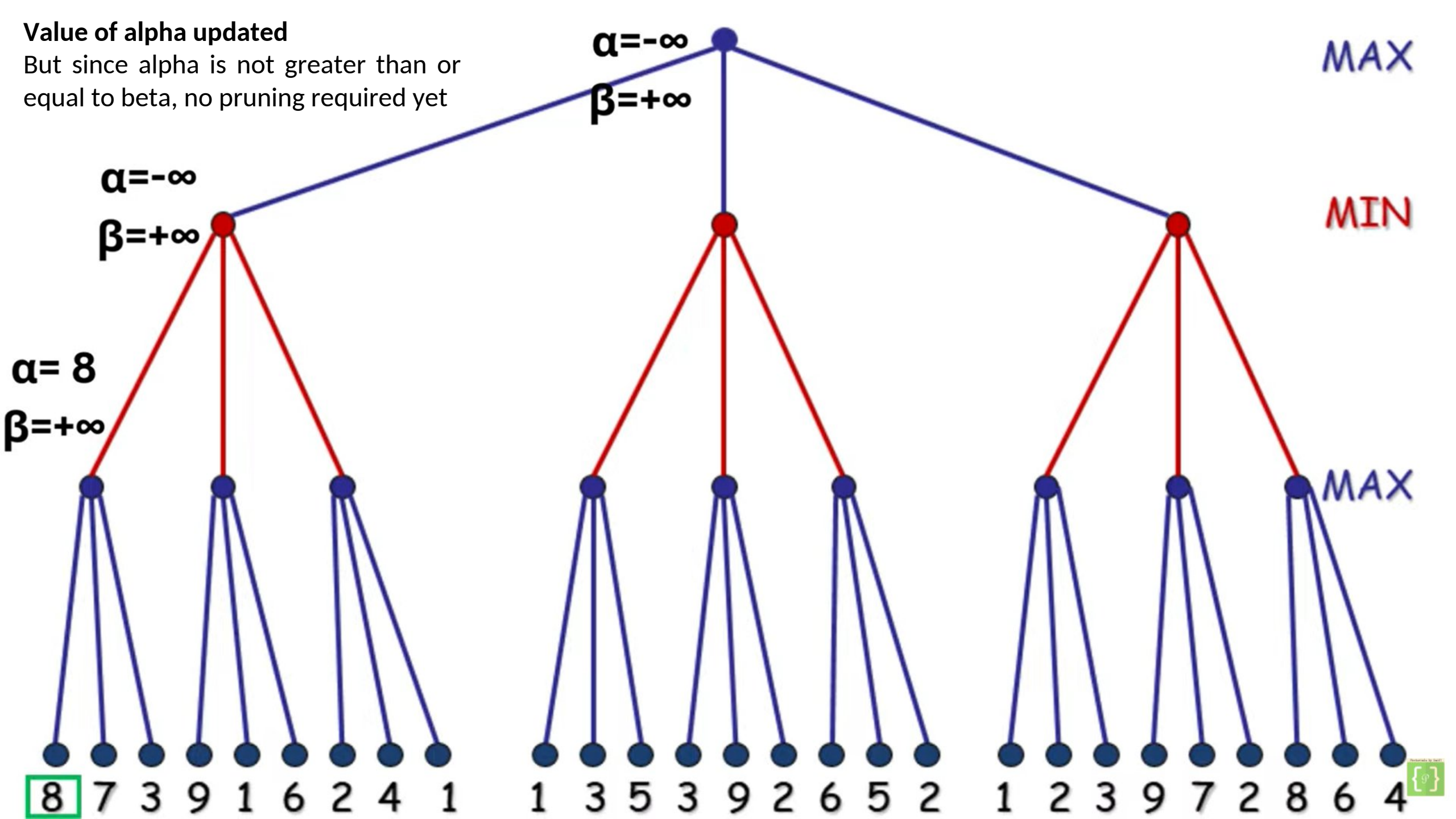

α=-∞
β=+∞
MAX
α=-∞
β=+∞
MIN
α=-∞
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

Compare 8 with alpha's current value which is negative infinity, and 8 is bigger than that so we update alpha's value

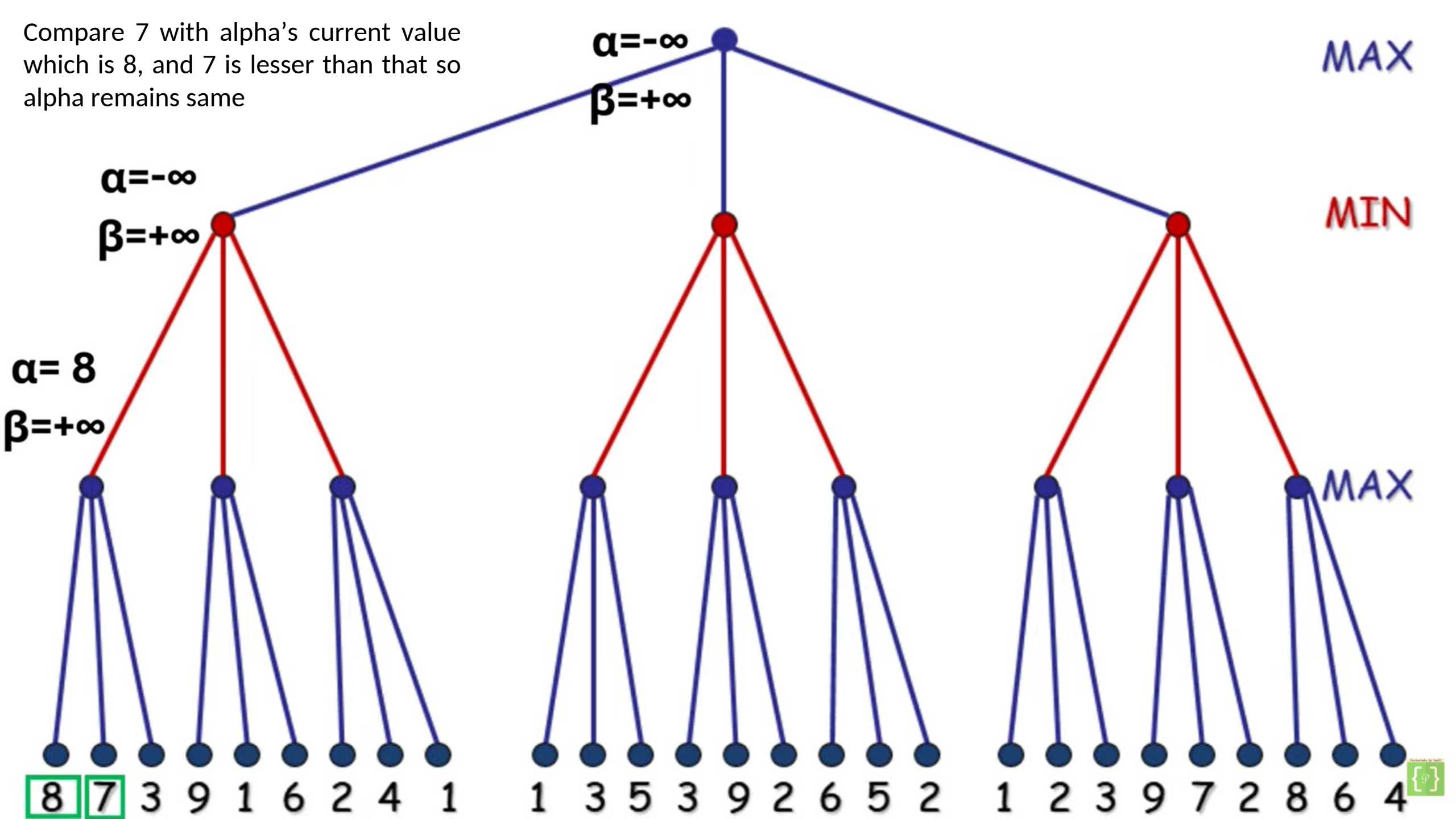


Value of alpha updated

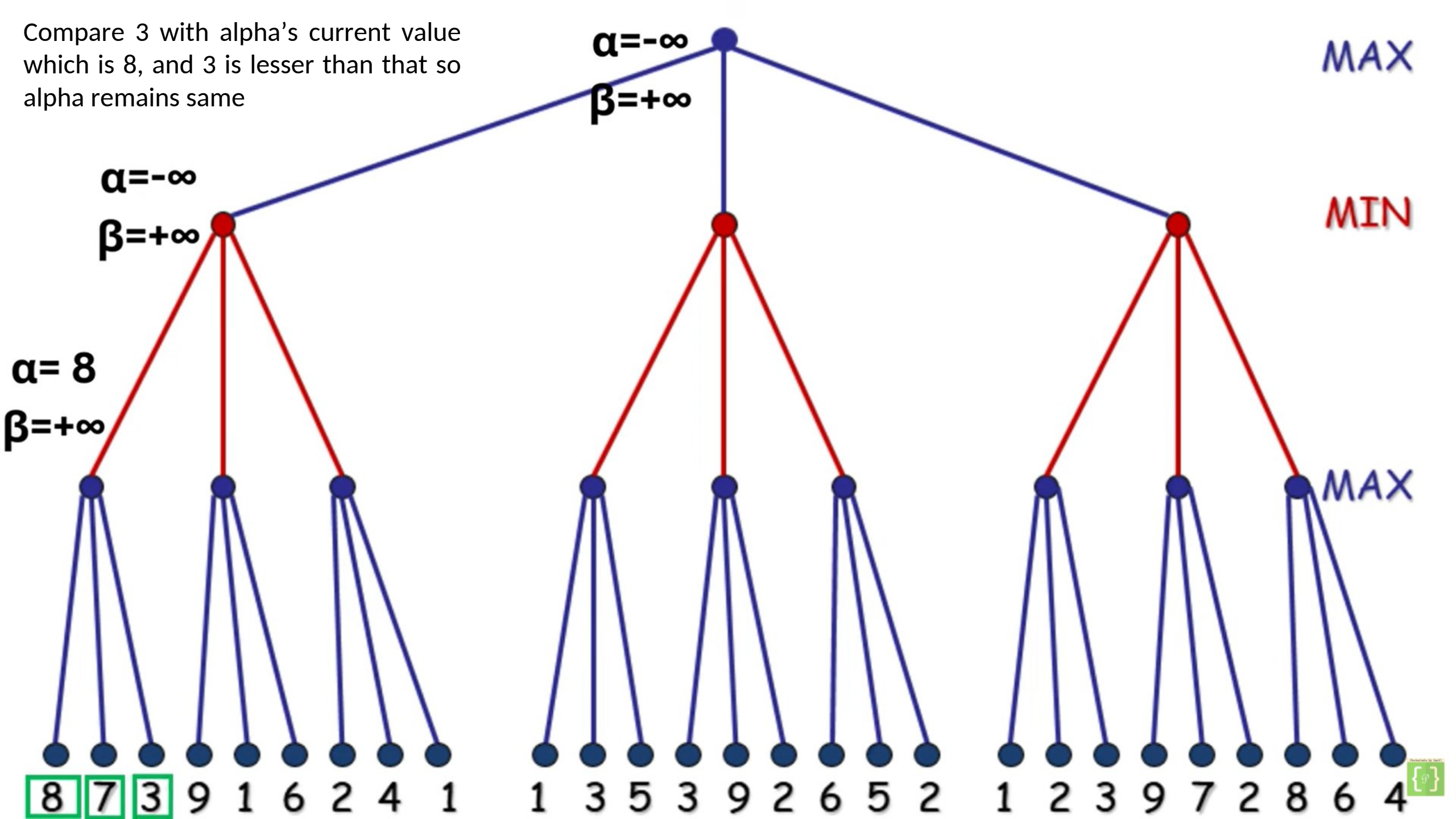
But since alpha is not greater than or equal to beta, no pruning required yet

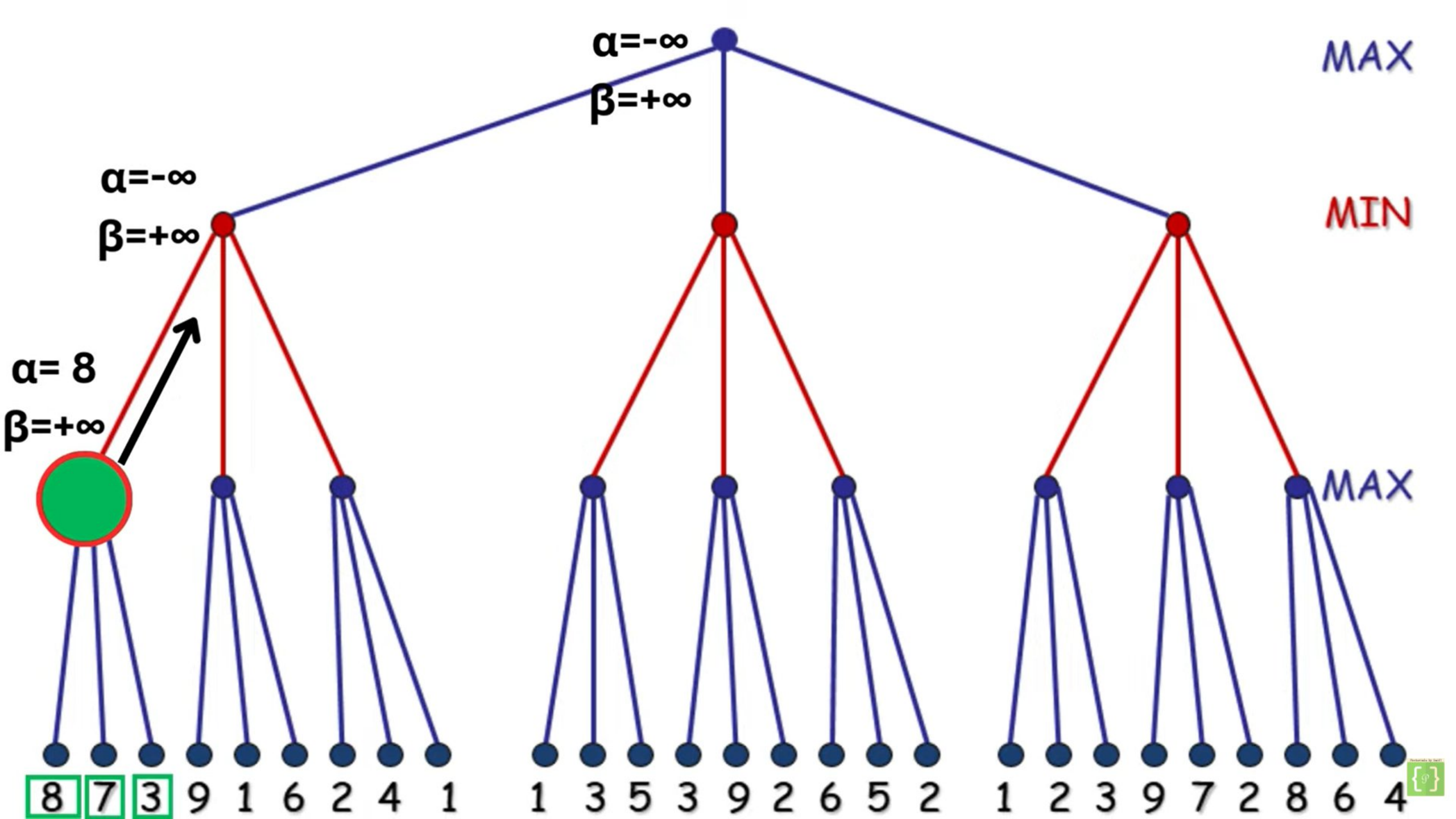


Compare 7 with alpha's current value which is 8, and 7 is lesser than that so alpha remains same

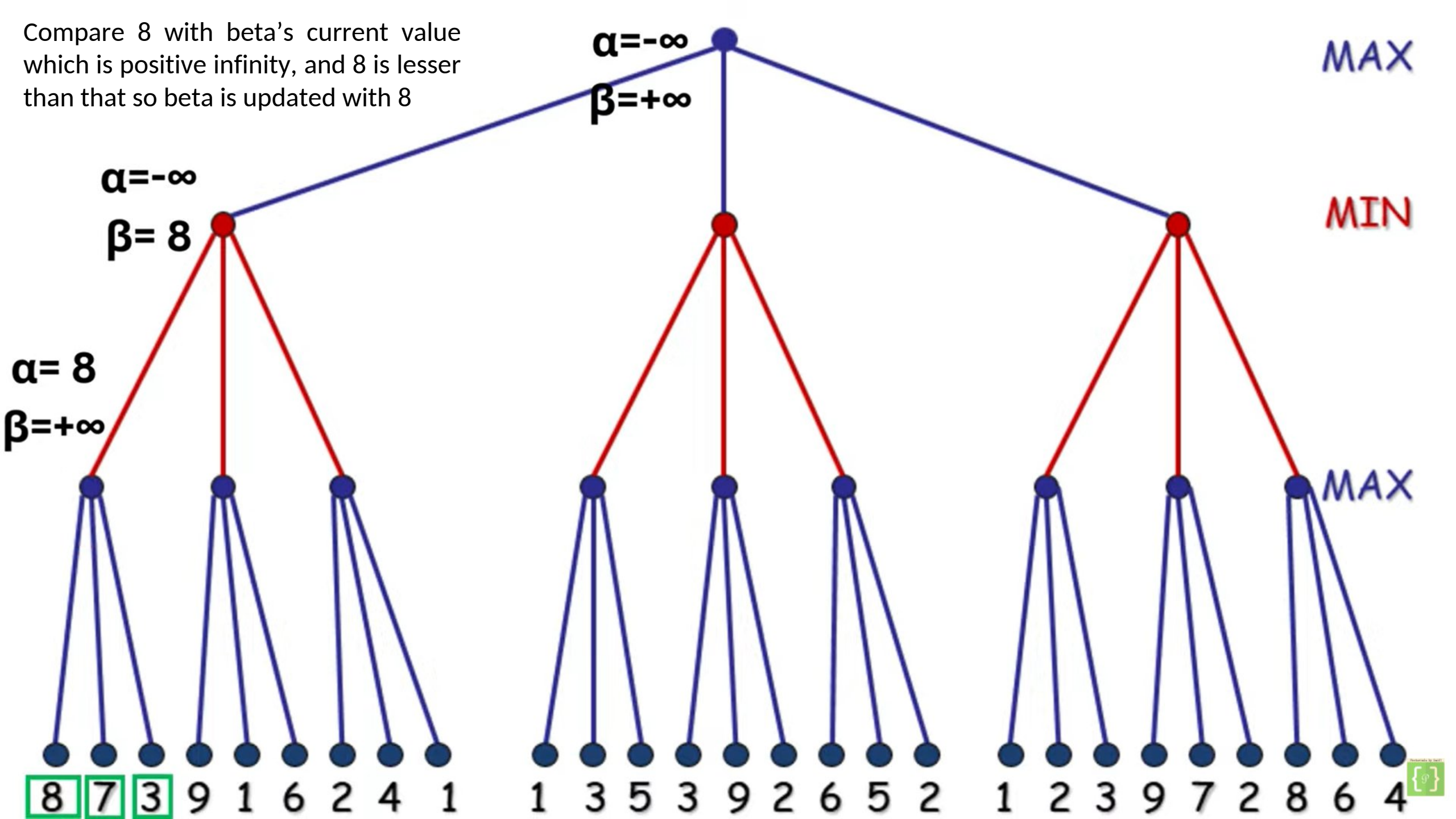


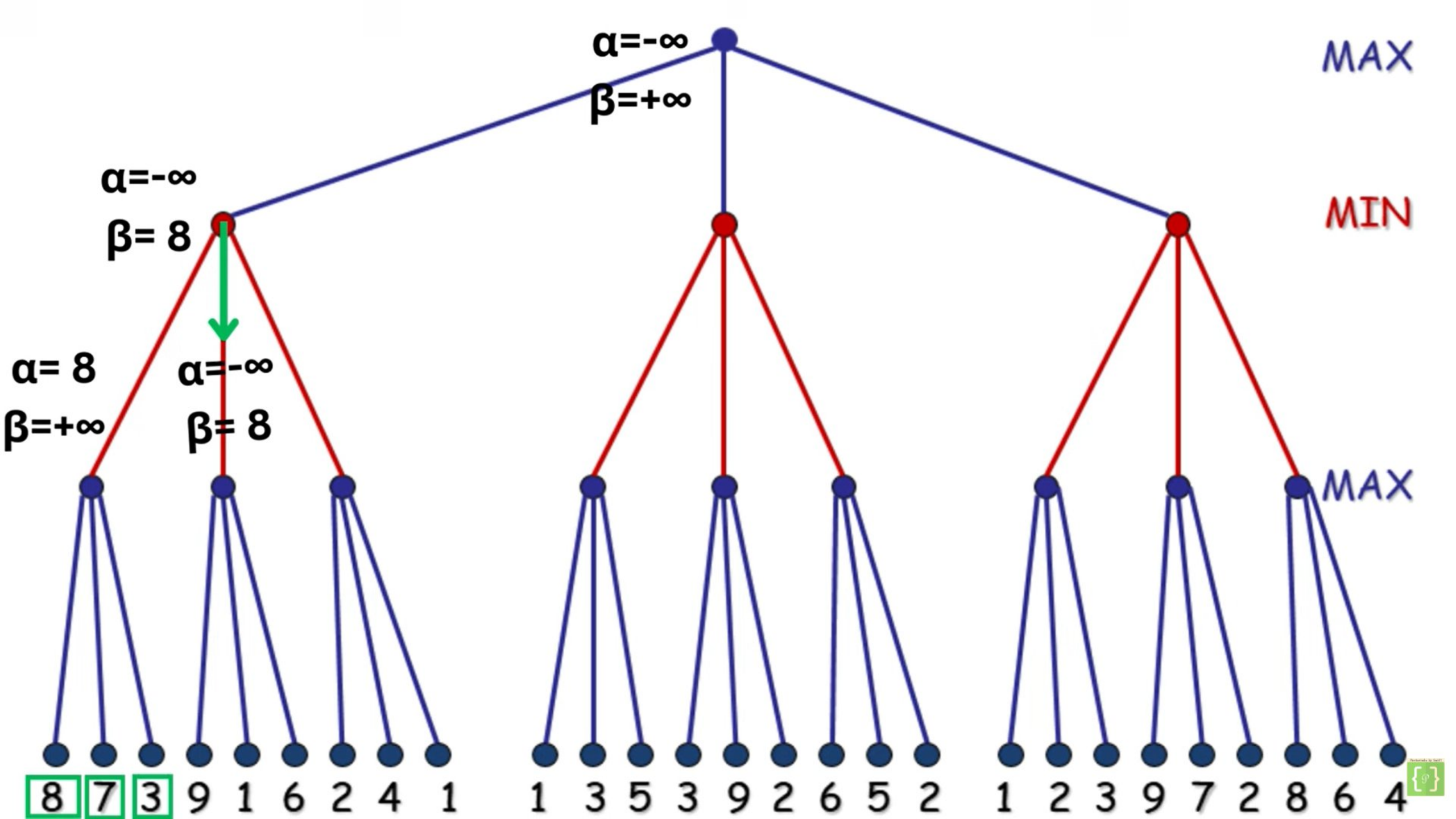
Compare 3 with alpha's current value which is 8, and 3 is lesser than that so alpha remains same

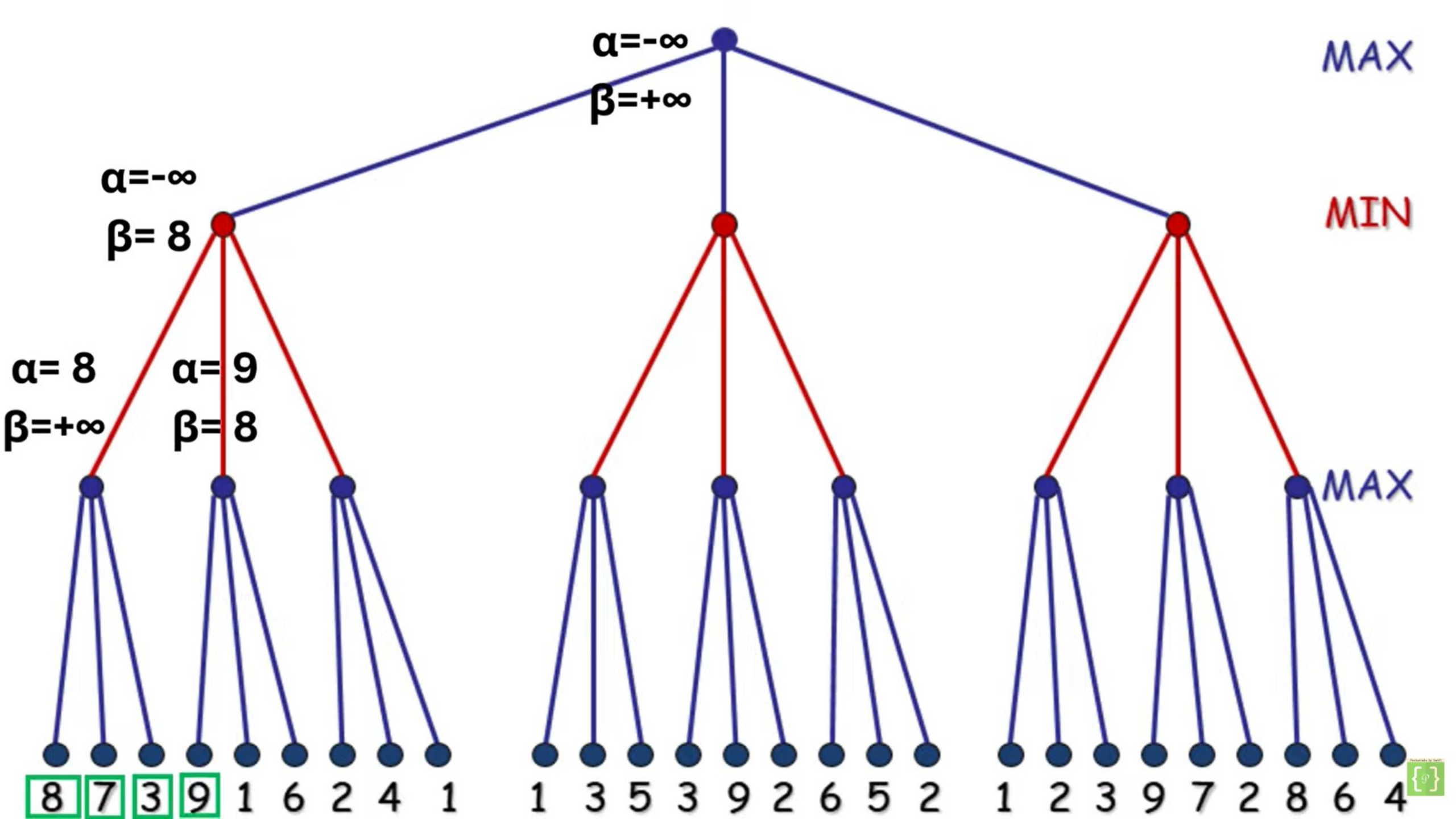


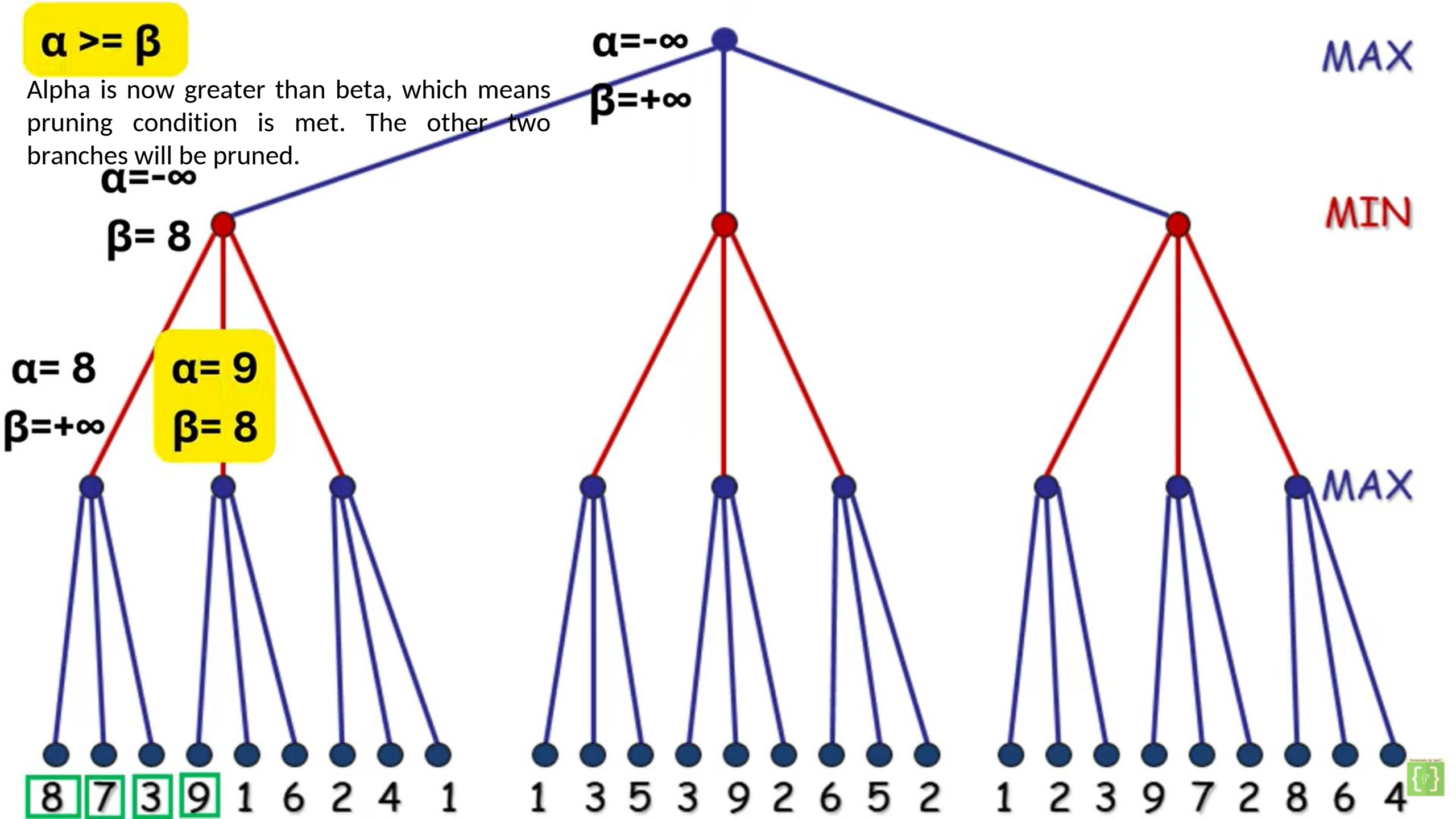

α=-∞
β=+∞
MAX
α=-∞
β=+∞
MIN
α= 8
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

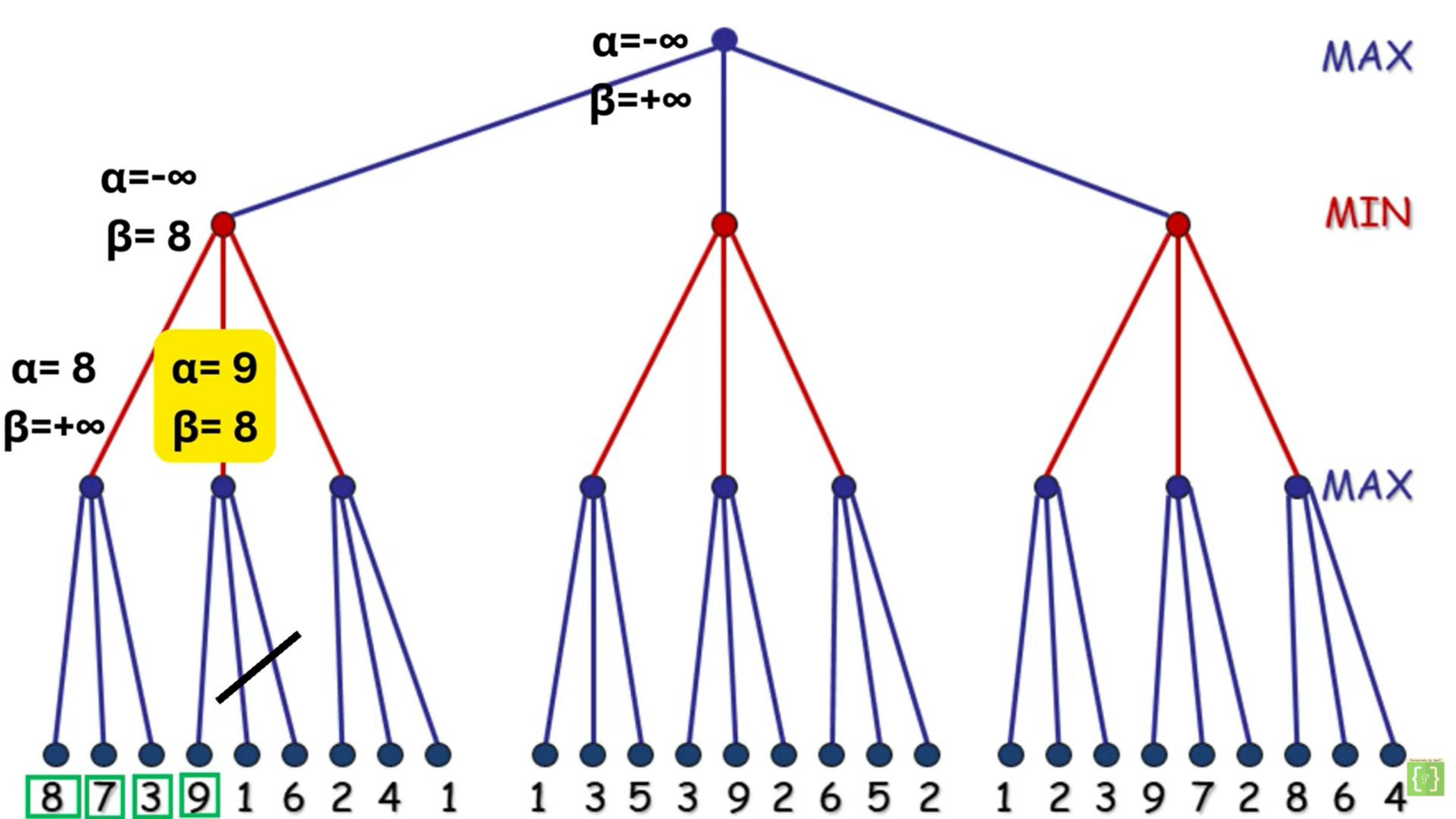
Compare 8 with beta's current value which is positive infinity, and 8 is lesser than that so beta is updated with 8

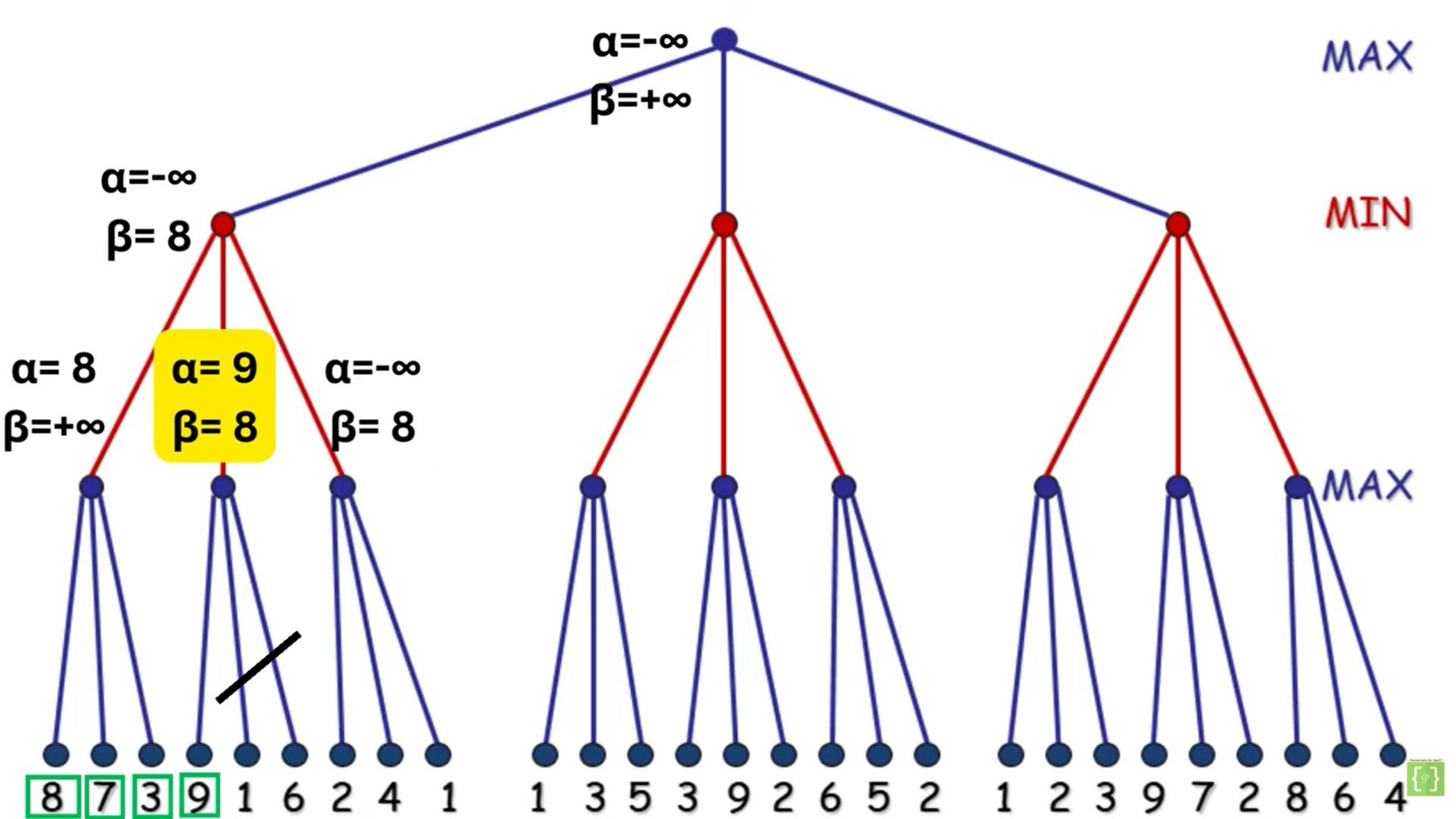



α=-∞
β=+∞
MAX
α=-∞
β= 8
MIN
α= 8
β=+∞
α=-∞
β= 8
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

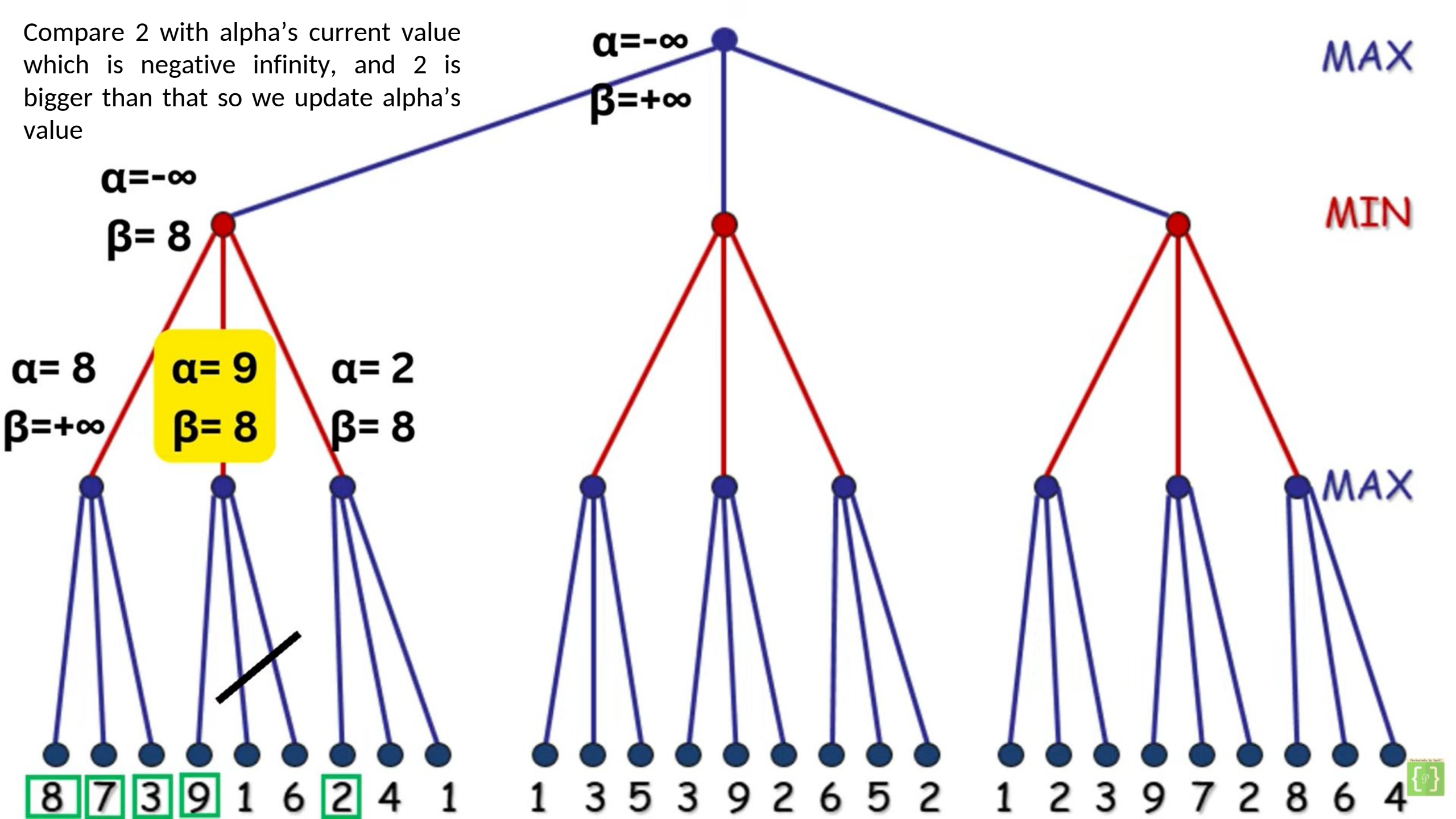

α=-∞
β=+∞
MAX
α=-∞
β= 8
MIN
α= 8
β=+∞
α= 9
β= 8
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4


α >= β
Alpha is now greater than beta, which means pruning condition is met. The other two branches will be pruned.
α=-∞
β=+∞
MAX
α=-∞
β= 8
MIN
α= 8
β=+∞
α= 9
β= 8
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

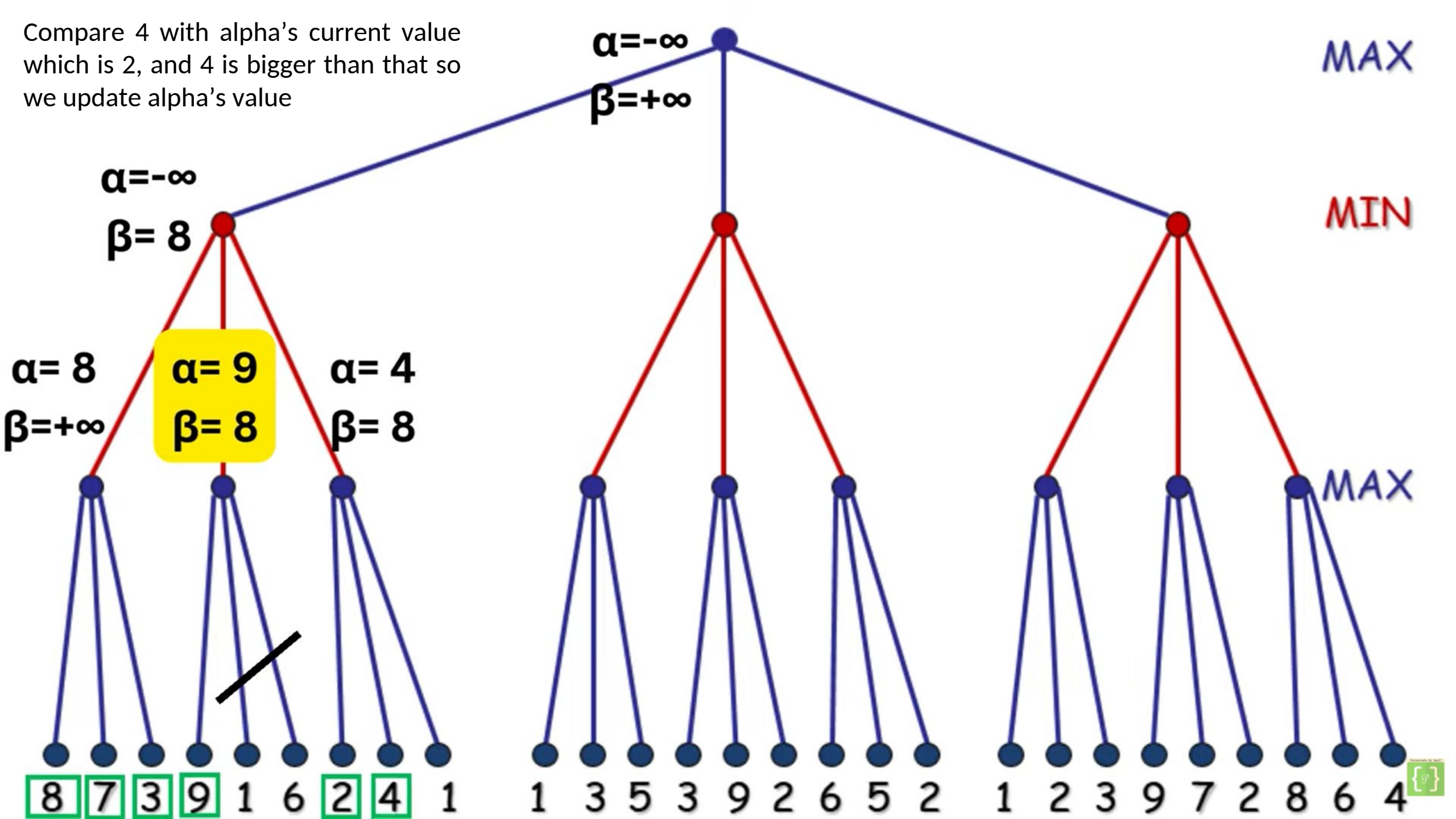

α=-∞
β=+∞
MAX
α=-∞
β= 8
MIN
α= 8
β=+∞
α= 9
β= 8
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4


α=-∞
β=+∞
MAX
α=-∞
β= 8
MIN
α= 8
β=+∞
α= 9
β= 8
α=-∞
β= 8
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

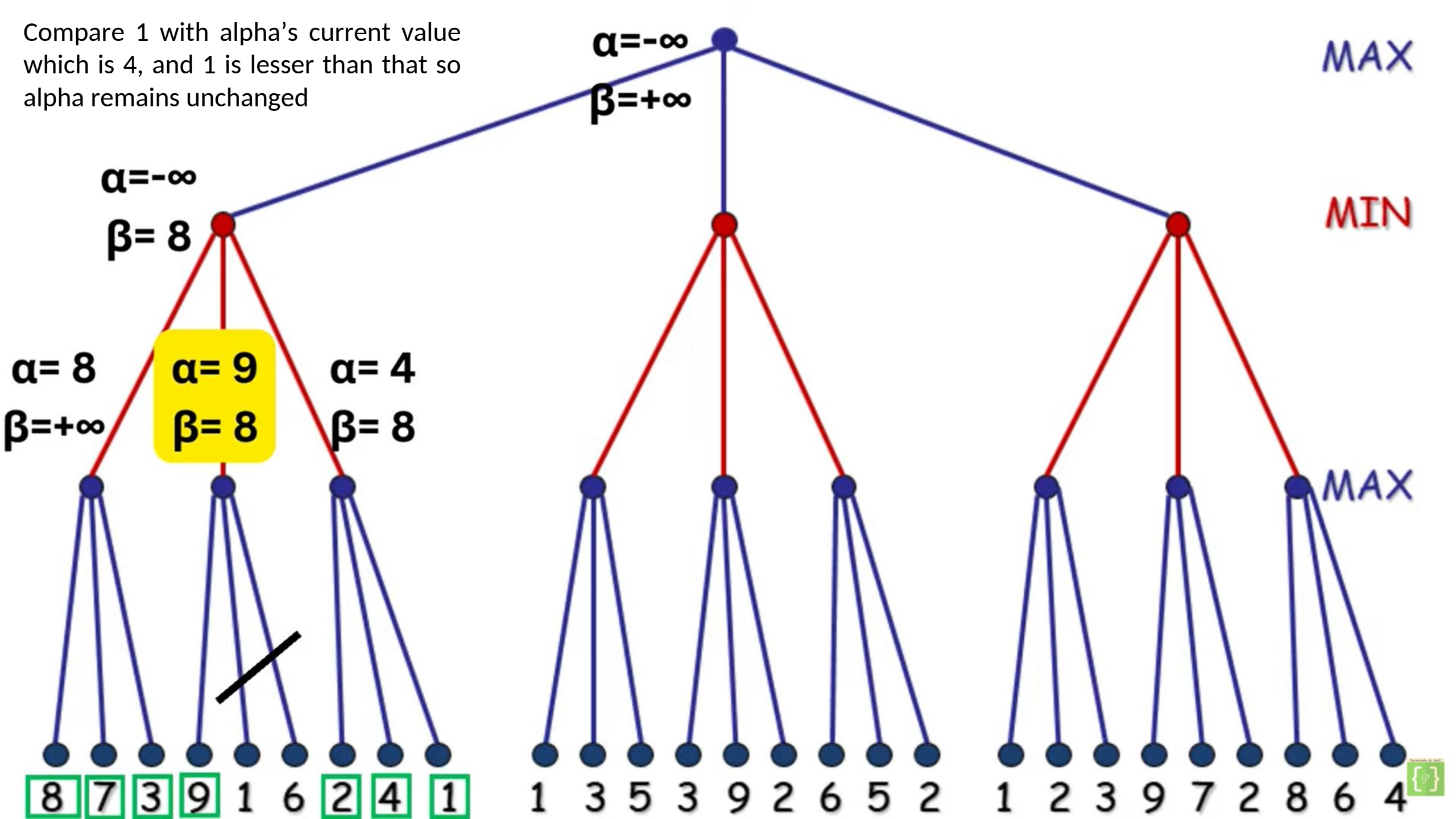
Compare 2 with alpha's current value which is negative infinity, and 2 is bigger than that so we update alpha's value



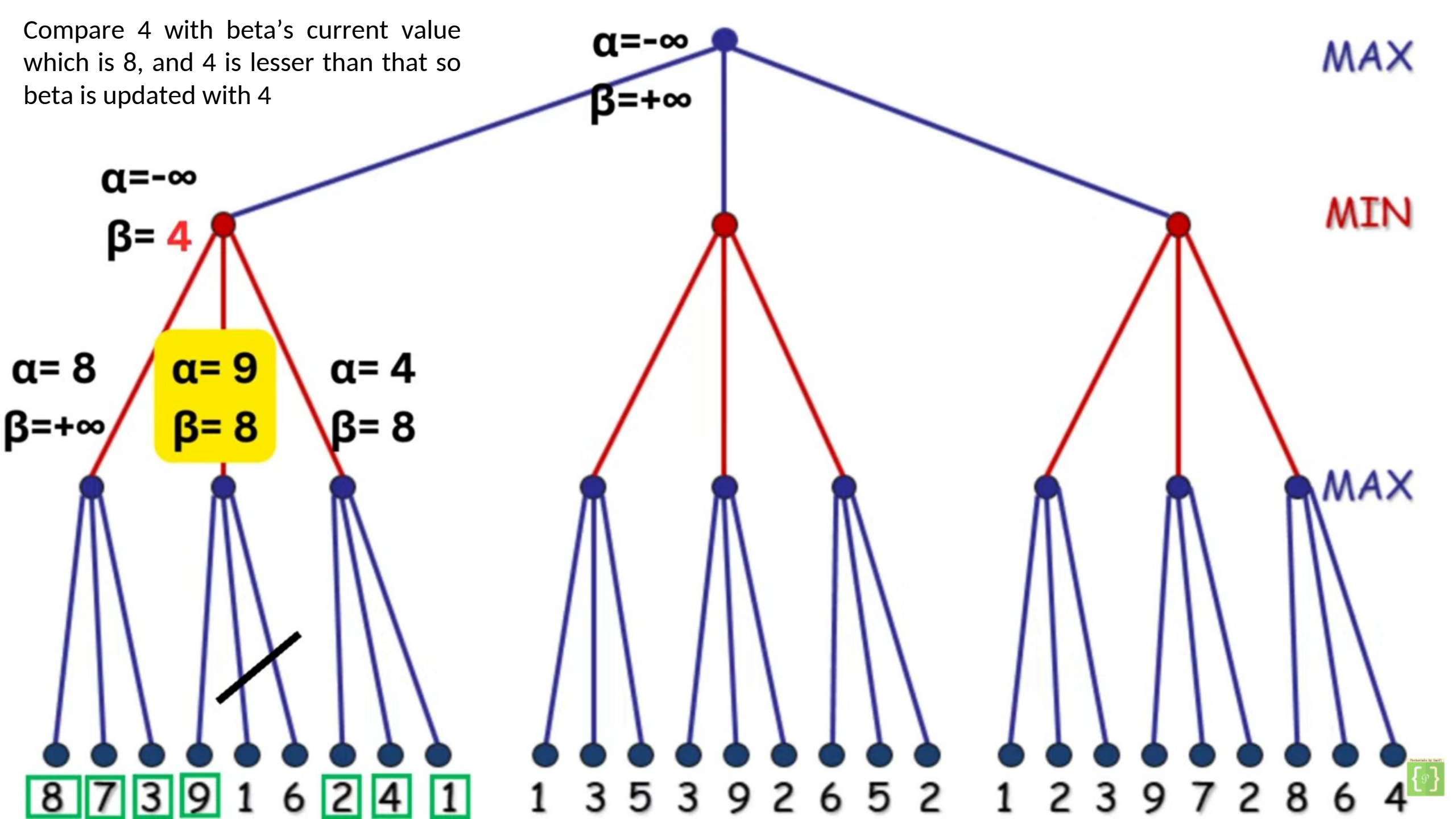
Compare 4 with alpha's current value which is 2, and 4 is bigger than that so we update alpha's value



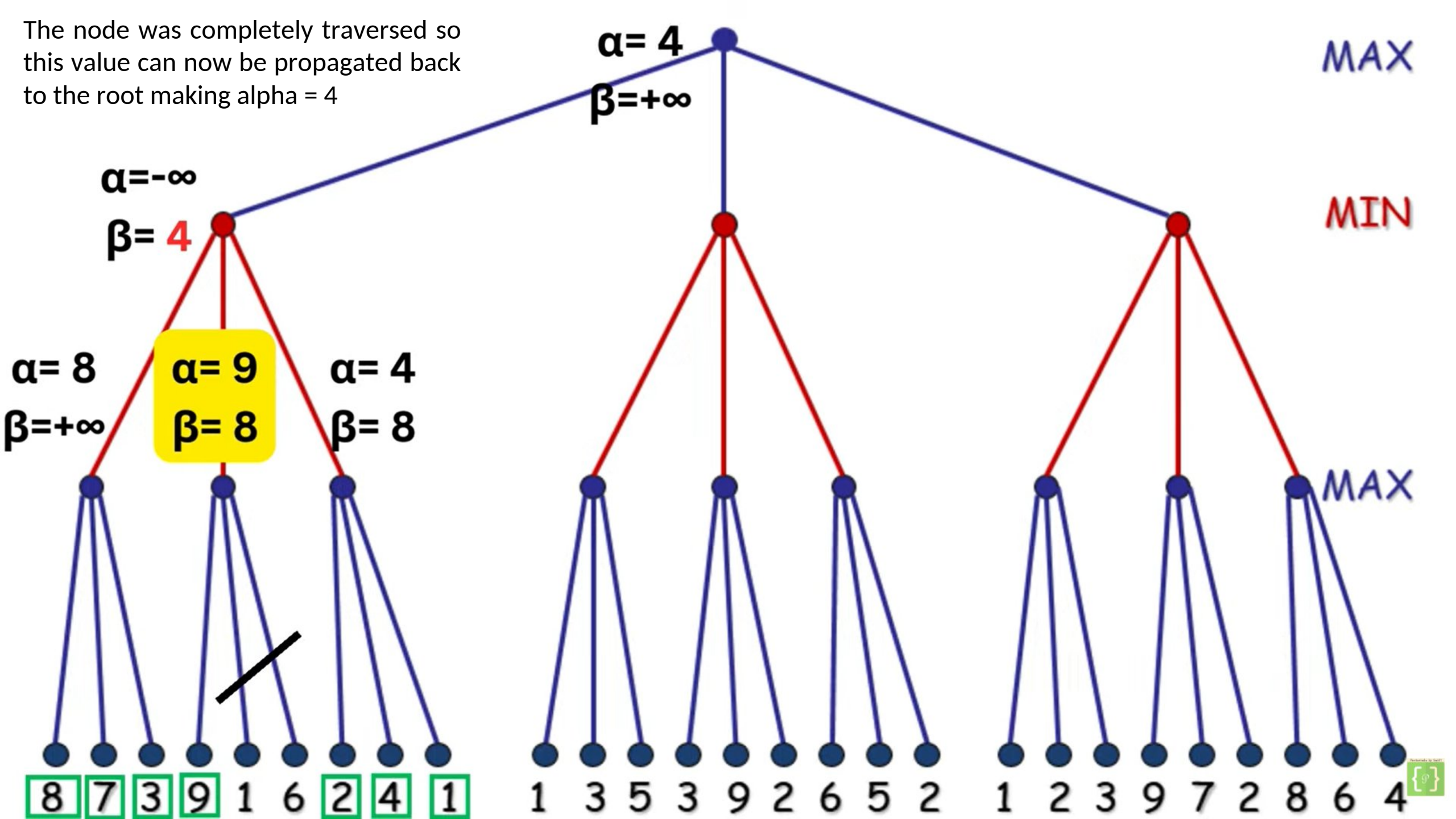
Compare 1 with alpha's current value which is 4, and 1 is lesser than that so alpha remains unchanged

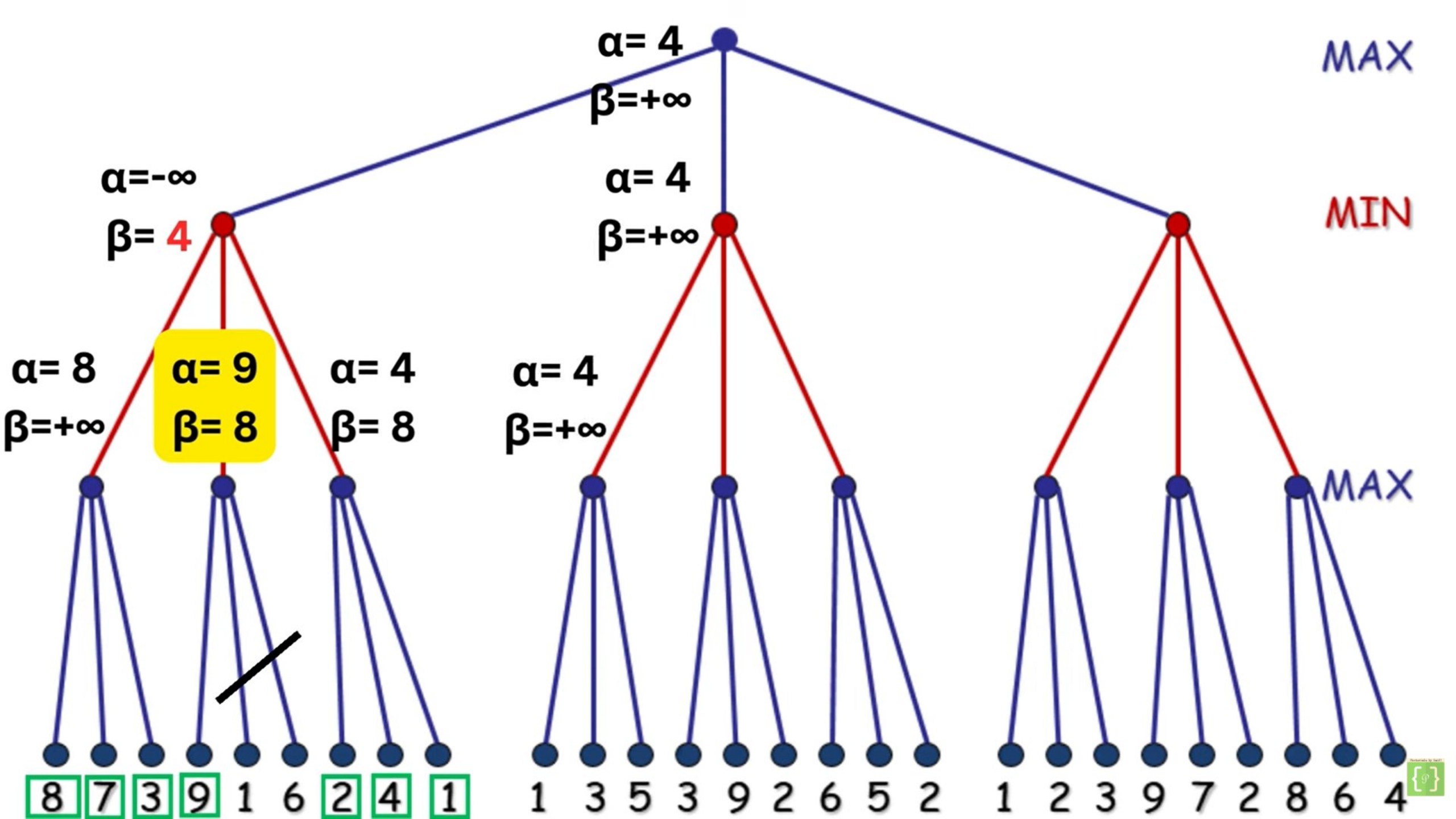


Compare 4 with beta's current value which is 8, and 4 is lesser than that so beta is updated with 4

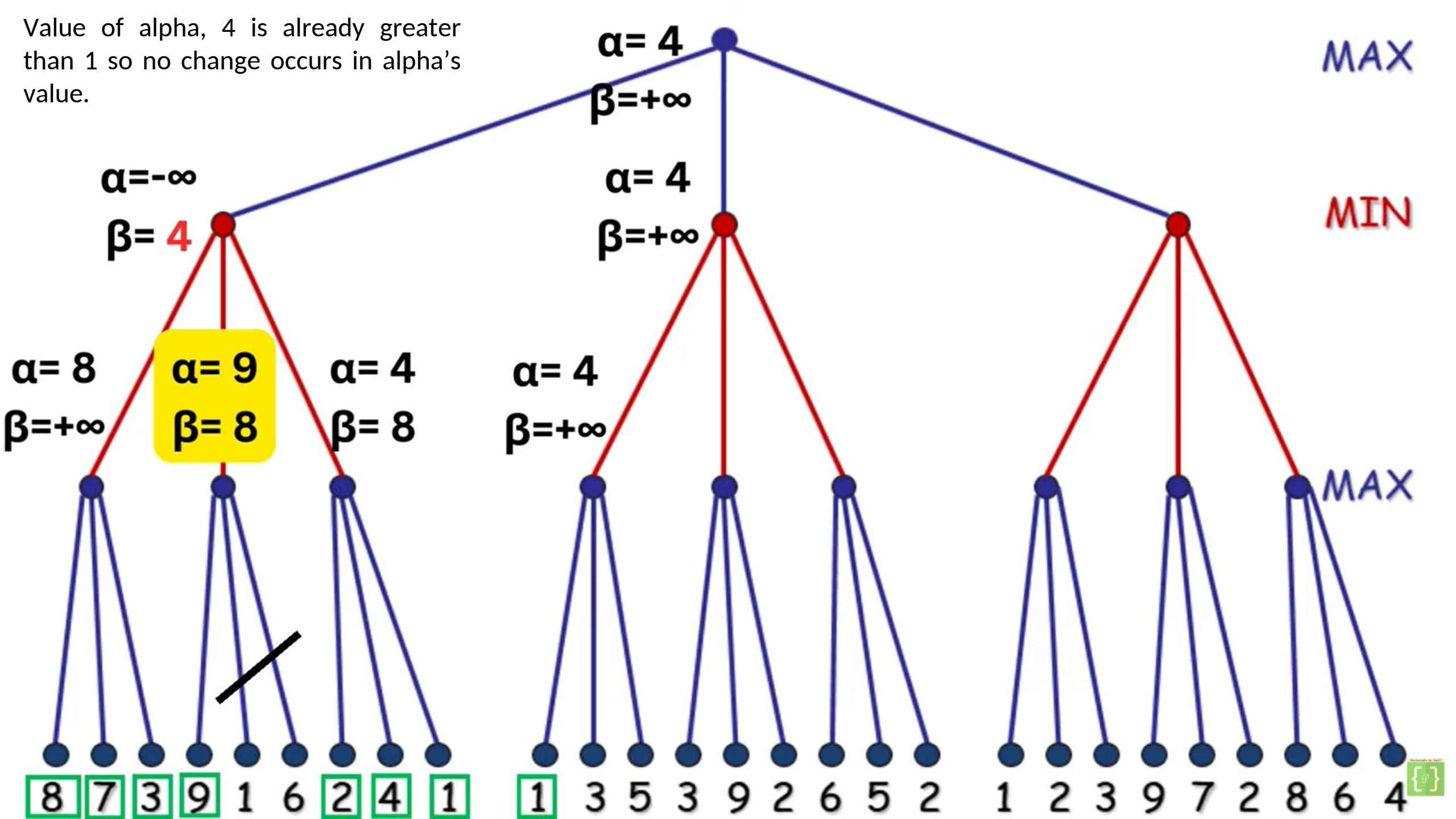


The node was completely traversed so this value can now be propagated back to the root making alpha = 4

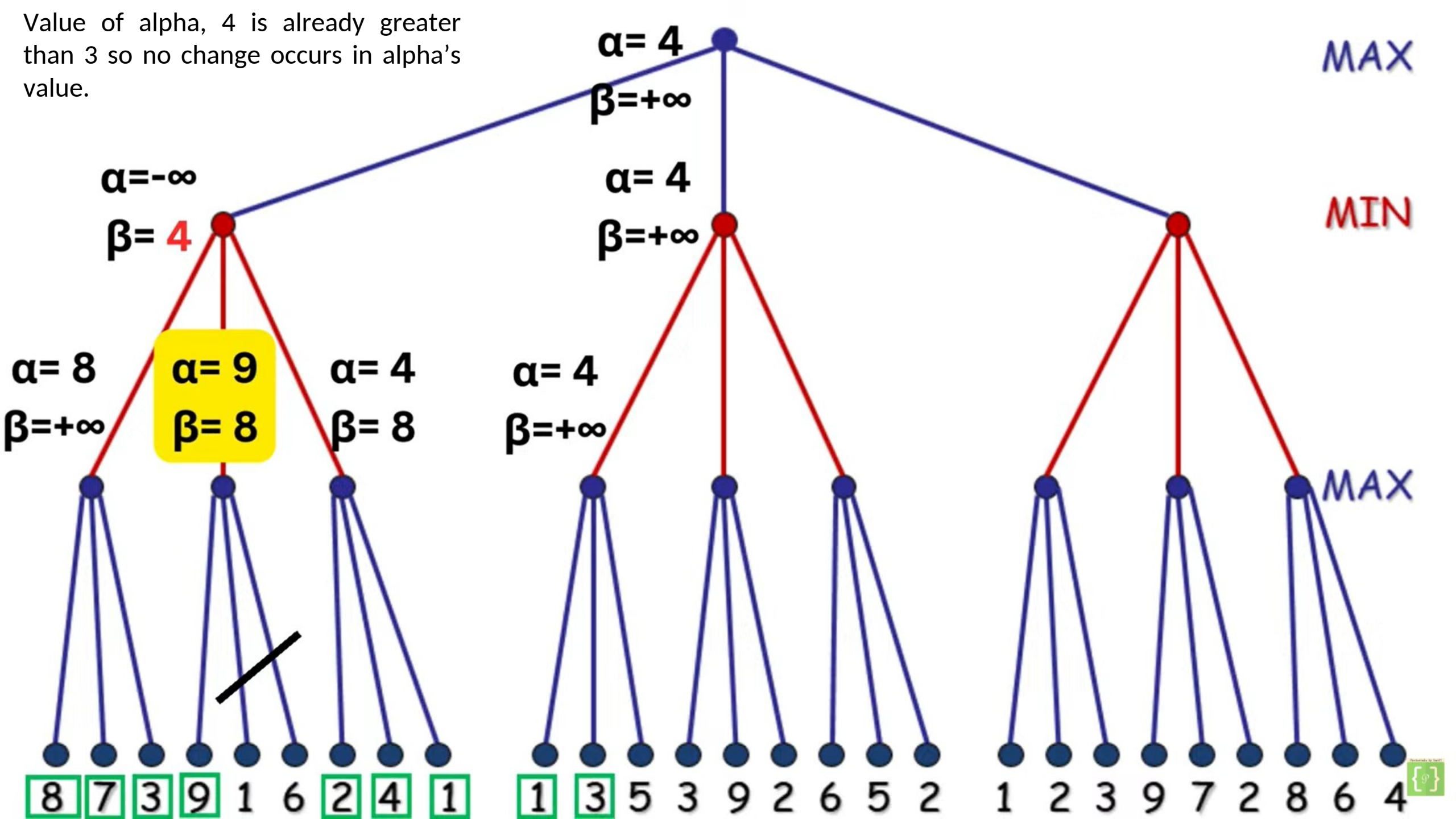



α= 4
β=+∞
MAX
α=-∞
β= 4
α= 4
β=+∞
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 4
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

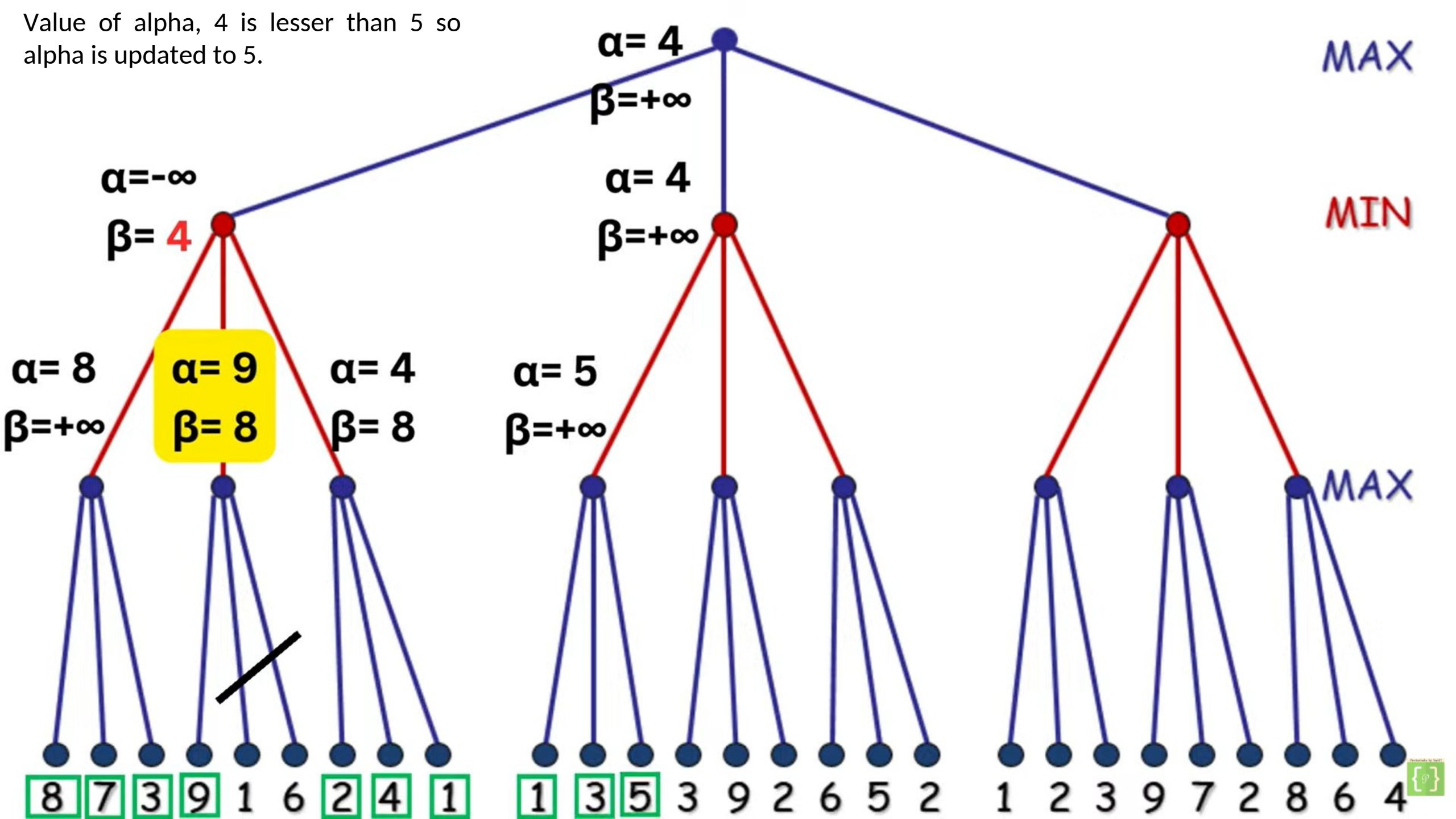
Value of alpha, 4 is already greater than 1 so no change occurs in alpha's value.



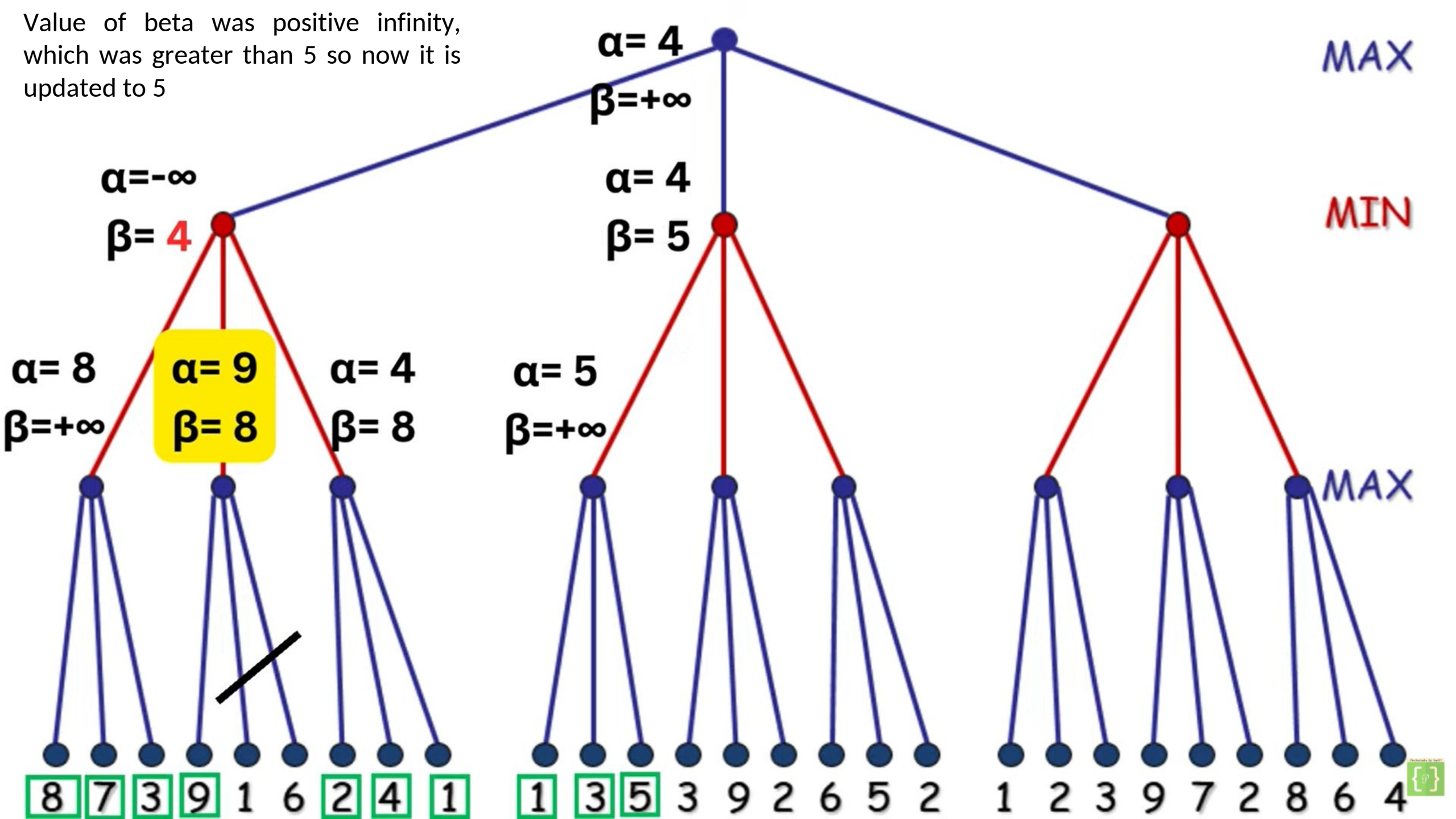
Value of alpha, 4 is already greater than 3 so no change occurs in alpha's value.

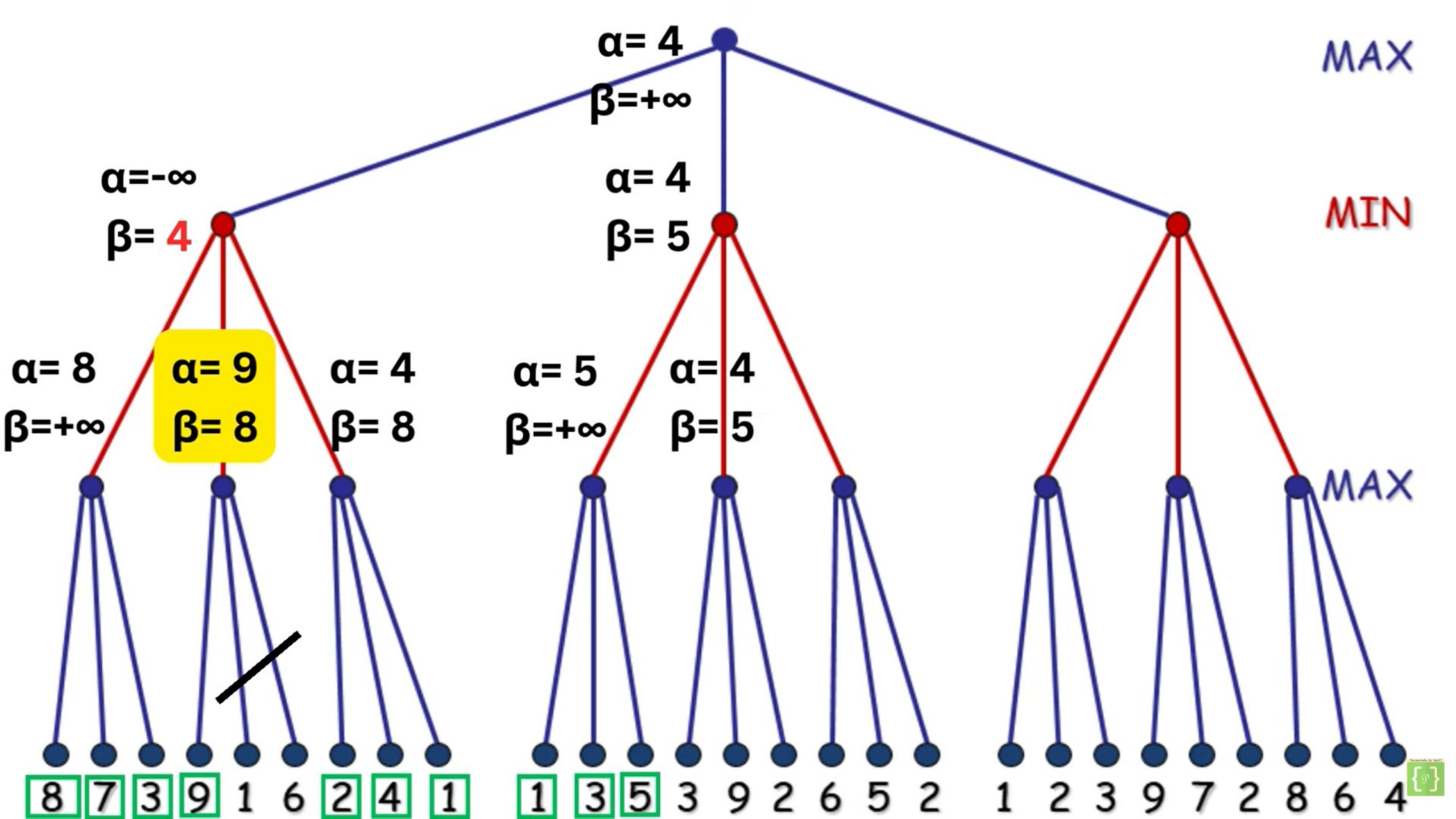


Value of alpha, 4 is lesser than 5 so alpha is updated to 5.

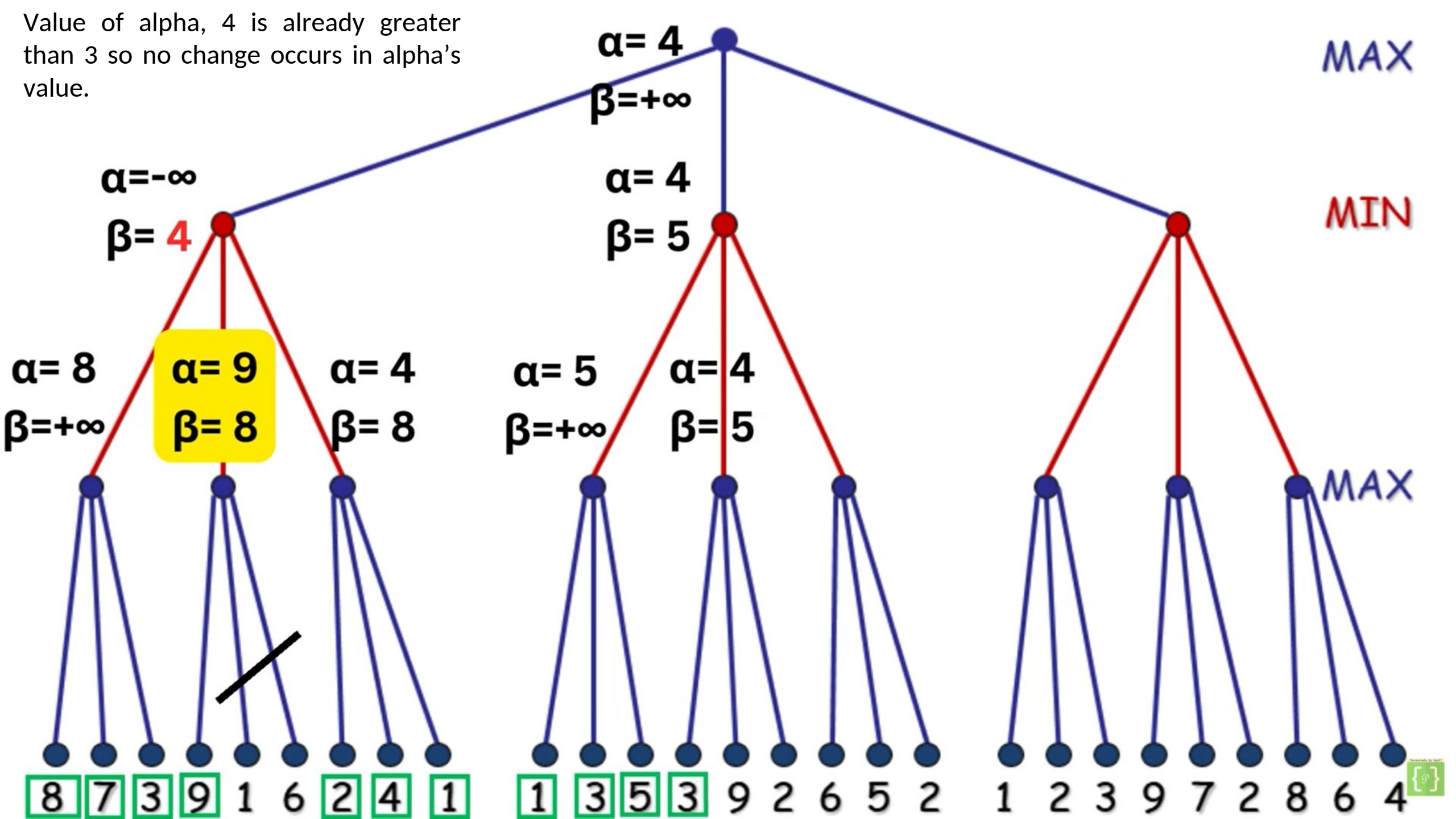


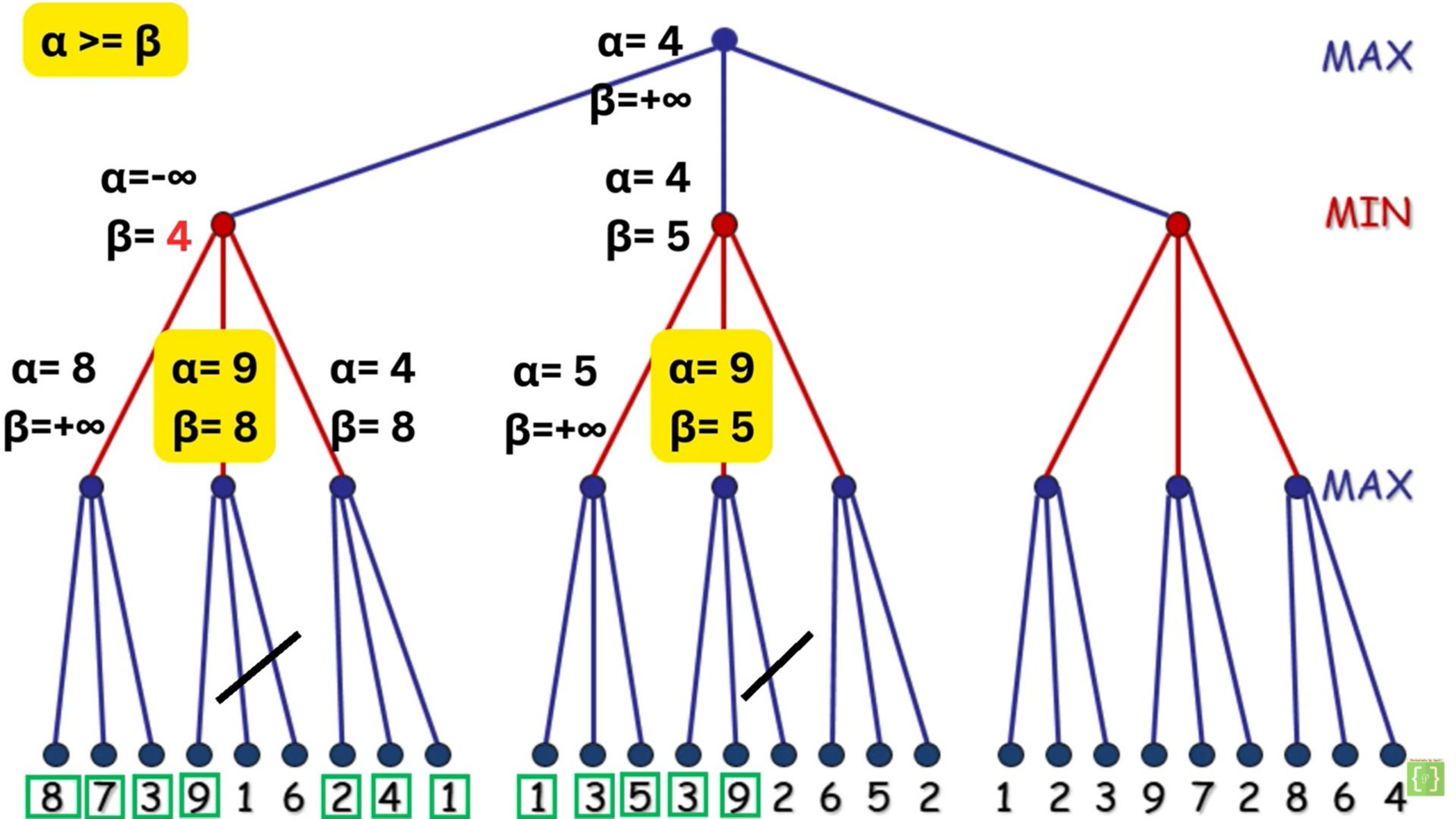
Value of beta was positive infinity, which was greater than 5 so now it is updated to 5

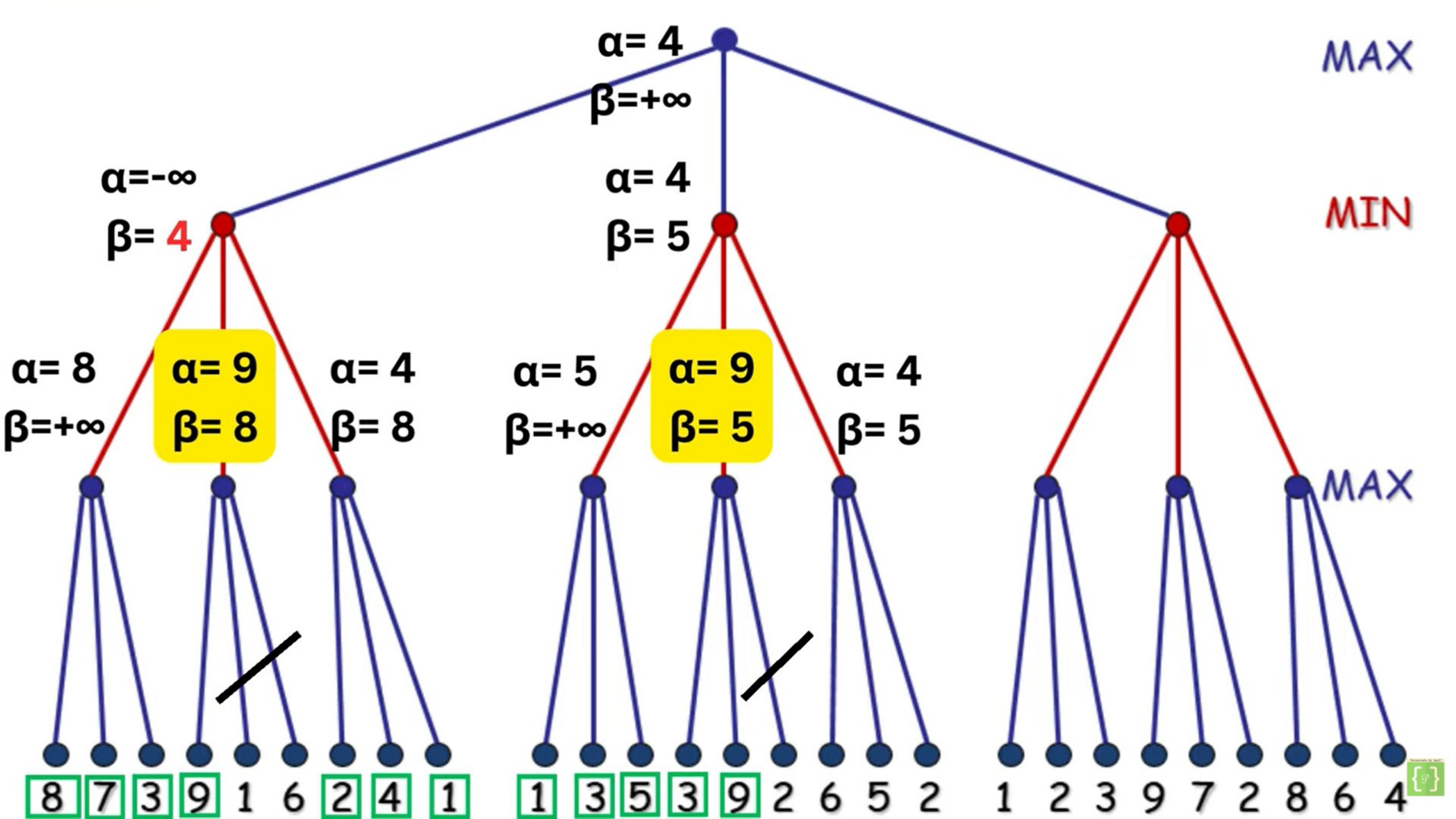



α= 4
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 4
β= 5
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

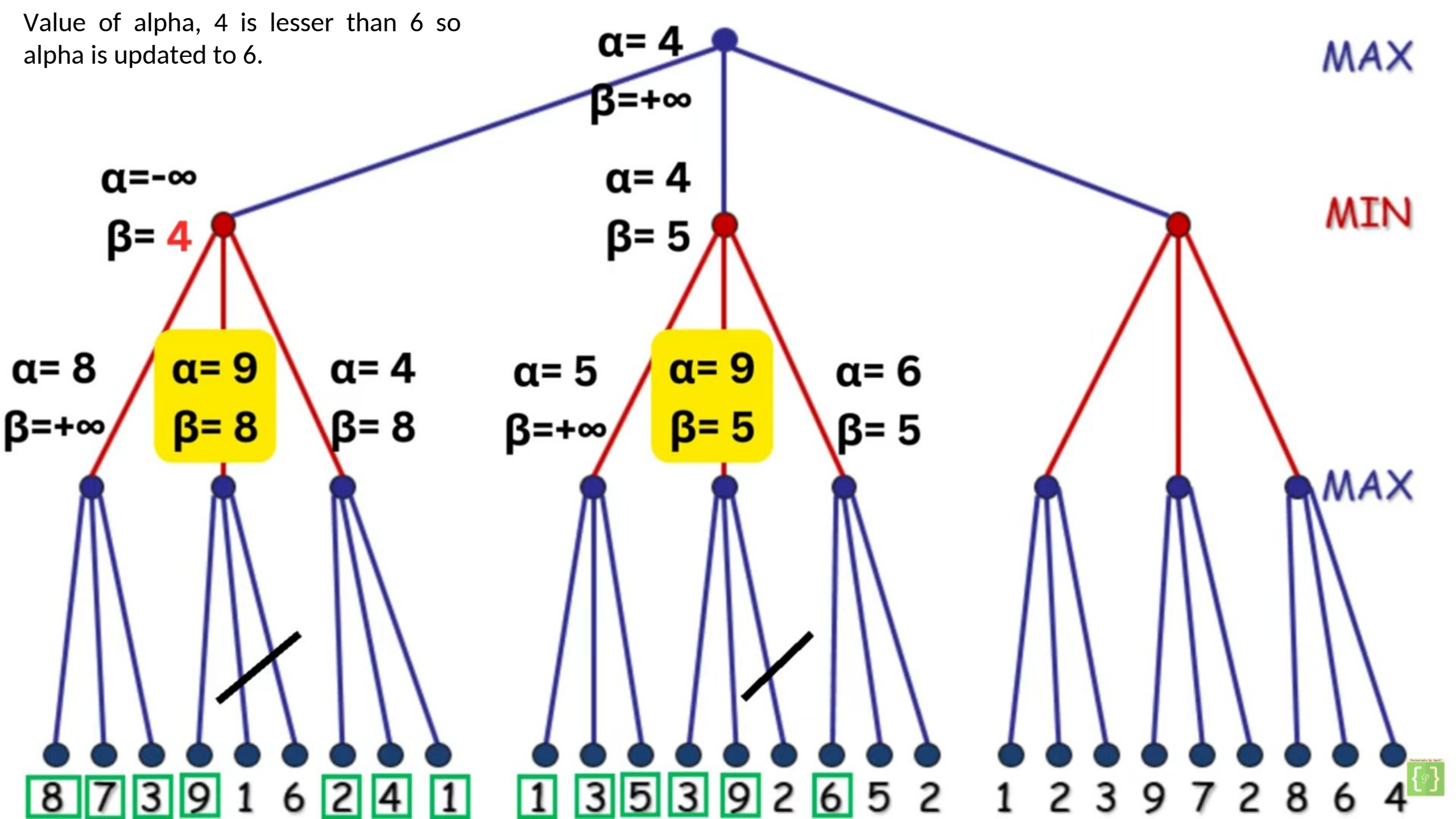
Value of alpha, 4 is already greater than 3 so no change occurs in alpha's value.

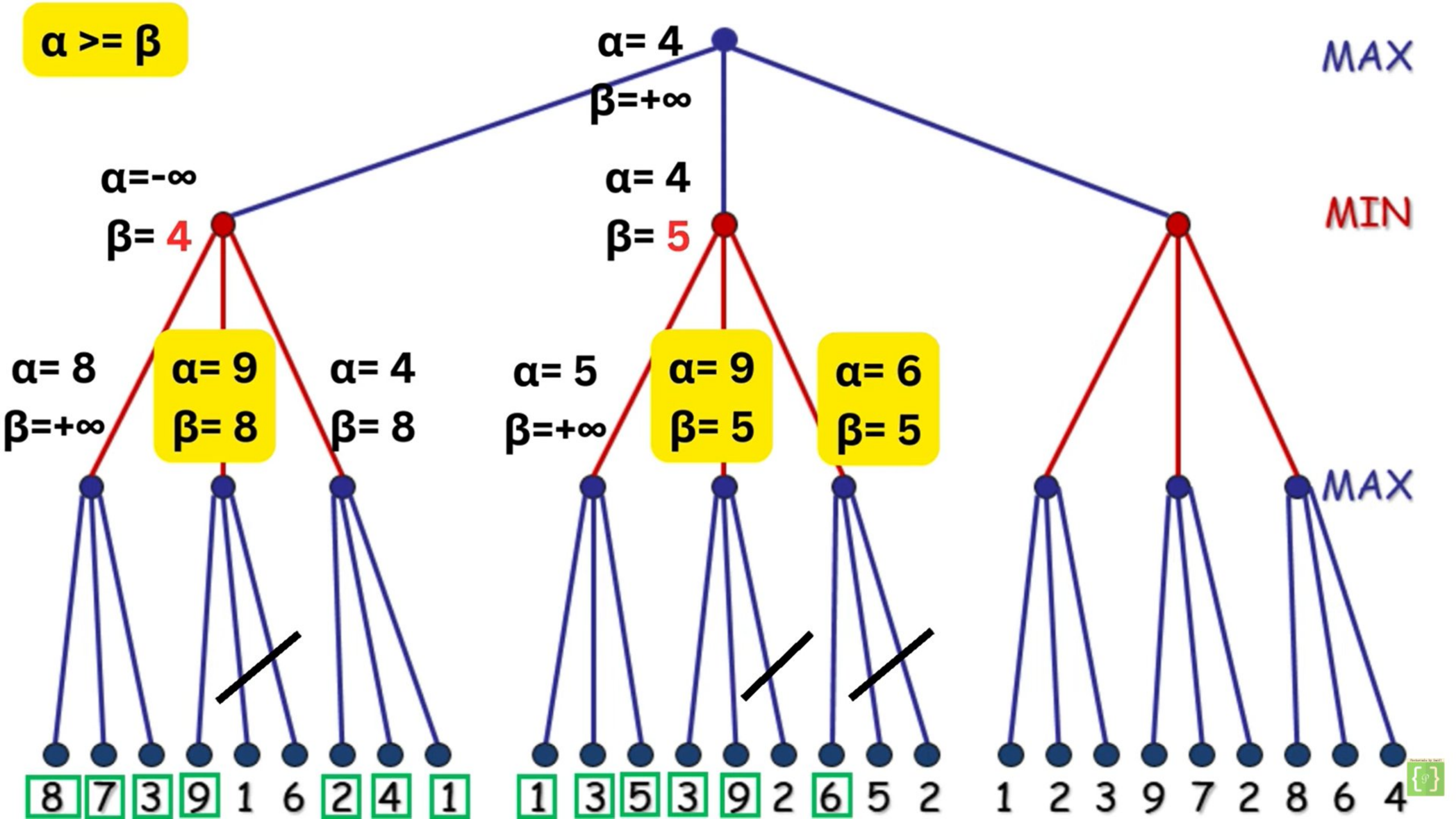


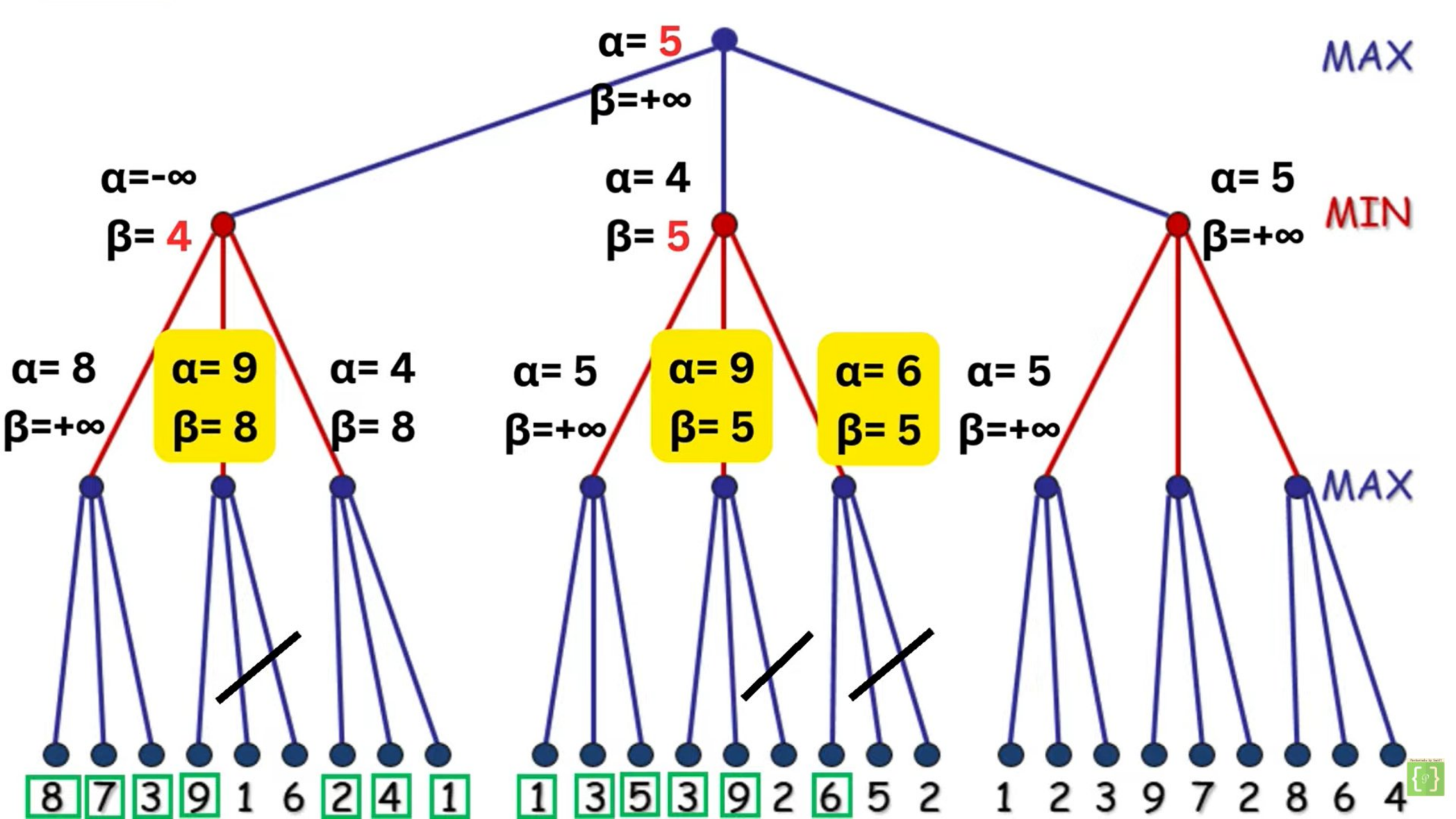

α >= β
α= 4
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 9
β= 5
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4


α= 4
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 9
β= 5
α= 4
β= 5
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

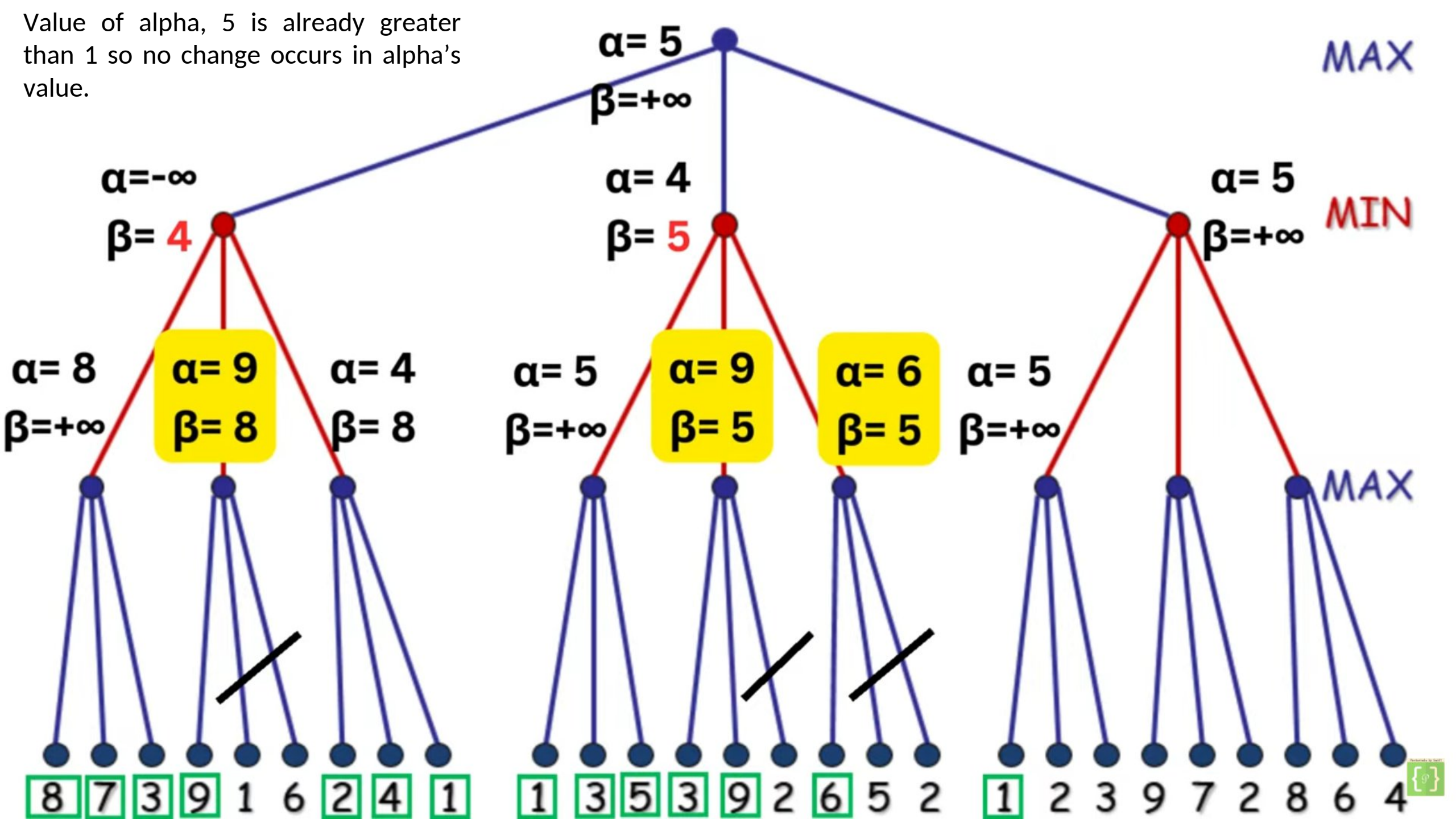
Value of alpha, 4 is lesser than 6 so alpha is updated to 6.



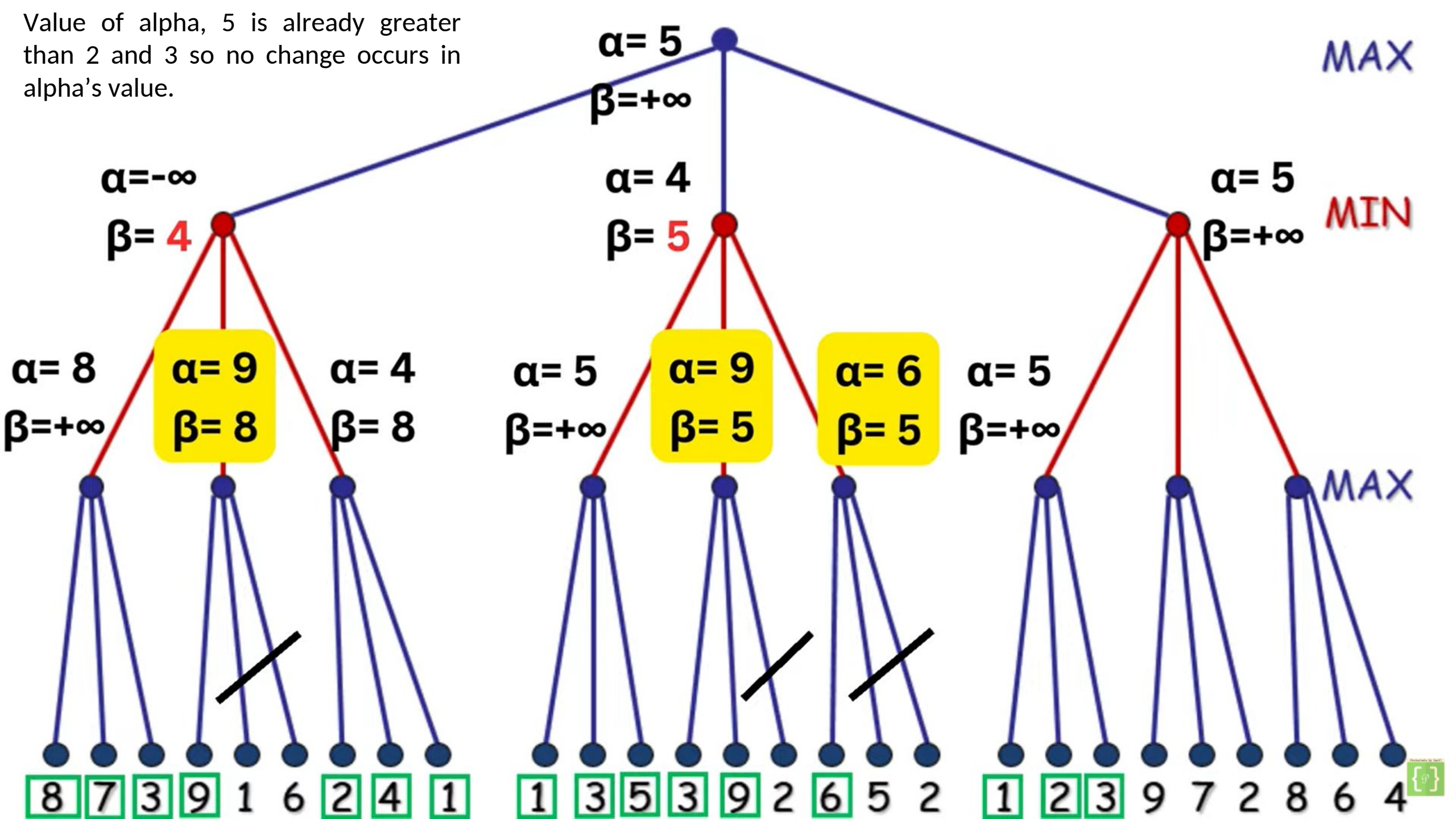

α >= β
α= 4
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 9
β= 5
α= 6
β= 5
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

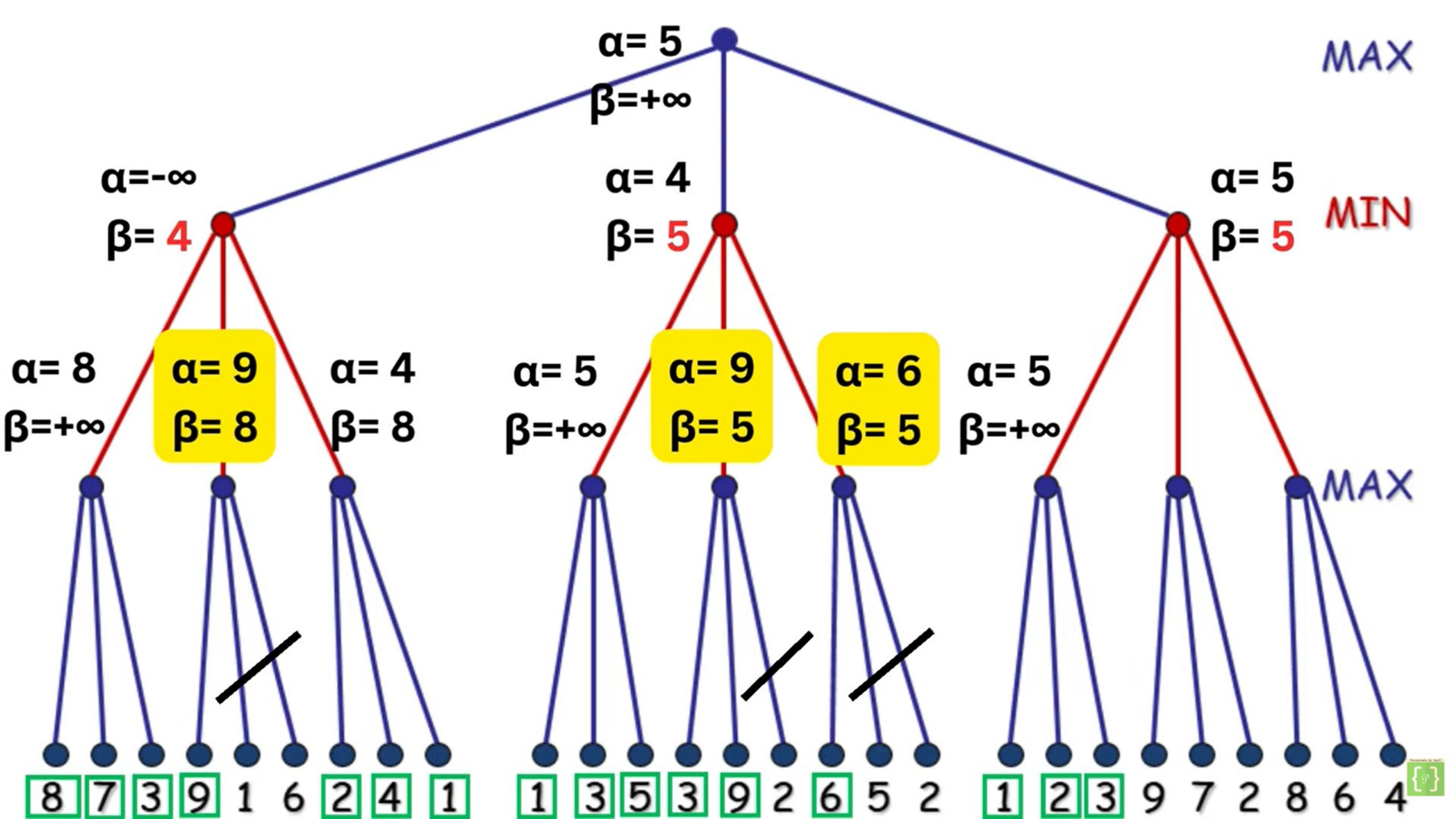

α= 5
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
α= 5
β=+∞
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 9
β= 5
α= 6
β= 5
α= 5
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4

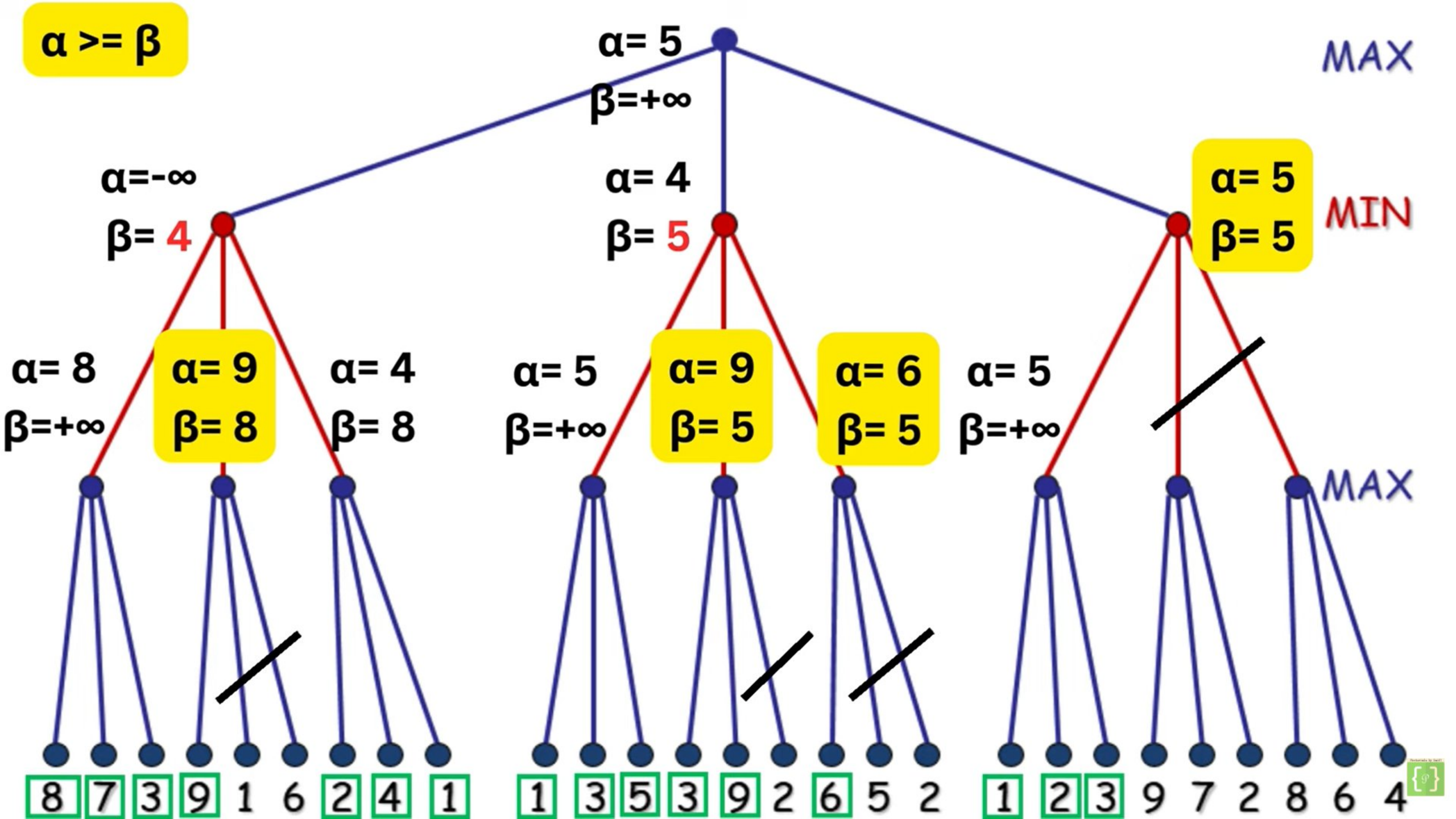
Value of alpha, 5 is already greater than 1 so no change occurs in alpha's value.



Value of alpha, 5 is already greater than 2 and 3 so no change occurs in alpha's value.




α= 5
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
α= 5
β= 5
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 9
β= 5
α= 6
β= 5
α= 5
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4


α >= β
α= 5
β=+∞
MAX
α=-∞
β= 4
α= 4
β= 5
α= 5
β= 5
MIN
α= 8
β=+∞
α= 9
β= 8
α= 4
β= 8
α= 5
β=+∞
α= 9
β= 5
α= 6
β= 5
α= 5
β=+∞
MAX
8 7 3 9 1 6 2 4 1
1 3 5 3 9 2 6 5 2
1 2 3 9 7 2 8 6 4